

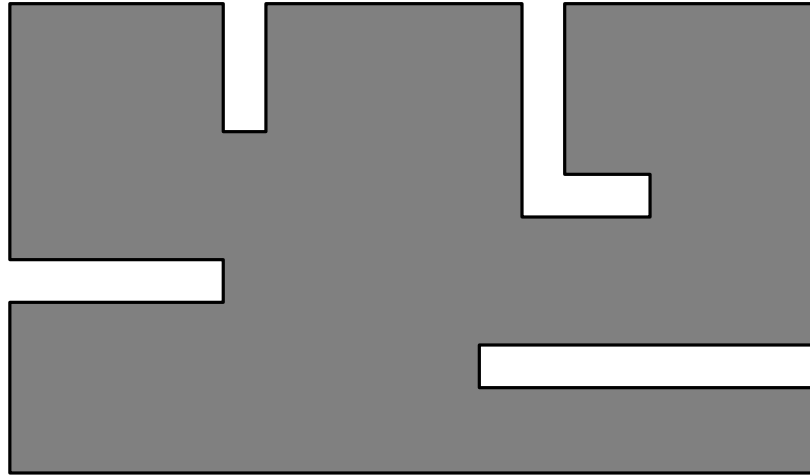
Polygon Triangulation & Art-Gallery Theorem

Sujoy Bhore

Indian Institute of Technology Bombay

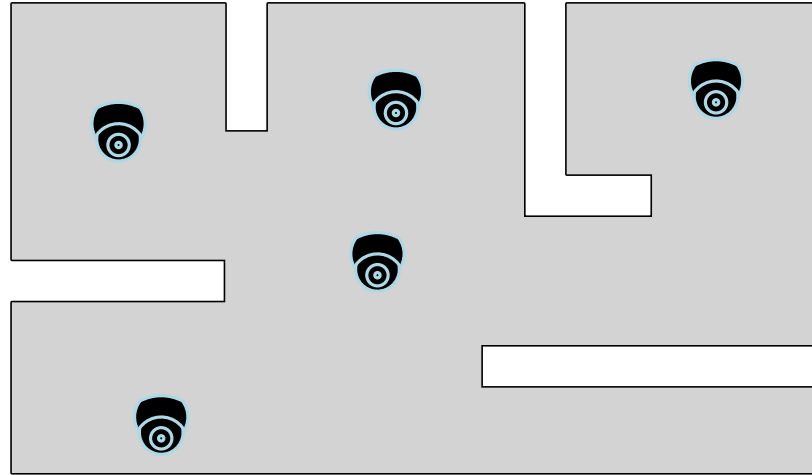
The Art-Gallery-Problem

Task: Install a number of cameras in an art gallery so that every point of the gallery is visible to at least one camera.



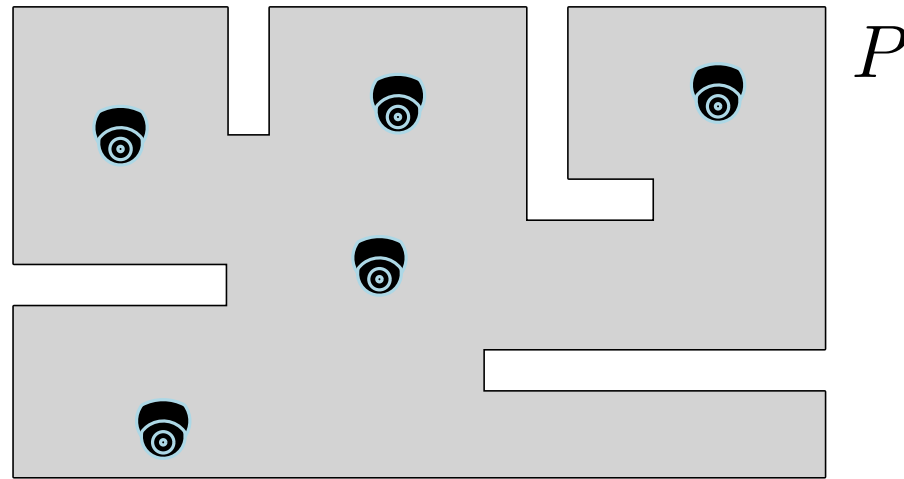
The Art-Gallery-Problem

Task: Install a number of cameras in an art gallery so that every point of the gallery is visible to at least one camera.



The Art-Gallery-Problem

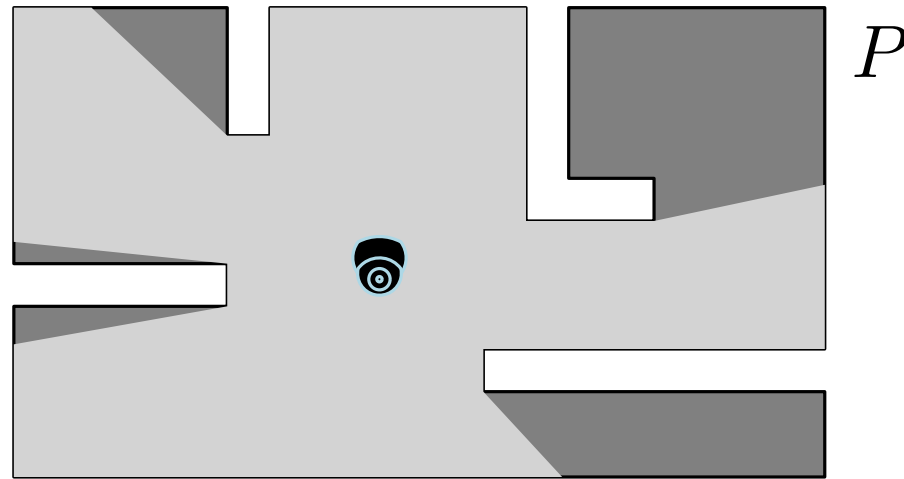
Task: Install a number of cameras in an art gallery so that every point of the gallery is visible to at least one camera.



Assumption: art gallery is a *simple* polygon P with n corners
(no self-intersections, no holes)

The Art-Gallery-Problem

Task: Install a number of cameras in an art gallery so that every point of the gallery is visible to at least one camera.

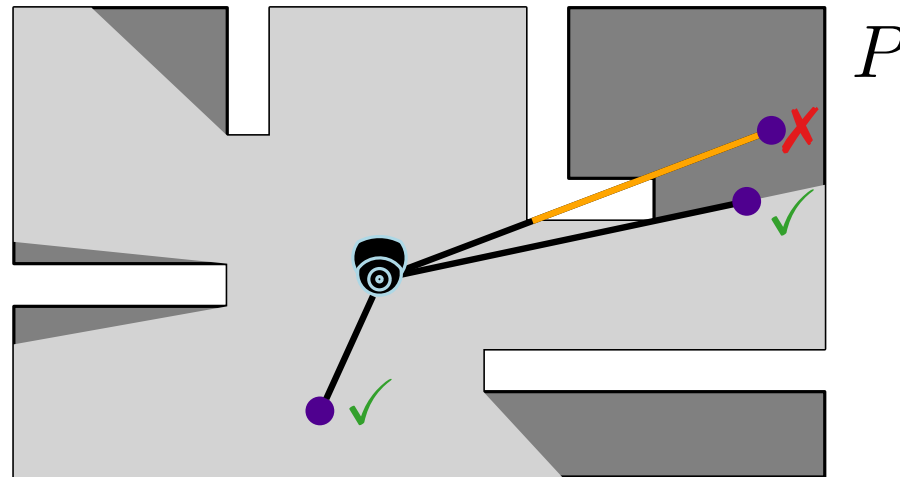


Assumption: art gallery is a *simple* polygon P with n corners
(no self-intersections, no holes)

Observation: each camera observes a star-shaped region

The Art-Gallery-Problem

Task: Install a number of cameras in an art gallery so that every point of the gallery is visible to at least one camera.



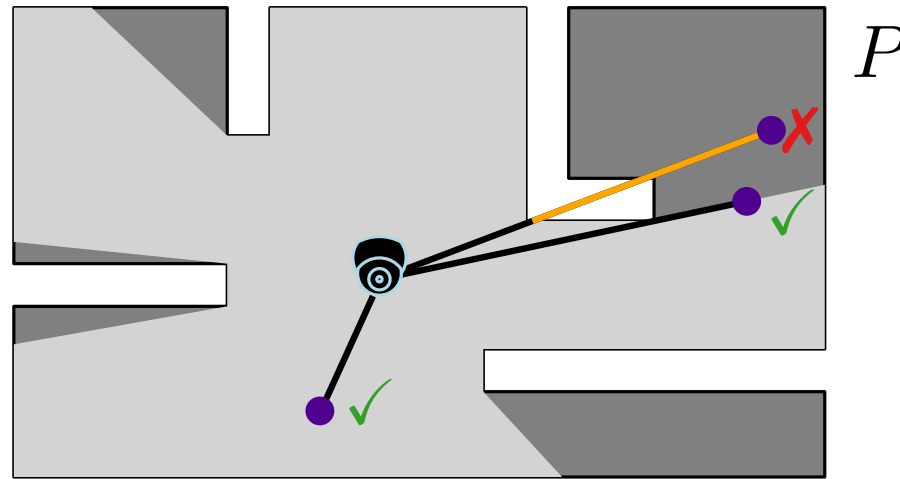
Assumption: art gallery is a *simple* polygon P with n corners (no self-intersections, no holes)

Observation: each camera observes a star-shaped region

Definition: point $p \in P$ is *visible* from $c \in P$ if $\overline{cp} \in P$

The Art-Gallery-Problem

Task: Install a number of cameras in an art gallery so that every point of the gallery is visible to at least one camera.



Assumption: art gallery is a *simple* polygon P with n corners (no self-intersections, no holes)

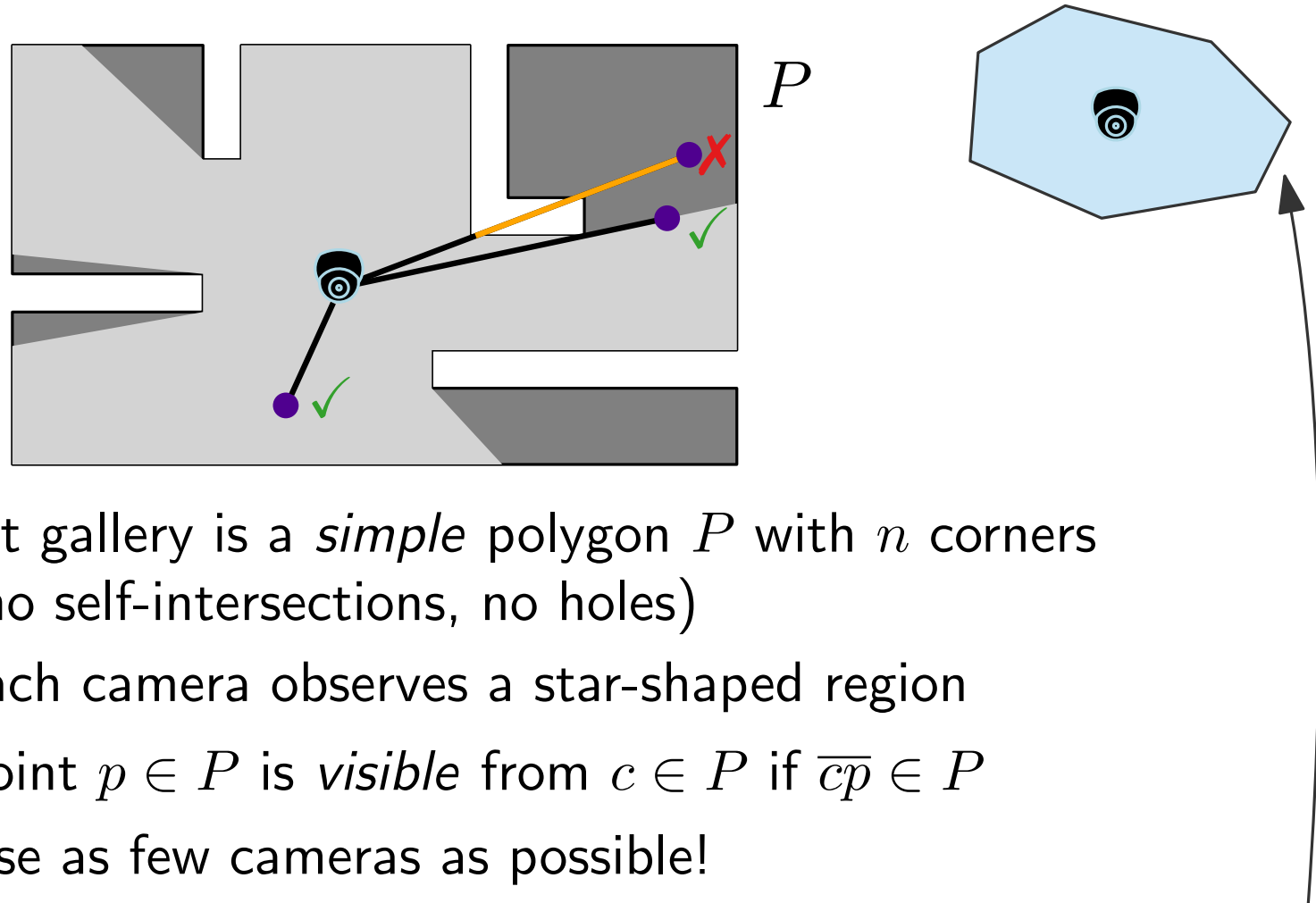
Observation: each camera observes a star-shaped region

Definition: point $p \in P$ is *visible* from $c \in P$ if $\overline{cp} \in P$

Goal: Use as few cameras as possible!

The Art-Gallery-Problem

Task: Install a number of cameras in an art gallery so that every point of the gallery is visible to at least one camera.



Assumption: art gallery is a *simple* polygon P with n corners (no self-intersections, no holes)

Observation: each camera observes a star-shaped region

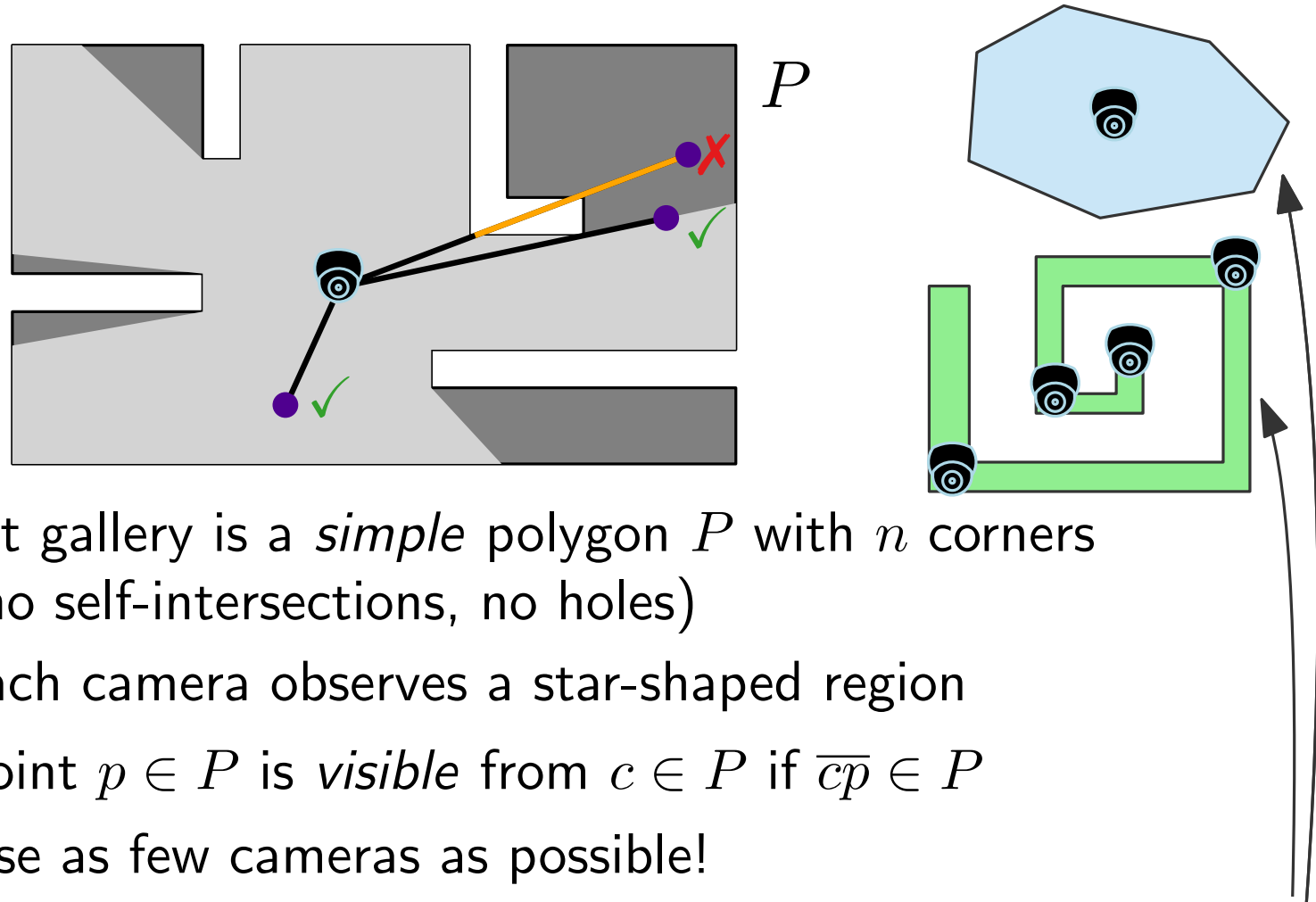
Definition: point $p \in P$ is *visible* from $c \in P$ if $\overline{cp} \in P$

Goal: Use as few cameras as possible!

→ The number depends on the number of corners n and on the shape of P

The Art-Gallery-Problem

Task: Install a number of cameras in an art gallery so that every point of the gallery is visible to at least one camera.



Assumption: art gallery is a *simple* polygon P with n corners
(no self-intersections, no holes)

Observation: each camera observes a star-shaped region

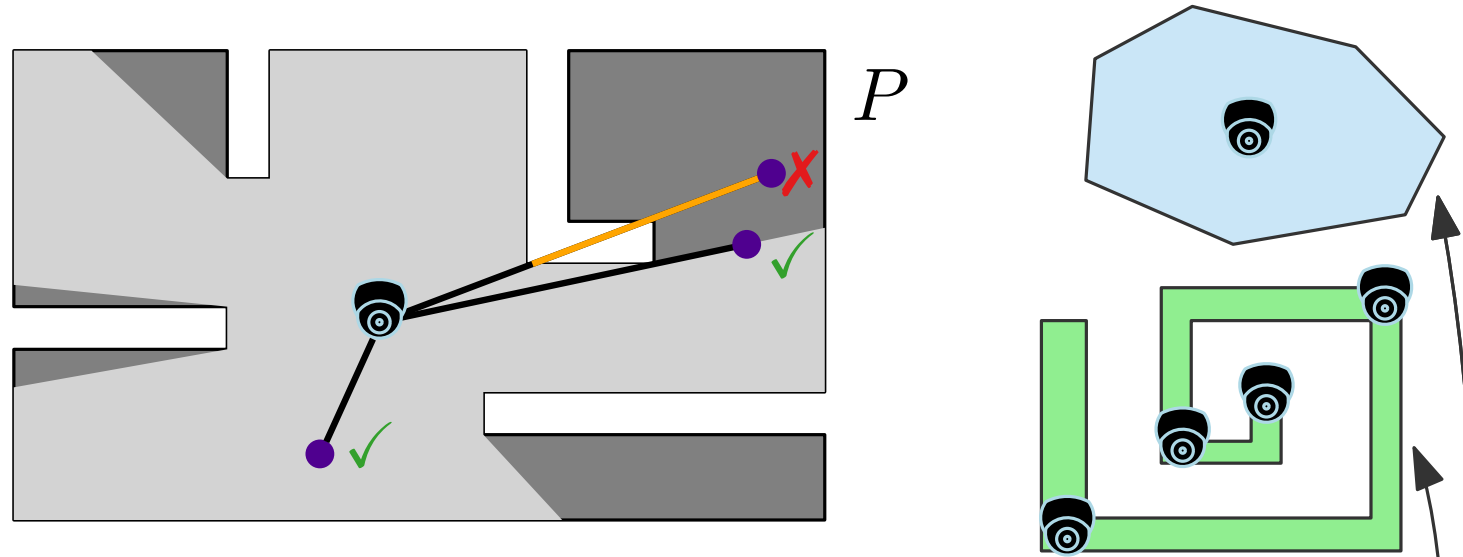
Definition: point $p \in P$ is *visible* from $c \in P$ if $\overline{cp} \in P$

Goal: Use as few cameras as possible!

→ The number depends on the number of corners n and on the shape of P

The Art-Gallery-Problem

Task: Install a number of cameras in an art gallery so that every point of the gallery is visible to at least one camera.



Assumption: art gallery is a *simple* polygon P with n corners
(no self-intersections, no holes)

Observation: each camera observes a star-shaped region

Definition: point $p \in P$ is *visible* from $c \in P$ if $\overline{cp} \in P$

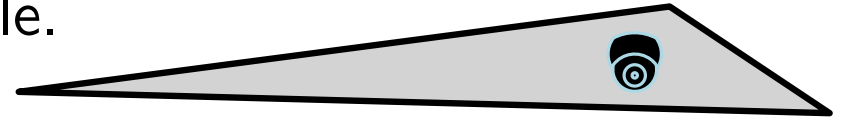
Goal: Use as few cameras as possible!

NP-hard!

→ The number depends on the number of corners n and on the shape of P

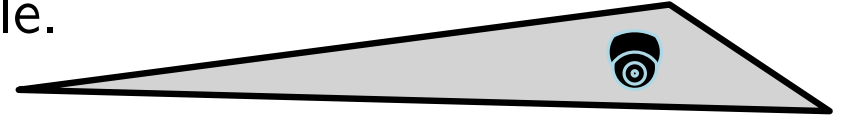
Problem Simplification.

Observation: It is easy to guard a triangle.

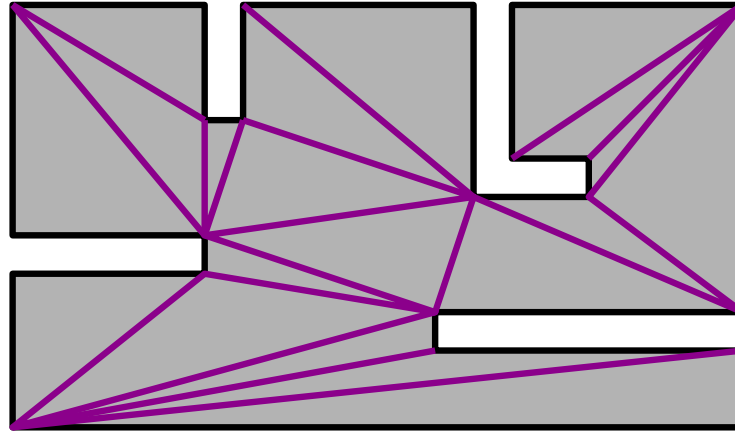


Problem Simplification.

Observation: It is easy to guard a triangle.

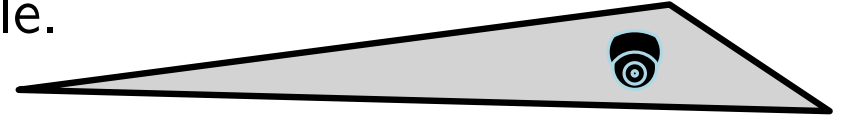


Idea: Decompose P into triangles and guard each of them.

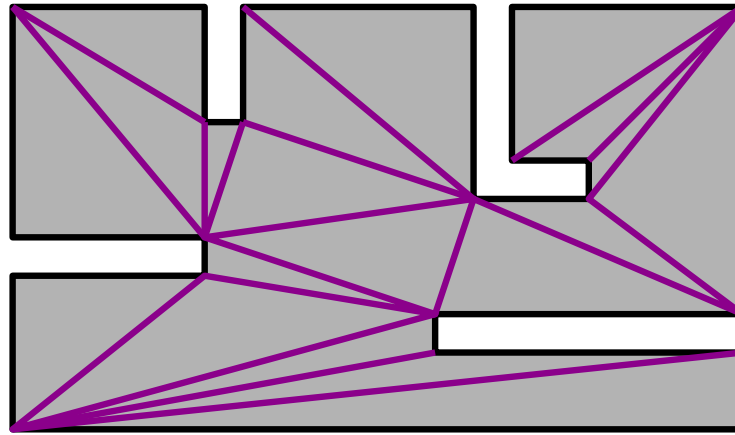


Problem Simplification.

Observation: It is easy to guard a triangle.



Idea: Decompose P into triangles and guard each of them.



Theorem 1. Every simple polygon with n corners admits a triangulation; any such triangulation contains exactly $n - 2$ triangles.

Problem Simplification.

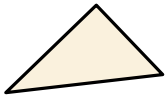
Theorem 1. Every simple polygon with n corners admits a triangulation; any such triangulation contains exactly $n - 2$ triangles.

Proof: by induction on n

Problem Simplification.

Theorem 1. Every simple polygon with n corners admits a triangulation; any such triangulation contains exactly $n - 2$ triangles.

Proof: by induction on n

■ $n = 3$ 

■ $n > 3$ – we show there is a diagonal of P

Problem Simplification.

Theorem 1. Every simple polygon with n corners admits a triangulation; any such triangulation contains exactly $n - 2$ triangles.

Proof: by induction on n



■ $n > 3$ – we show there is a diagonal of P

Let v be the leftmost point, and u, w be its neighbors

– Case 1: $\overline{uw}$ is inside P , then $\overline{uw}$ is the diagonal.

– Case 2: there is a point in the triangle vuw .

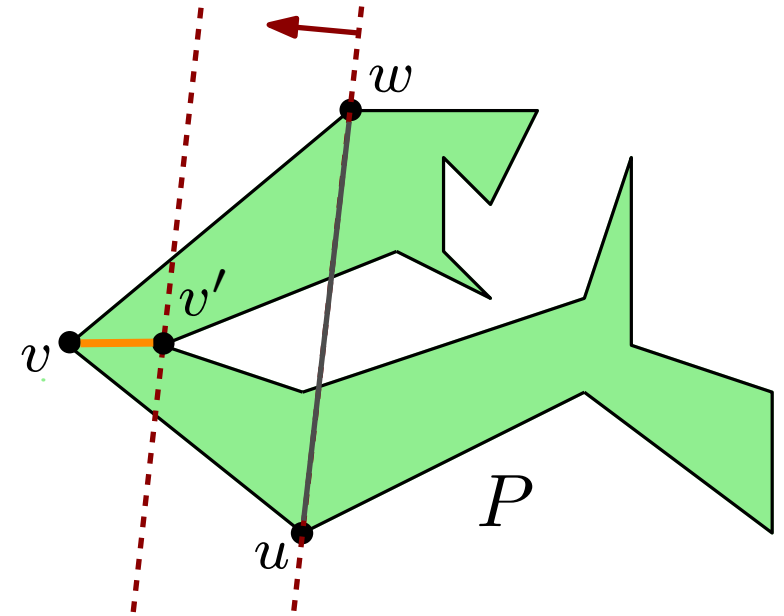
take the furthest one from $\overline{uw}$ – v'

vv' is a valid diagonal.

vv' splits P into P_1, P_2 , with $n_1 < n$ and $n_2 < n$ corners, respectively.

P_1, P_2 have triangulations with $n_1 - 2$ and $n_2 - 2$ triangles, respectively. $n = n_1 + n_2 - 2$

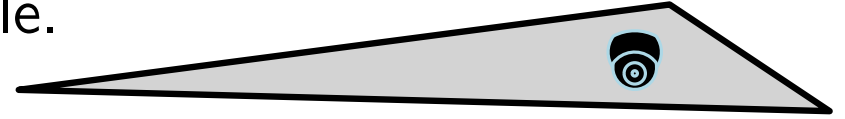
So, # of triangles of P , $n_1 + n_2 - 4 = n - 2$.



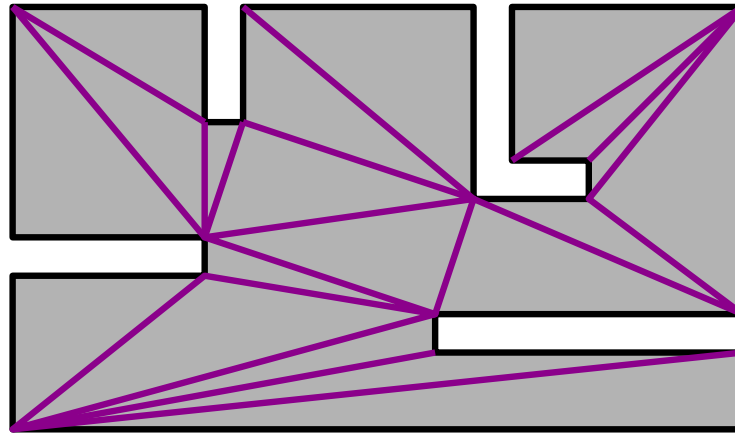
Done!

Problem Simplification.

Observation: It is easy to guard a triangle.



Idea: Decompose P into triangles and guard each of them.

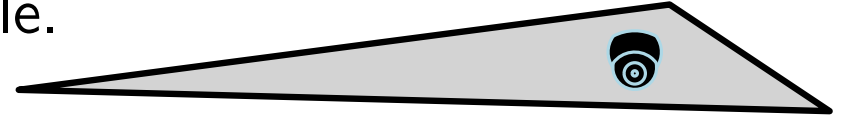


Theorem 1. Every simple polygon with n corners admits a triangulation; any such triangulation contains exactly $n - 2$ triangles.

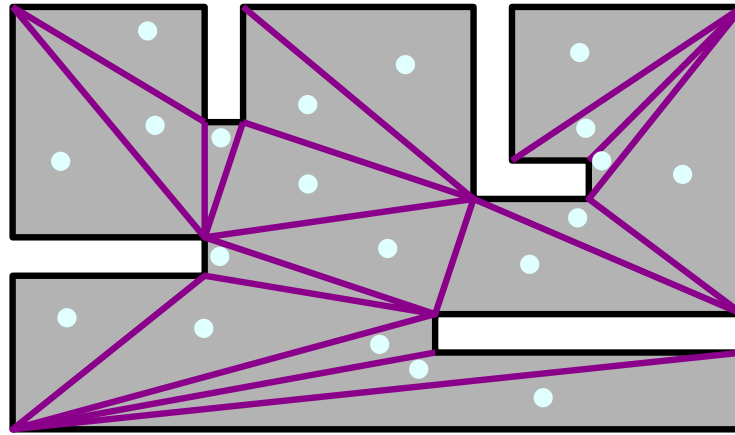
The proof implies a recursive $O(n^2)$ -algorithm!

Problem Simplification.

Observation: It is easy to guard a triangle.



Idea: Decompose P into triangles and guard each of them.



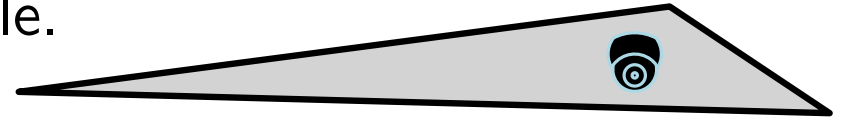
Theorem 1. Every simple polygon with n corners admits a triangulation; any such triangulation contains exactly $n - 2$ triangles.

The proof implies a recursive $O(n^2)$ -algorithm!

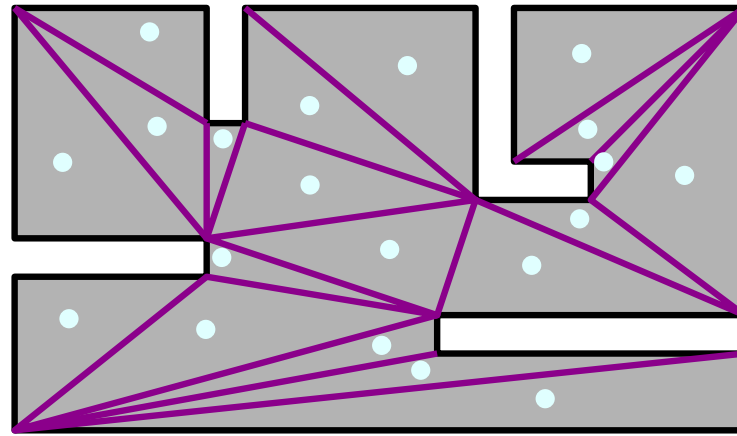
■ P could be guarded by $n - 2$ cameras placed in the $n - 2$ triangles

Problem Simplification.

Observation: It is easy to guard a triangle.



Idea: Decompose P into triangles and guard each of them.



Theorem 1. Every simple polygon with n corners admits a triangulation; any such triangulation contains exactly $n - 2$ triangles.

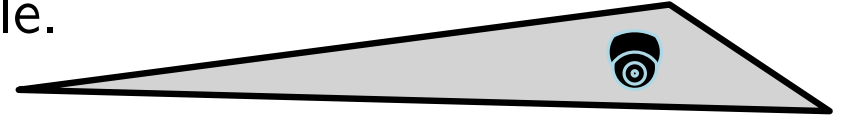
The proof implies a recursive $O(n^2)$ -algorithm!

■ P could be guarded by $n - 2$ cameras placed in the $n - 2$ triangles

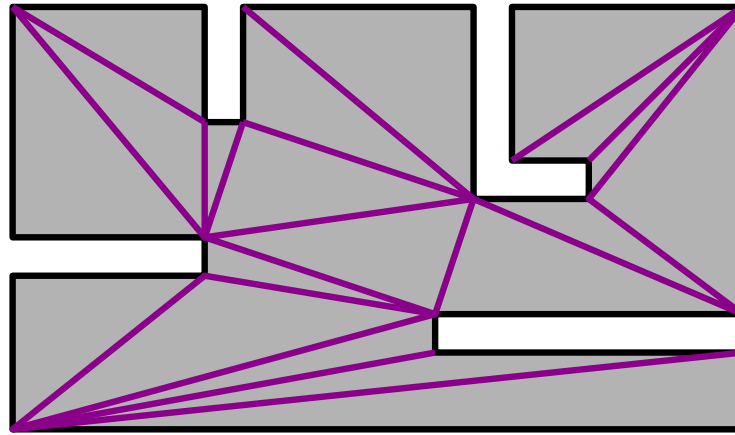
Can we do better?

Problem Simplification.

Observation: It is easy to guard a triangle.



Idea: Decompose P into triangles and guard each of them.

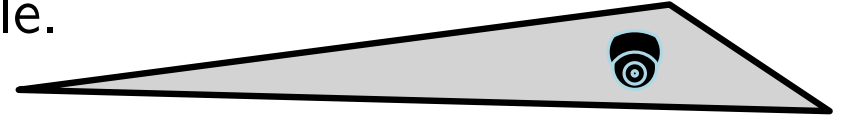


Theorem 1. Every simple polygon with n corners admits a triangulation; any such triangulation contains exactly $n - 2$ triangles.

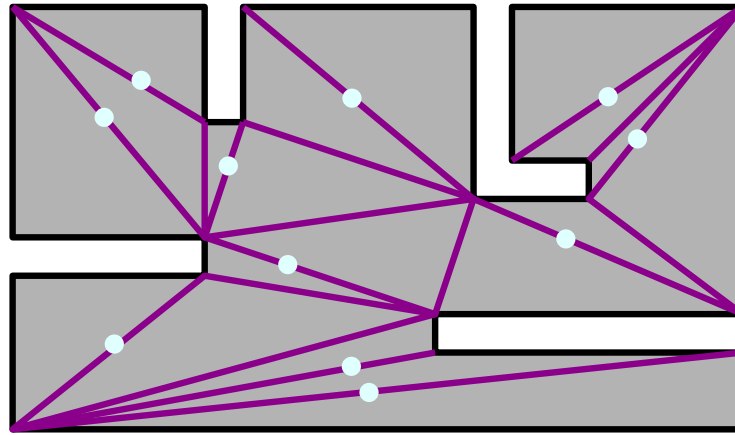
- P could be guarded by $n - 2$ cameras placed in the $n - 2$ triangles
- P can be guarded by $\approx n/2$ cameras placed on the diagonals

Problem Simplification.

Observation: It is easy to guard a triangle.



Idea: Decompose P into triangles and guard each of them.



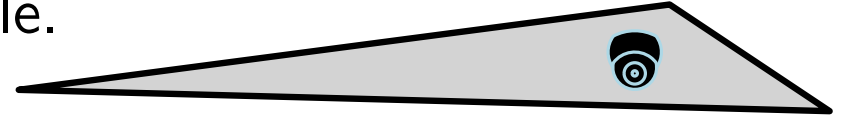
Already a bit better!

Theorem 1. Every simple polygon with n corners admits a triangulation; any such triangulation contains exactly $n - 2$ triangles.

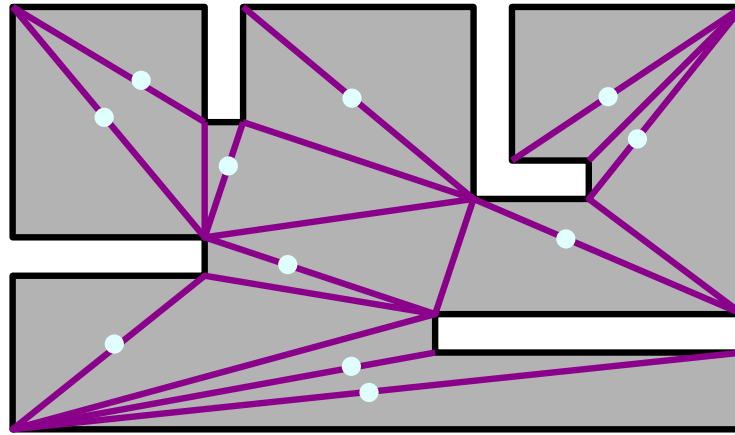
- P could be guarded by $n - 2$ cameras placed in the $n - 2$ triangles
- P can be guarded by $\approx n/2$ cameras placed on the diagonals

Problem Simplification.

Observation: It is easy to guard a triangle.



Idea: Decompose P into triangles and guard each of them.



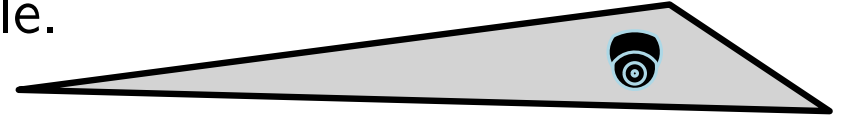
Already a bit better!

Theorem 1. Every simple polygon with n corners admits a triangulation; any such triangulation contains exactly $n - 2$ triangles.

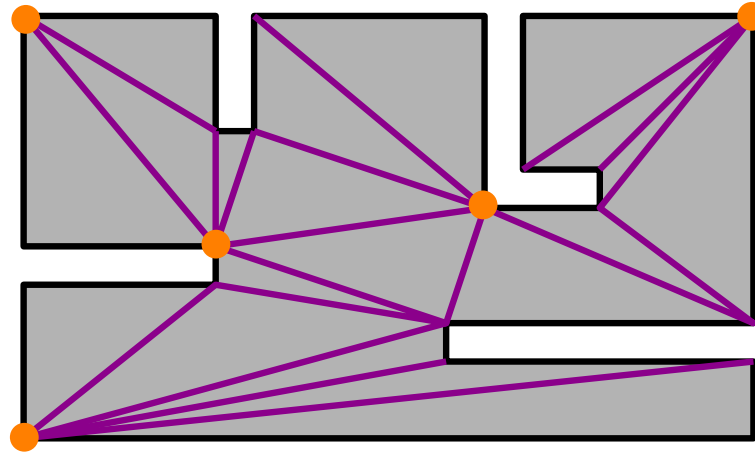
- P could be guarded by $n - 2$ cameras placed in the $n - 2$ triangles
- P can be guarded by $\approx n/2$ cameras placed on the diagonals
- P can be guarded by even fewer cameras placed on the corners

Problem Simplification.

Observation: It is easy to guard a triangle.



Idea: Decompose P into triangles and guard each of them.



Theorem 1. Every simple polygon with n corners admits a triangulation; any such triangulation contains exactly $n - 2$ triangles.

- P could be guarded by $n - 2$ cameras placed in the $n - 2$ triangles
- P can be guarded by $\approx n/2$ cameras placed on the diagonals
- P can be guarded by even fewer cameras placed on the corners

The Art-Gallery-Theorem [Chvátal '75]

Theorem 2. For a simple polygon P with n vertices, $\lfloor n/3 \rfloor$ cameras are sometimes necessary and always sufficient to guard it.

The Art-Gallery-Theorem [Chvátal '75]

Theorem 2. For a simple polygon P with n vertices, $\lfloor n/3 \rfloor$ cameras are sometimes necessary and always sufficient to guard it.

Proof:

- Necessity: Find a simple polygon with n corners that requires $\approx n/3$ cameras!

The Art-Gallery-Theorem [Chvátal '75]

Theorem 2. For a simple polygon P with n vertices, $\lfloor n/3 \rfloor$ cameras are sometimes necessary and always sufficient to guard it.

Proof:

- Necessity: Find a simple polygon with n corners that requires $\approx n/3$ cameras!



The Art-Gallery-Theorem [Chvátal '75]

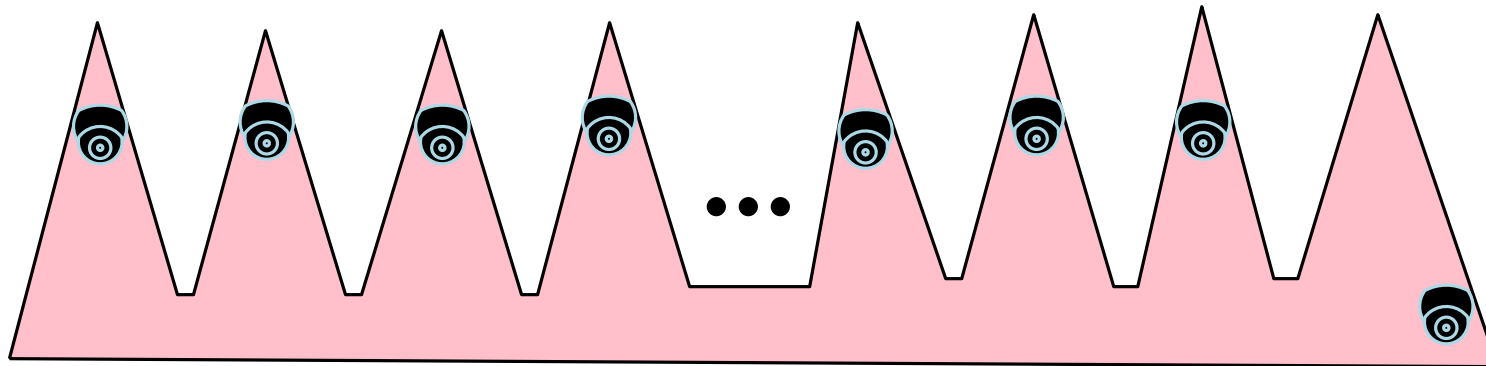
Theorem 2. For a simple polygon P with n vertices, $\lfloor n/3 \rfloor$ cameras are sometimes necessary and always sufficient to guard it.

Proof:

- Necessity: Find a simple polygon with n corners that requires $\approx n/3$ cameras!



$\lfloor n/3 \rfloor$ prongs



The Art-Gallery-Theorem [Chvátal '75]

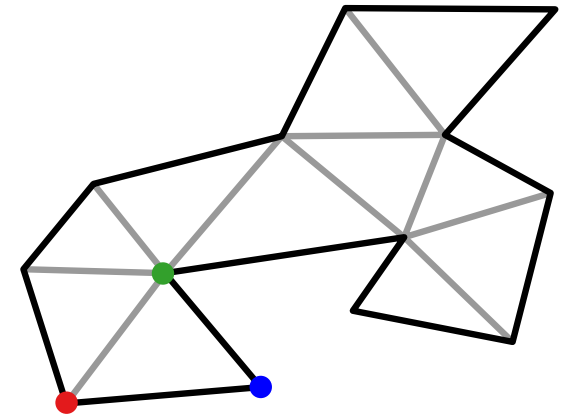
Theorem 2. For a simple polygon P with n vertices, $\lfloor n/3 \rfloor$ cameras are sometimes necessary and always sufficient to guard it.

- Sufficiency: Triangulate P and find set V' of corners, s.t. each triangle is incident to at least one $v \in V'$ and $|V'| \leq \lfloor n/3 \rfloor$.

The Art-Gallery-Theorem [Chvátal '75]

Theorem 2. For a simple polygon P with n vertices, $\lfloor n/3 \rfloor$ cameras are sometimes necessary and always sufficient to guard it.

- Sufficiency: Triangulate P and find set V' of corners, s.t. each triangle is incident to at least one $v \in V'$ and $|V'| \leq \lfloor n/3 \rfloor$.
- translate into a graph coloring problem with 3 colors, s.t. no two adjacent vertices have the same color; if this works with 3 colors, assign V' to be the smallest color class. $\Rightarrow$ size is $\leq \lfloor n/3 \rfloor$.



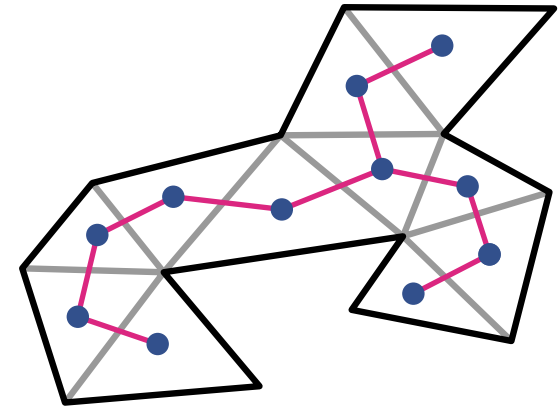
The Art-Gallery-Theorem [Chvátal '75]

Theorem 2. For a simple polygon P with n vertices, $\lfloor n/3 \rfloor$ cameras are sometimes necessary and always sufficient to guard it.

- Sufficiency: Triangulate P and find set V' of corners, s.t. each triangle is incident to at least one $v \in V'$ and $|V'| \leq \lfloor n/3 \rfloor$.
- translate into a graph coloring problem with 3 colors, s.t. no two adjacent vertices have the same color; if this works with 3 colors, assign V' to be the smallest color class. $\Rightarrow$ size is $\leq \lfloor n/3 \rfloor$.

Consider the dual graph of the triangulation:

- since diagonals always split P into two parts, the dual graph is acyclic and connected (actually a tree).



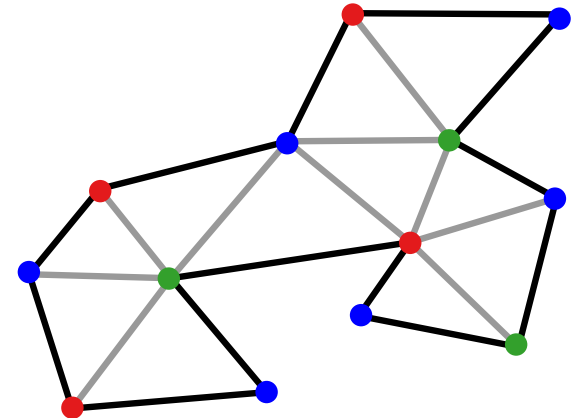
The Art-Gallery-Theorem [Chvátal '75]

Theorem 2. For a simple polygon P with n vertices, $\lfloor n/3 \rfloor$ cameras are sometimes necessary and always sufficient to guard it.

- Sufficiency: Triangulate P and find set V' of corners, s.t. each triangle is incident to at least one $v \in V'$ and $|V'| \leq \lfloor n/3 \rfloor$.
- translate into a graph coloring problem with 3 colors, s.t. no two adjacent vertices have the same color; if this works with 3 colors, assign V' to be the smallest color class. $\Rightarrow$ size is $\leq \lfloor n/3 \rfloor$.

Consider the dual graph of the triangulation:

- since diagonals always split P into two parts, the dual graph is acyclic and connected (actually a tree).



$\Rightarrow$ color the corners of P in a DFS traversal of T , and assign the unique “free” color to the next triangle. This yields a 3-coloring of the triangulation. □

The Art-Gallery-Theorem [Chvátal '75]

Theorem 2. For a simple polygon P with n vertices, $\lfloor n/3 \rfloor$ cameras are sometimes necessary and always sufficient to guard it.

Conclusion. Given a triangulation, the at most $\lfloor n/3 \rfloor$ cameras that guard the polygon can be placed in $O(n)$ time.



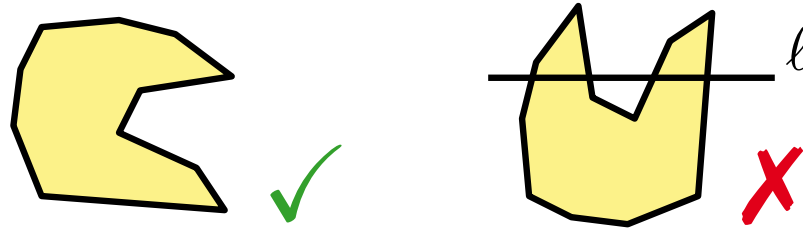
DFS traversal

Faster Triangulation Algorithm: Overview

3-step process:

- Step 1: Decompose P into y -monotone polygons

Definition: A polygon is y -monotone, if for any horizontal line ℓ , the intersection $\ell \cap P$ is connected.



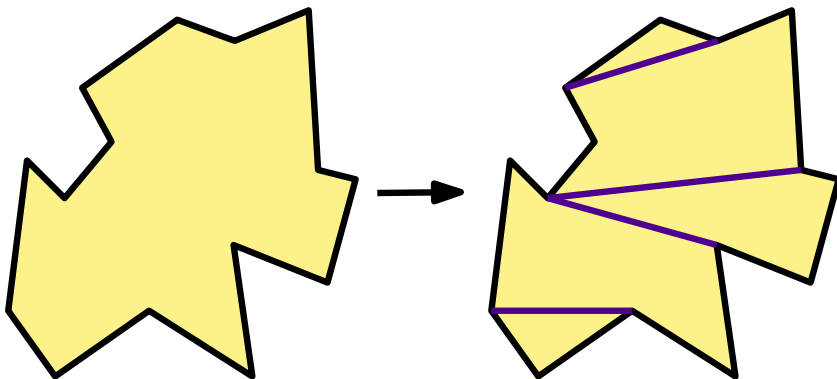
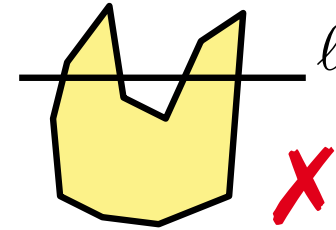
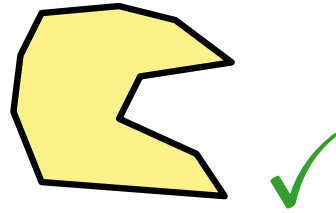
Faster Triangulation Algorithm: Overview

3-step process:

- Step 1: Decompose P into y -monotone polygons

Definition: A polygon is y -monotone, if for any horizontal line ℓ , the intersection $\ell \cap P$ is connected.

two paths from topmost to bottommost vertex never go upward



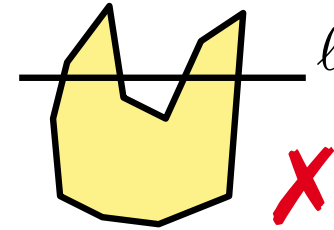
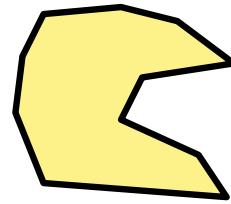
Faster Triangulation Algorithm: Overview

3-step process:

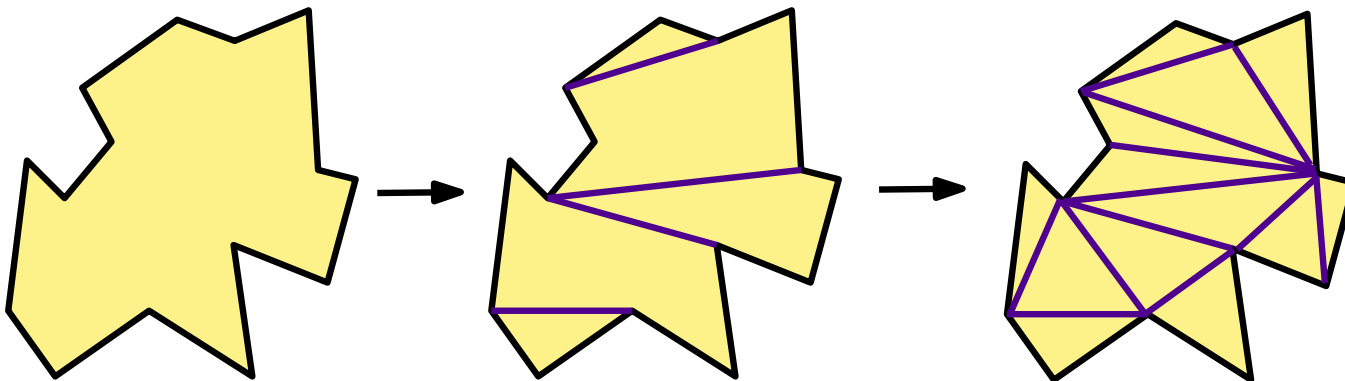
- Step 1: Decompose P into y -monotone polygons

Definition: A polygon is y -monotone, if for any horizontal line ℓ , the intersection $\ell \cap P$ is connected.

two paths from topmost to bottommost vertex never go upward



- Step 2: Triangulate the resulting y -monotone polygons



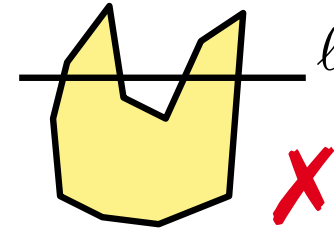
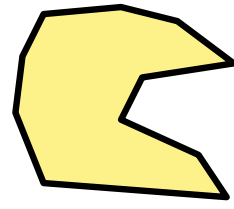
Faster Triangulation Algorithm: Overview

3-step process:

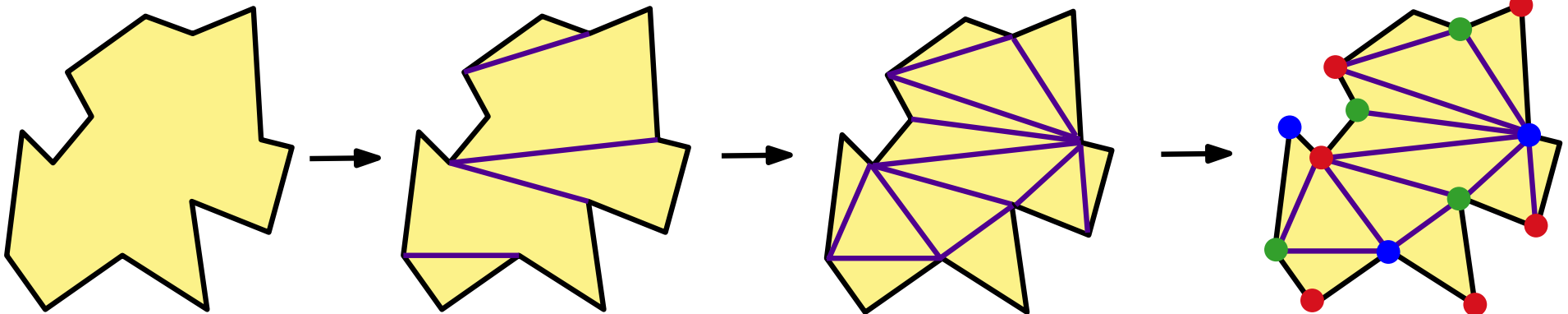
- Step 1: Decompose P into y -monotone polygons

Definition: A polygon is y -monotone, if for any horizontal line ℓ , the intersection $\ell \cap P$ is connected.

two paths from topmost to bottommost vertex never go upward



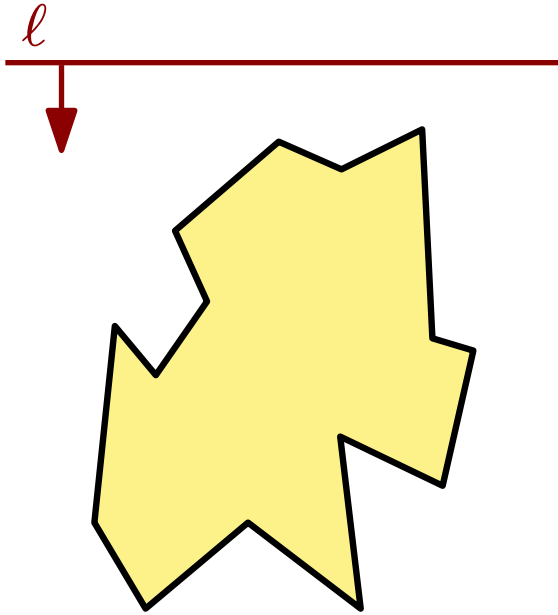
- Step 2: Triangulate the resulting y -monotone polygons
- Step 3: use DFS to color the vertices of the triangulation ✓



Step 1: Split into y -monotone polygons.

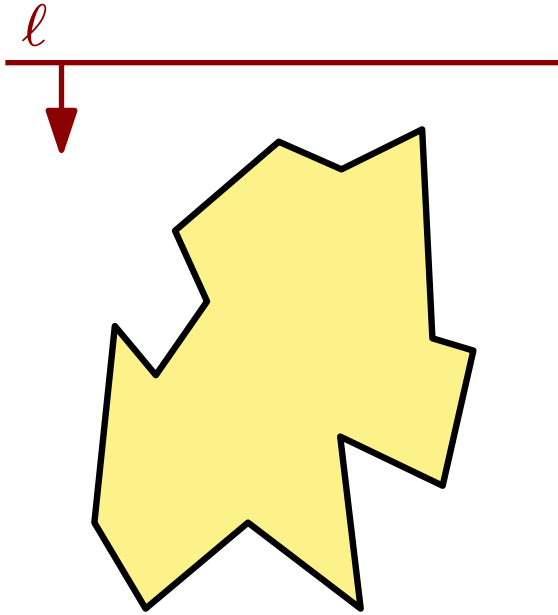
Partition into y -monotone Polygons

Idea: Five different types of vertices



Partition into y -monotone Polygons

Idea: Five different types of vertices
– *Turn vertices:*

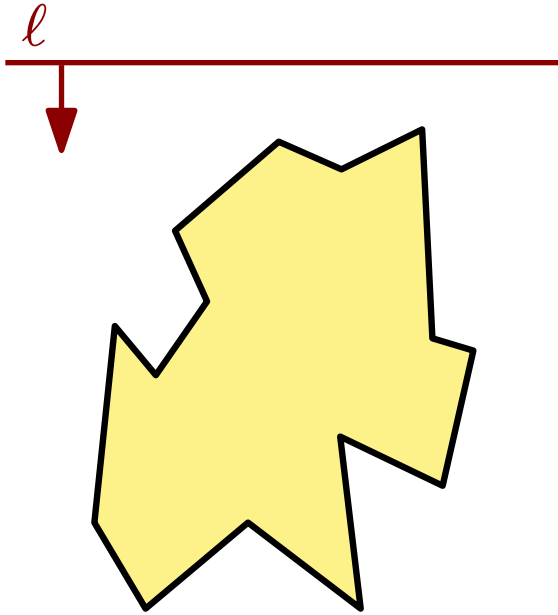


– *Regular vertices*

Partition into y -monotone Polygons

Idea: Five different types of vertices

- *Turn vertices*:
change in vertical direction



- *Regular vertices*

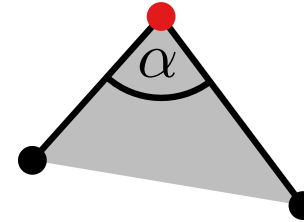
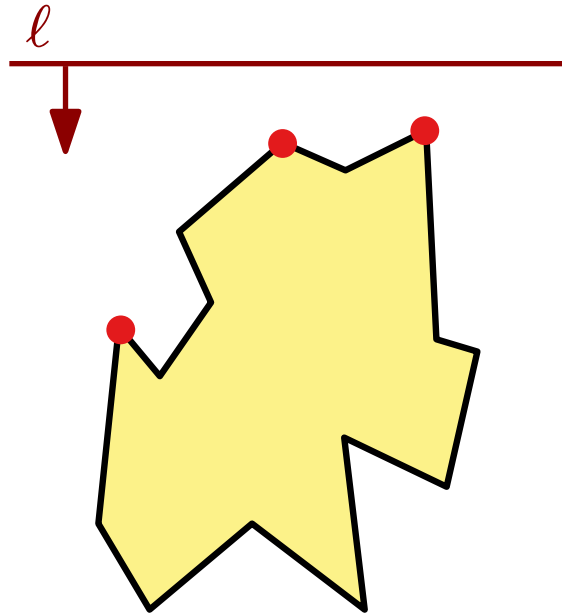
Partition into y -monotone Polygons

Idea: Five different types of vertices

– *Turn vertices:*

change in vertical direction

■ *start* vertices



if $\alpha < 180^\circ$

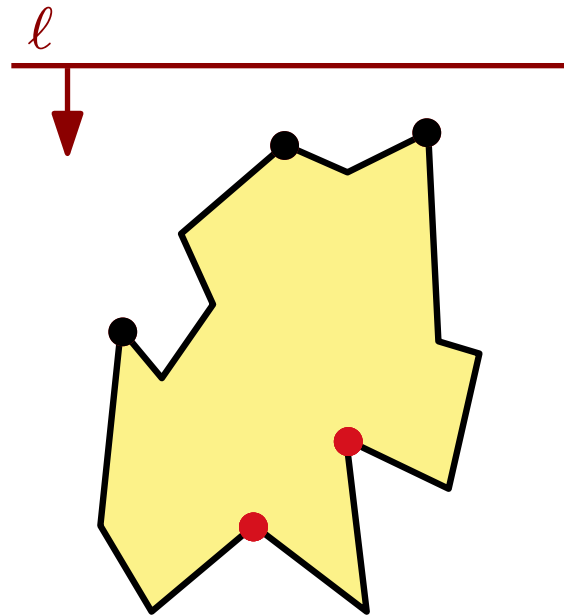
– *Regular* vertices

Partition into y -monotone Polygons

Idea: Five different types of vertices

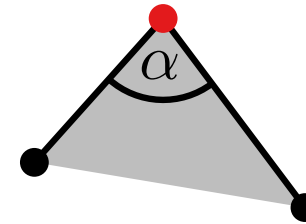
– *Turn vertices:*

change in vertical direction

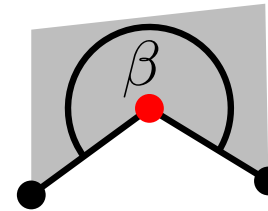


■ *start* vertices

■ *split* vertices



if $\alpha < 180^\circ$



if $\beta > 180^\circ$

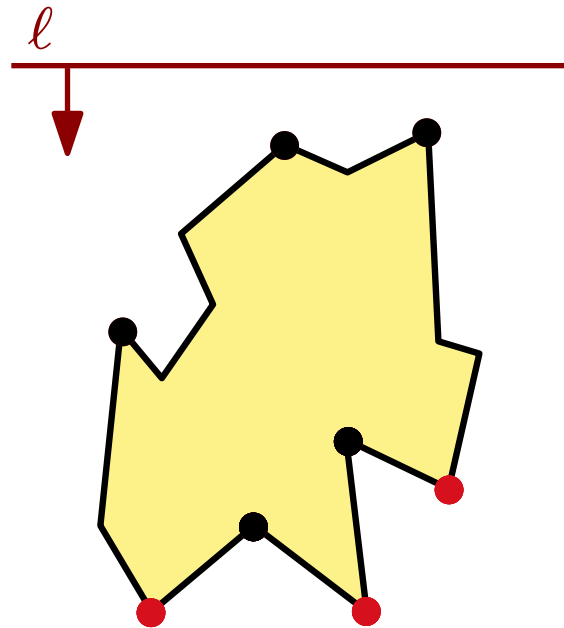
– *Regular* vertices

Partition into y -monotone Polygons

Idea: Five different types of vertices

– *Turn vertices:*

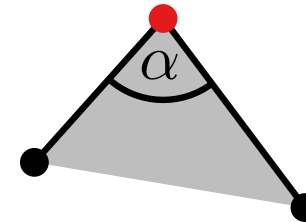
change in vertical direction



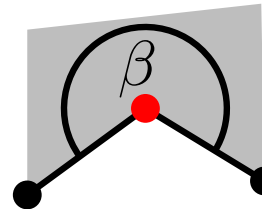
■ *start* vertices

■ *split* vertices

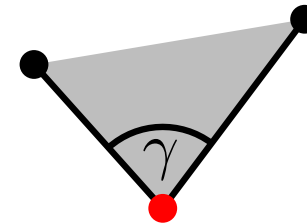
■ *end* vertices



if $\alpha < 180^\circ$



if $\beta > 180^\circ$



if $\gamma < 180^\circ$

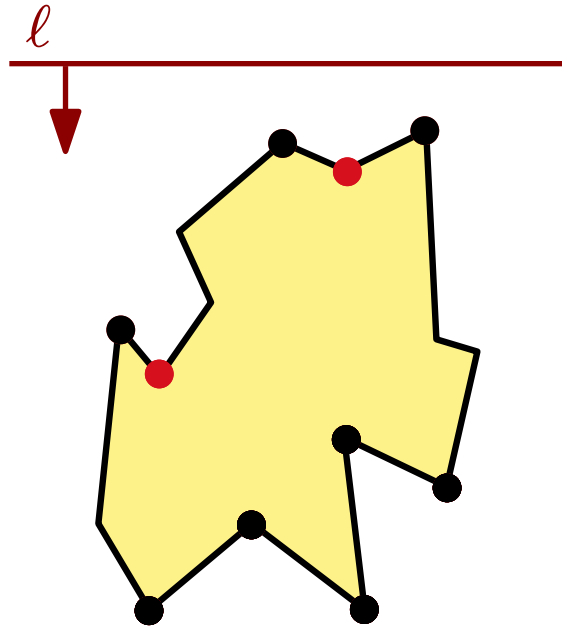
– *Regular* vertices

Partition into y -monotone Polygons

Idea: Five different types of vertices

– *Turn vertices:*

change in vertical direction

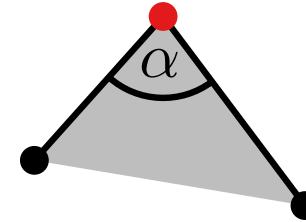


■ *start* vertices

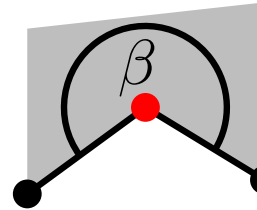
■ *split* vertices

■ *end* vertices

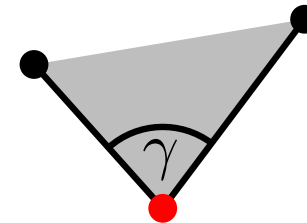
■ *merge* vertices



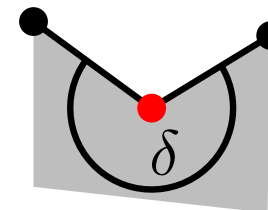
if $\alpha < 180^\circ$



if $\beta > 180^\circ$



if $\gamma < 180^\circ$



if $\delta > 180^\circ$

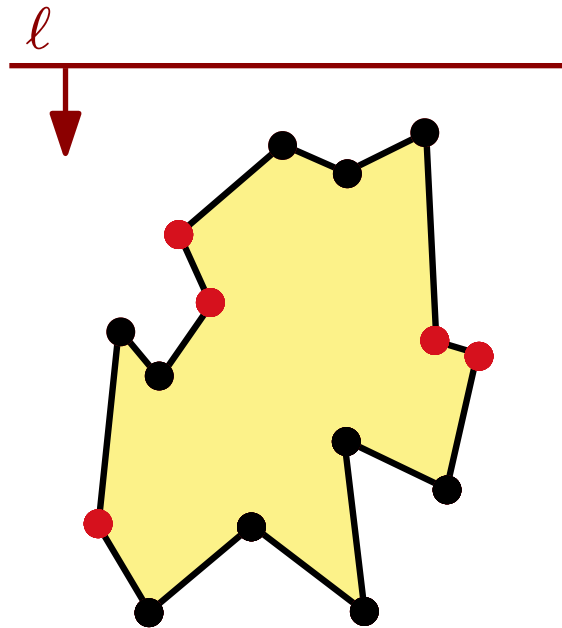
– *Regular* vertices

Partition into y -monotone Polygons

Idea: Five different types of vertices

– *Turn vertices:*

change in vertical direction



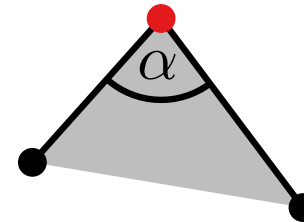
■ *start* vertices

■ *split* vertices

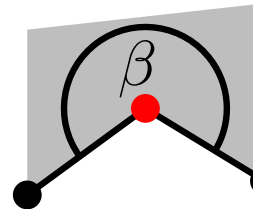
■ *end* vertices

■ *merge* vertices

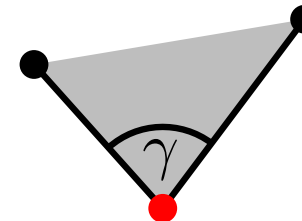
– *Regular* vertices



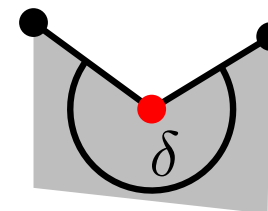
if $\alpha < 180^\circ$



if $\beta > 180^\circ$



if $\gamma < 180^\circ$



if $\delta > 180^\circ$

Characterization

Lemma 1: A polygon is y -monotone if it has no split vertices or merge vertices.

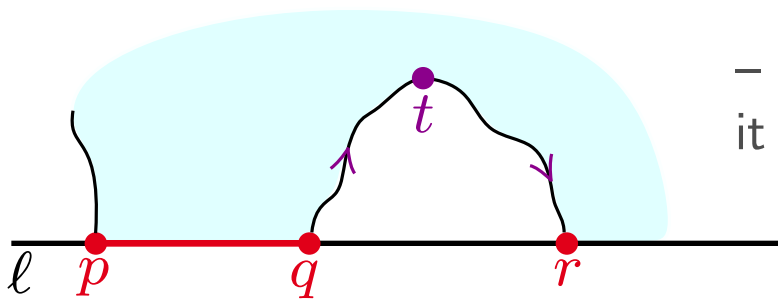
Characterization

Lemma 1: A polygon is y -monotone if it has no split vertices or merge vertices.

Proof:

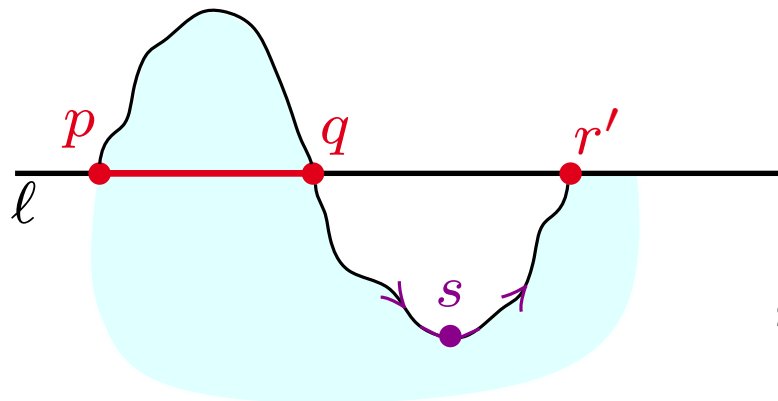
Assume P is not y -monotone and show there is a split or merge vertex in P .

Let ℓ be a horizontal line intersecting P in ≥ 2 segments and let $\overline{pq}$ be the leftmost such segment.



– start with q walk along P with p on the left hand side until it hits ℓ at point r .

Case-1: $r \neq p \implies r$ is to the right of $q \implies$ there is a point t (local maximum point) is a split vertex.



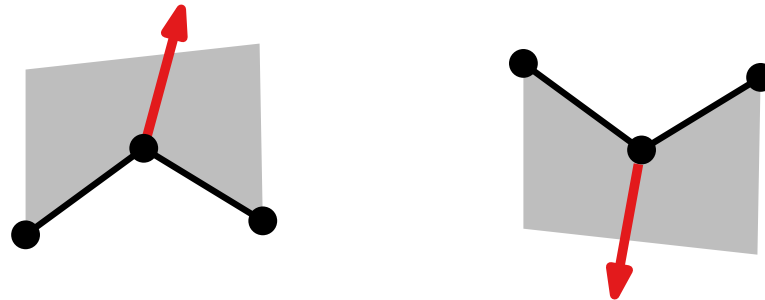
Case-2: $r = p$; start with q but with P on the right hand side; r' is the right of q ; point s is a merge vertex.



Characterization

Lemma 1: A polygon is y -monotone if it has no split vertices or merge vertices.

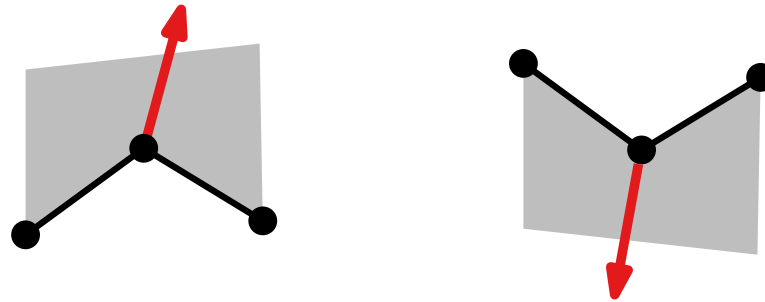
⇒ We need to eliminate all split and merge vertices by adding diagonals



Characterization

Lemma 1: A polygon is y -monotone if it has no split vertices or merge vertices.

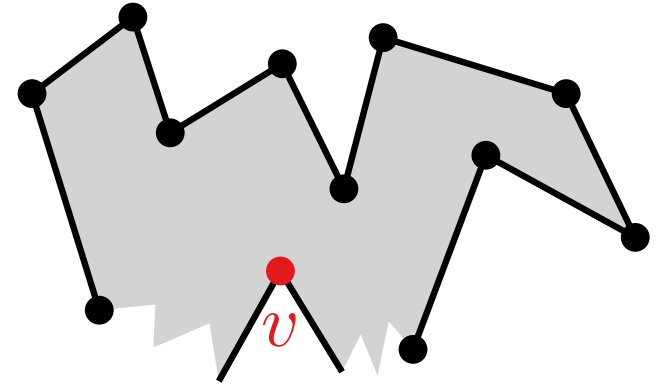
⇒ We need to eliminate all split and merge vertices by adding diagonals



Observation: The diagonals should neither cross the edges of P nor the other diagonals.

Towards a Sweep-Line-Algorithm

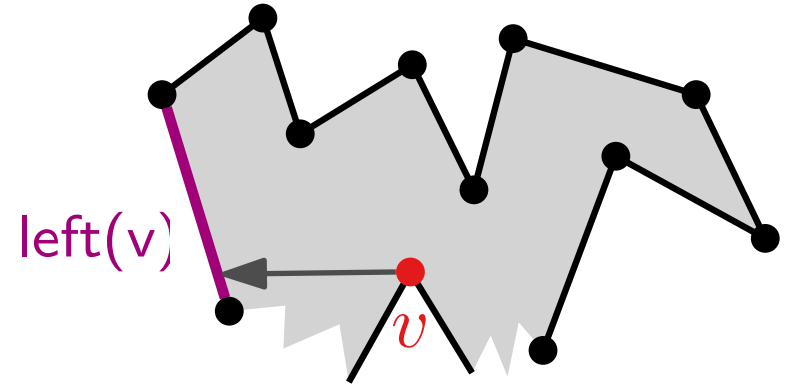
1) Diagonals for the split vertices



Towards a Sweep-Line-Algorithm

1) Diagonals for the split vertices

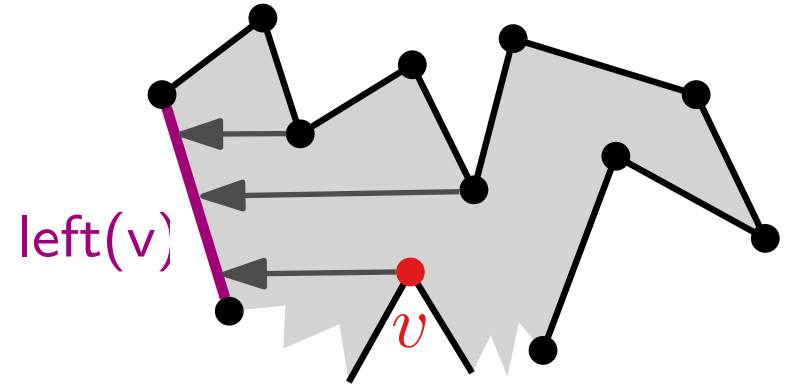
- find for each vertex v its left adjacent edge $\text{left}(v)$ wrt. horizontal sweep line ℓ



Towards a Sweep-Line-Algorithm

1) Diagonals for the split vertices

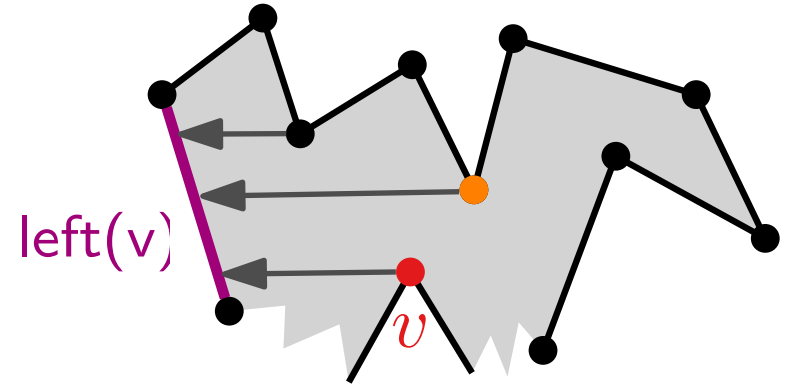
- find for each vertex v its left adjacent edge $\text{left}(v)$ wrt. horizontal sweep line ℓ



Towards a Sweep-Line-Algorithm

1) Diagonals for the split vertices

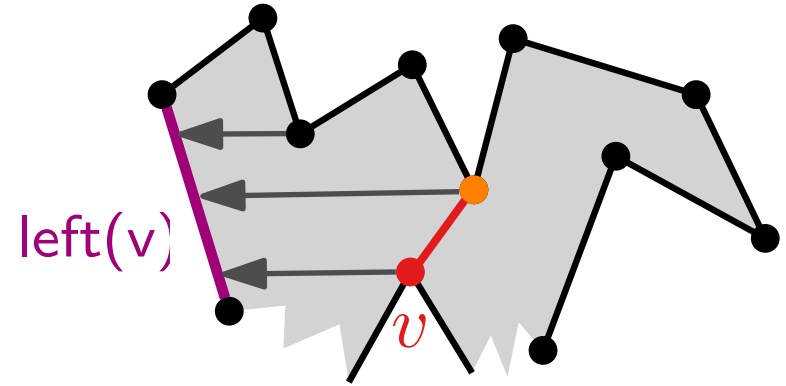
- find for each vertex v its left adjacent edge $\text{left}(v)$ wrt. horizontal sweep line ℓ
- connect split vertex v to lowest vertex w above v with $\text{left}(w) = \text{left}(v)$



Towards a Sweep-Line-Algorithm

1) Diagonals for the split vertices

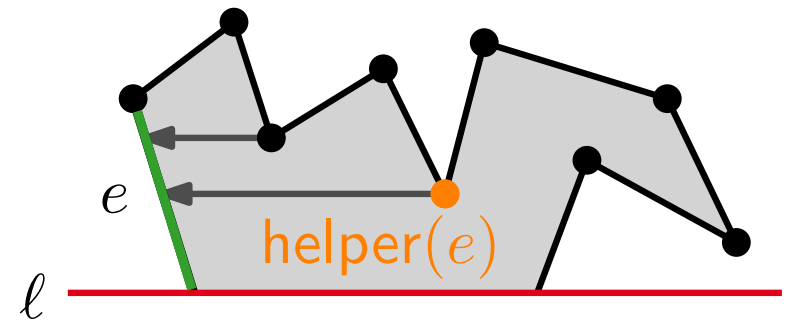
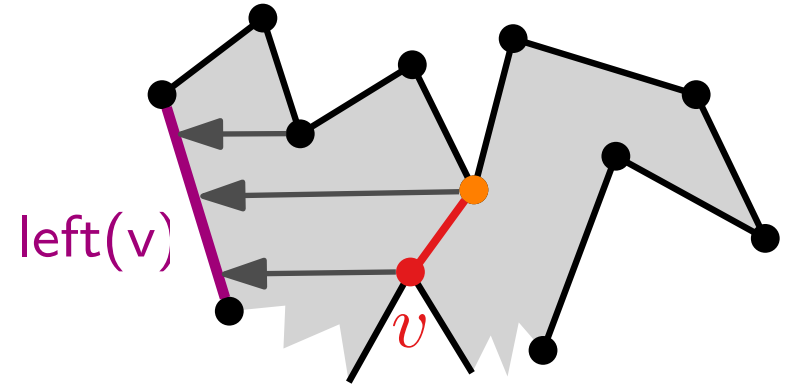
- find for each vertex v its left adjacent edge $\text{left}(v)$ wrt. horizontal sweep line ℓ
- connect split vertex v to lowest vertex w above v with $\text{left}(w) = \text{left}(v)$



Towards a Sweep-Line-Algorithm

1) Diagonals for the split vertices

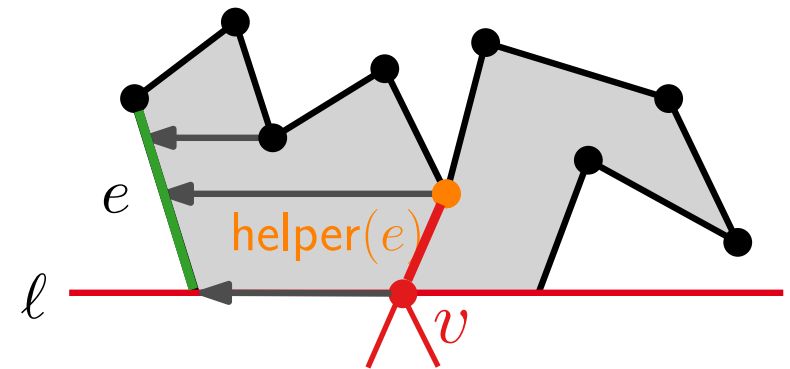
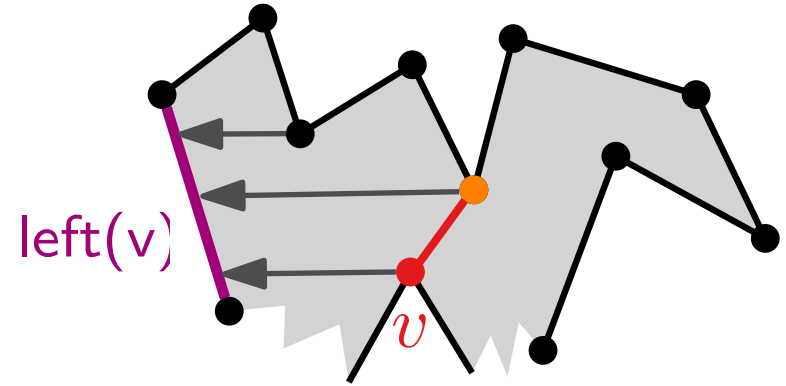
- find for each vertex v its left adjacent edge $\text{left}(v)$ wrt. horizontal sweep line ℓ
- connect split vertex v to lowest vertex w above v with $\text{left}(w) = \text{left}(v)$
- for each edge e store the lowest vertex w such that $\text{left}(w) = e$;
notation $\text{helper}(e) := w$



Towards a Sweep-Line-Algorithm

1) Diagonals for the split vertices

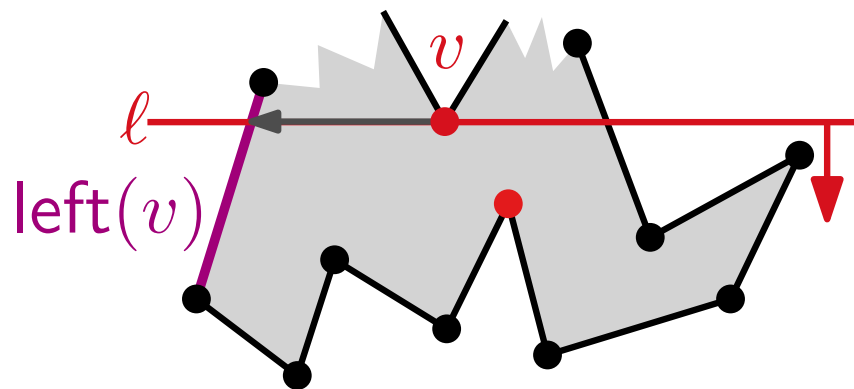
- find for each vertex v its left adjacent edge $\text{left}(v)$ wrt. horizontal sweep line ℓ
- connect split vertex v to lowest vertex w above v with $\text{left}(w) = \text{left}(v)$
- for each edge e store the lowest vertex w such that $\text{left}(w) = e$;
notation $\text{helper}(e) := w$
- when ℓ reaches a split vertex v , we connect v with $\text{helper}(\text{left}(v))$



Towards a sweep line algorithm

2) Diagonals for merge vertices

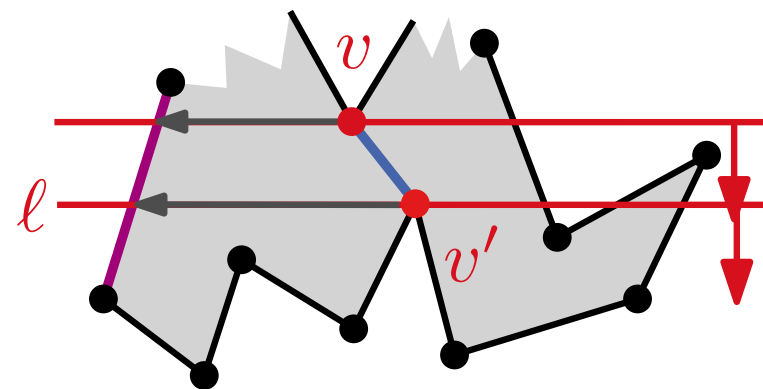
- when merge vertex v is reached, we set $\text{helper}(\text{left}(v)) = v$



Towards a sweep line algorithm

2) Diagonals for merge vertices

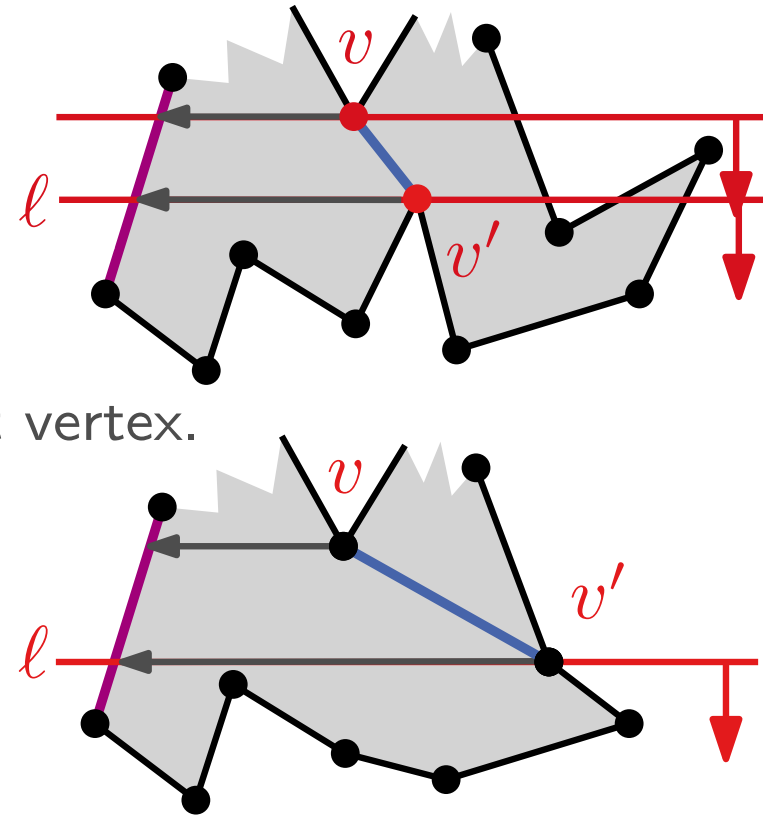
- when merge vertex v is reached, we set $\text{helper}(\text{left}(v)) = v$
- when split vertex v' with $\text{left}(v') = \text{left}(v)$ is reached, introduce diagonal (v, v')
- However, not necessary that next vertex is a split vertex.



Towards a sweep line algorithm

2) Diagonals for merge vertices

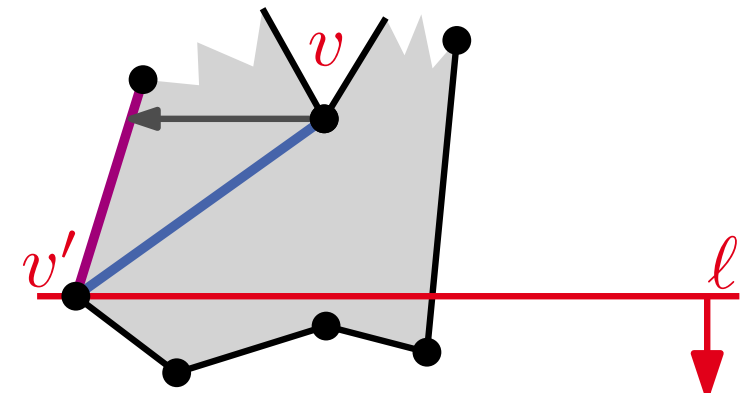
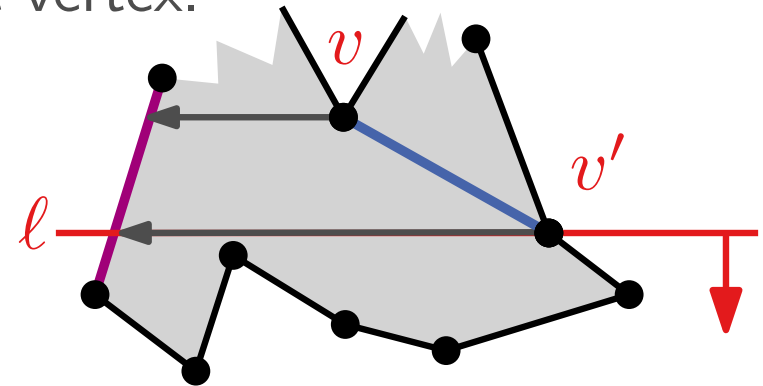
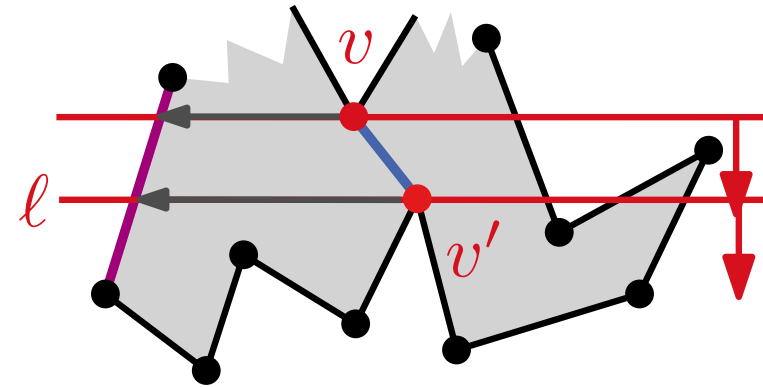
- when merge vertex v is reached, we set $\text{helper}(\text{left}(v)) = v$
- when split vertex v' with $\text{left}(v') = \text{left}(v)$ is reached, introduce diagonal (v, v')
- However, not necessary that next vertex is a split vertex.
- when $\text{helper}(\text{left}(v))$ is replaced by v' , introduce diagonal (v, v')
- Since the next vertex is a regular vertex.



Towards a sweep line algorithm

2) Diagonals for merge vertices

- when merge vertex v is reached, we set $\text{helper}(\text{left}(v)) = v$
- when split vertex v' with $\text{left}(v') = \text{left}(v)$ is reached, introduce diagonal (v, v')
- However, not necessary that next vertex is a split vertex.
- when $\text{helper}(\text{left}(v))$ is replaced by v' , introduce diagonal (v, v')
- Since the next vertex is a regular vertex.



Algorithm MakeMonotone(P)

MakeMonotone(Polygon P)  planar embedded graphs or polygons (face info.), etc.

$\mathcal{D} \leftarrow$ doubly-connected edge list for $(V(P), E(P))$

$\mathcal{Q} \leftarrow$ priority queue for $V(P)$ sorted lexicographically by y and x

$\mathcal{T} \leftarrow \emptyset$ (binary search tree for sweep-line status)

while $\mathcal{Q} \neq \emptyset$ **do**

$v \leftarrow \mathcal{Q}.\text{nextVertex}()$
 $\mathcal{Q}.\text{deleteVertex}(v)$
 $\text{handleVertex}(v)$
 based on type

return $\mathcal{D}$

Algorithm MakeMonotone(P)

MakeMonotone(Polygon P) planar embedded graphs or polygons (face info.), etc.

$\mathcal{D} \leftarrow$ doubly-connected edge list for $(V(P), E(P))$

$\mathcal{Q} \leftarrow$ priority queue for $V(P)$ sorted lexicographically by y and x

$\mathcal{T} \leftarrow \emptyset$ (binary search tree for sweep-line status)

while $\mathcal{Q} \neq \emptyset$ **do** keeps currently intersected left edges.

$v \leftarrow \mathcal{Q}.\text{nextVertex}()$
 $\mathcal{Q}.\text{deleteVertex}(v)$
 $\text{handleVertex}(v)$

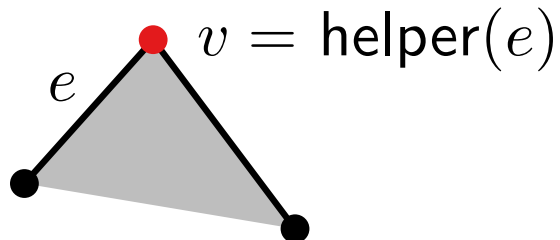
based on type

return $\mathcal{D}$

handleStartVertex(vertex v)

$\mathcal{T} \leftarrow$ add the left edge e

$\text{helper}(e) \leftarrow v$



Algorithm MakeMonotone(P)

MakeMonotone(Polygon P) planar embedded graphs or polygons (face info.), etc.

$\mathcal{D} \leftarrow$ doubly-connected edge list for $(V(P), E(P))$

$Q \leftarrow$ priority queue for $V(P)$ sorted lexicographically by y and x

$\mathcal{T} \leftarrow \emptyset$ (binary search tree for sweep-line status)

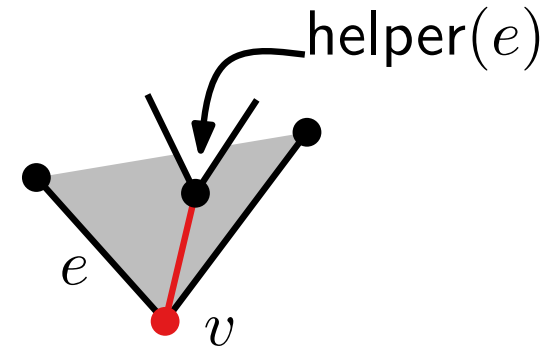
while $Q \neq \emptyset$ **do** keeps currently intersected left edges.

$v \leftarrow Q.\text{nextVertex}()$

$Q.\text{deleteVertex}(v)$

$\text{handleVertex}(v)$
based on type

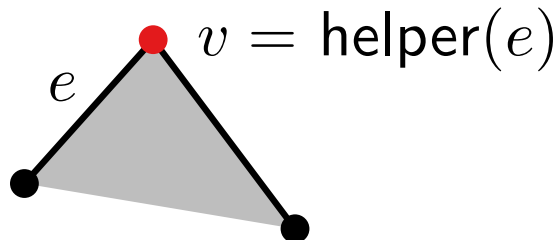
return $\mathcal{D}$



handleStartVertex(vertex v)

$\mathcal{T} \leftarrow$ add the left edge e

$\text{helper}(e) \leftarrow v$



handleEndVertex(vertex v)

$e \leftarrow$ left edge

if $\text{isMergeVertex}(\text{helper}(e))$ **then**

$\mathcal{D} \leftarrow$ add edge $(\text{helper}(e), v)$

remove e from $\mathcal{T}$

Algorithm MakeMonotone(P)

MakeMonotone(Polygon P) planar embedded graphs or polygons (face info.), etc.

$\mathcal{D} \leftarrow$ doubly-connected edge list for $(V(P), E(P))$

$\mathcal{Q} \leftarrow$ priority queue for $V(P)$ sorted lexicographically by y and x

$\mathcal{T} \leftarrow \emptyset$ (binary search tree for sweep-line status)

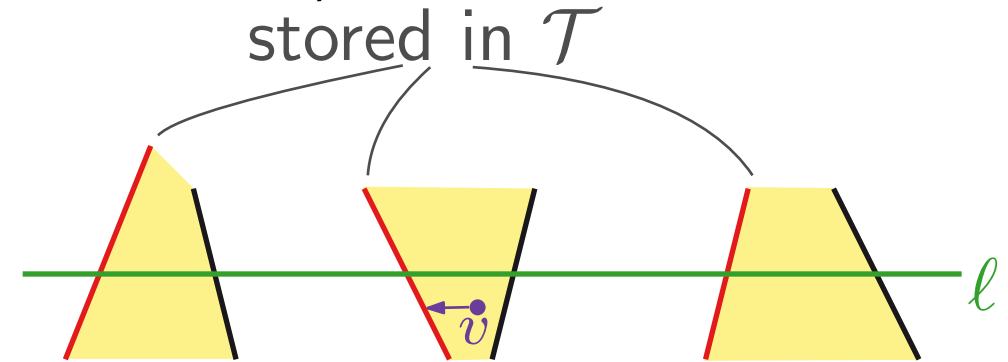
while $\mathcal{Q} \neq \emptyset$ **do**

$v \leftarrow \mathcal{Q}.\text{nextVertex}()$

$\mathcal{Q}.\text{deleteVertex}(v)$

$\text{handleVertex}(v)$
based on type

return $\mathcal{D}$



handleSplitVertex(vertex v)

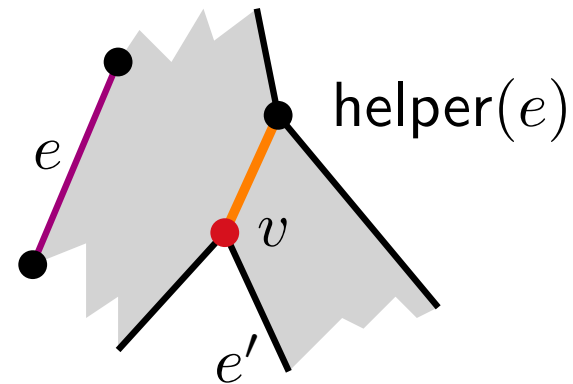
$e \leftarrow$ edge to the left of v in $\mathcal{T}$

$\mathcal{D} \leftarrow$ add edge $(\text{helper}(e), v)$

$\text{helper}(e) \leftarrow v$

$\mathcal{T} \leftarrow$ add the right edge e' of v

$\text{helper}(e') \leftarrow v$



Algorithm MakeMonotone(P)

MakeMonotone(Polygon P) planar embedded graphs or polygons (face info.), etc.

$\mathcal{D} \leftarrow$ doubly-connected edge list for $(V(P), E(P))$

$\mathcal{Q} \leftarrow$ priority queue for $V(P)$ sorted lexicographically by y and x

$\mathcal{T} \leftarrow \emptyset$ (binary search tree for sweep-line status)

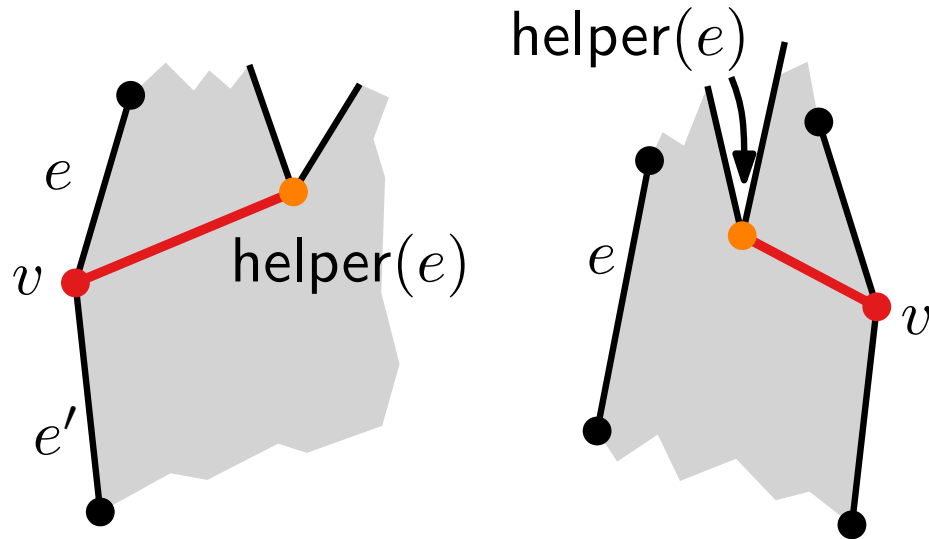
while $\mathcal{Q} \neq \emptyset$ **do**

$v \leftarrow \mathcal{Q}.\text{nextVertex}()$

$\mathcal{Q}.\text{deleteVertex}(v)$

$\text{handleVertex}(v)$
based on type

return $\mathcal{D}$



handleRegularVertex(vertex v)

if P lies locally to the right of v **then**

$e, e' \leftarrow$ upper, lower edge

if $\text{isMergeVertex}(\text{helper}(e))$ **then**

$\mathcal{D} \leftarrow$ add edge $(\text{helper}(e), v)$

remove e from $\mathcal{T}$

$\mathcal{T} \leftarrow$ add e' ; $\text{helper}(e') \leftarrow v$

else

$e \leftarrow$ edge to the left of v in $\mathcal{T}$

if $\text{isMergeVertex}(\text{helper}(e))$ **then**

$\mathcal{D} \leftarrow$ add edge $(\text{helper}(e), v)$

$\text{helper}(e) \leftarrow v$

Analysis

Lemma 2: Algorithm MakeMonotone computes a set of crossing-free diagonals of P , which partition P into y -monotone polygons.

Analysis

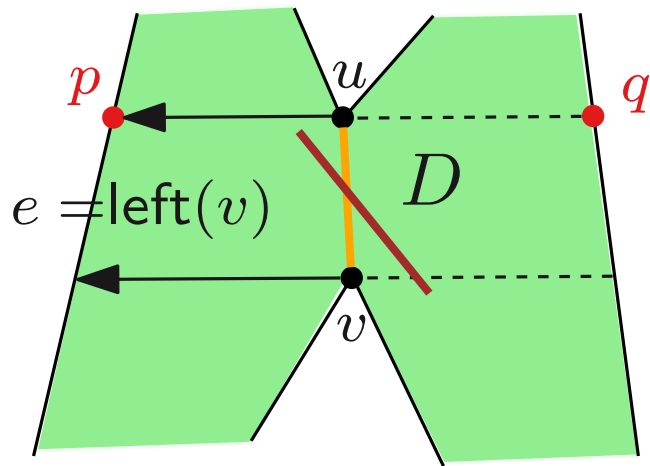
Lemma 2: Algorithm MakeMonotone computes a set of crossing-free diagonals of P , which partition P into y -monotone polygons.

Proof:

- by construction all split and merge vertices are removed
- need to show that all inserted diagonals are valid, i.e., they produce no crossings

here: handleSplitVertex

$$u = \text{helper}(e)$$



- no vertices in region D .
- potentially crossing edge of $\overline{uv}$ multi intersect $\overline{pu}$ or $\overline{uq}$.
- $\text{left}(u) \neq \text{left}(v)$.

other subroutines use similar arguments.

Analysis

Lemma 2: Algorithm MakeMonotone computes a set of crossing-free diagonals of P , which partition P into y -monotone polygons.

Theorem 3. A simple polygon with n vertices can be partitioned into y -monotone polygons in $O(n \log n)$ time and $O(n)$ space.

Analysis

Lemma 2: Algorithm MakeMonotone computes a set of crossing-free diagonals of P , which partition P into y -monotone polygons.

Theorem 3. A simple polygon with n vertices can be partitioned into y -monotone polygons in $O(n \log n)$ time and $O(n)$ space.

- Construct priority queue Q : $O(n)$
- Initialize sweep-line status $\mathcal{T}$: $O(1)$
- Handle a single event: $O(\log n)$
 - $Q.deleteMax$: $O(\log n)$
 - Find, remove, add element in $\mathcal{T}$: $O(\log n)$
 - Add ≤ 2 diagonals to $\mathcal{D}$: $O(1)$
- Space: obviously $O(n)$

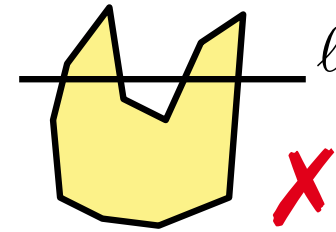
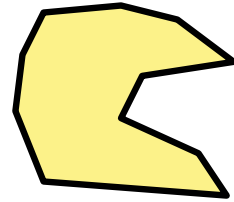
Fast Triangulation Algorithm: Overview

3-step process:

- Step 1: Decompose P into y -monotone polygons

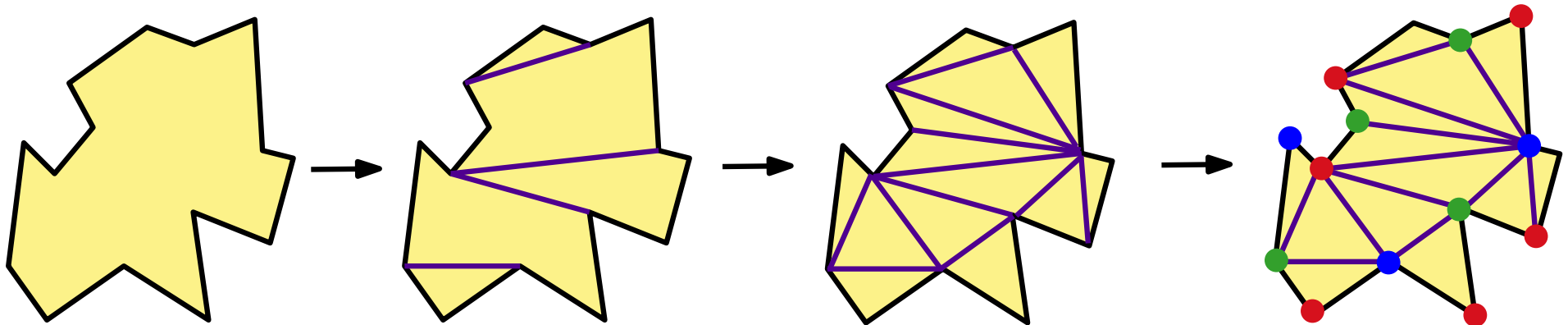
Definition: A polygon is y -monotone, if for any horizontal line ℓ , the intersection $\ell \cap P$ is connected.

two paths from topmost
to bottommost vertex
never go upward



ToDo!

- Step 2: Triangulate the resulting y -monotone polygons
- Step 3: use DFS to color the vertices of the triangulation ✓

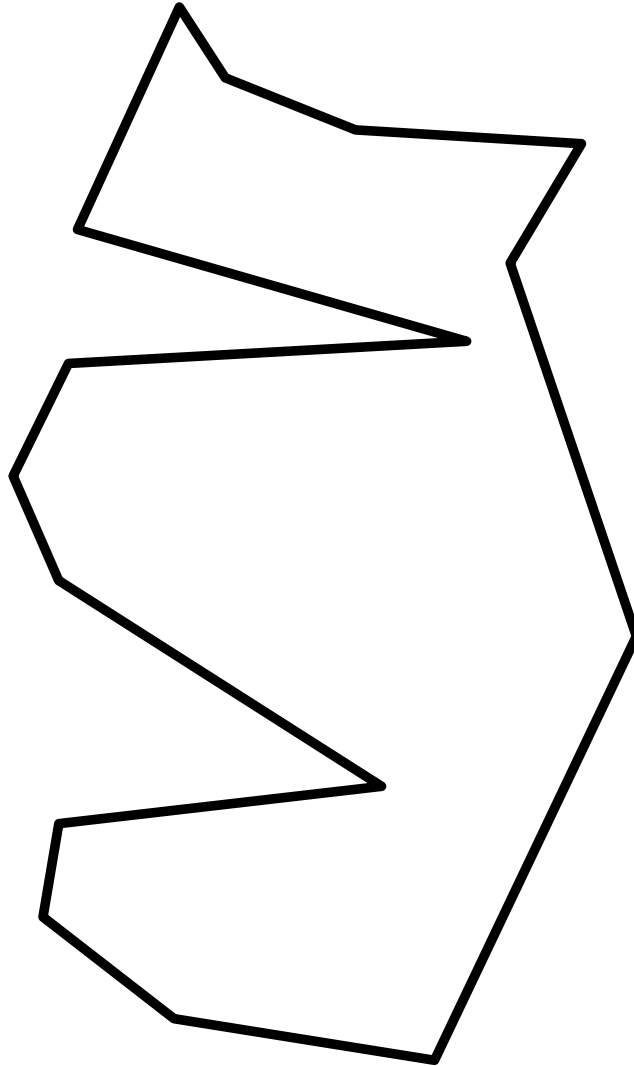


Step 1: Triangulate y -monotone polygons.

Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

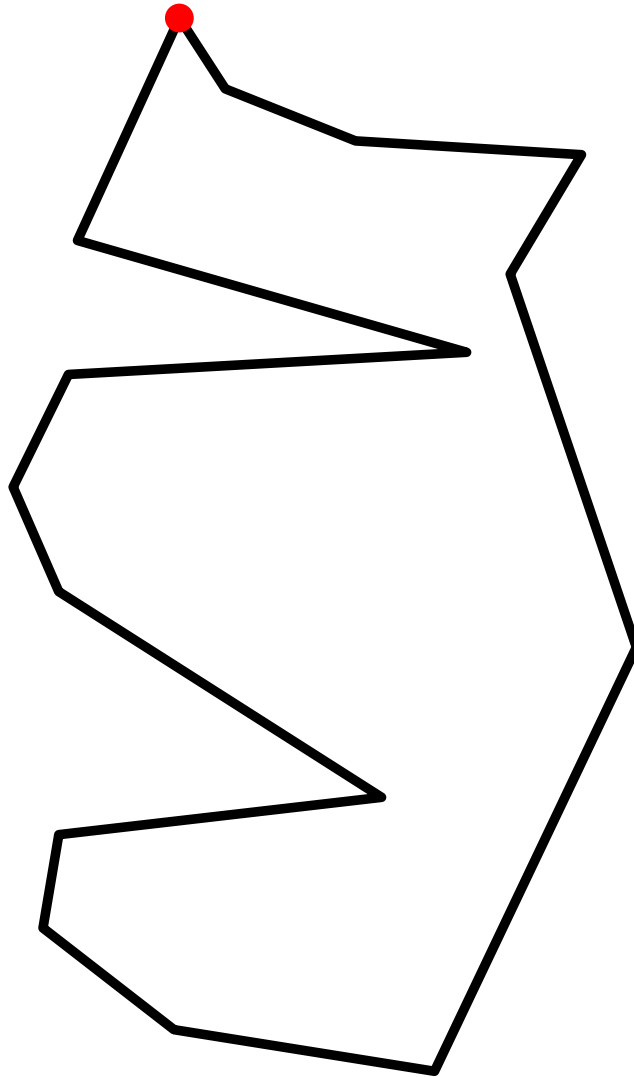
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

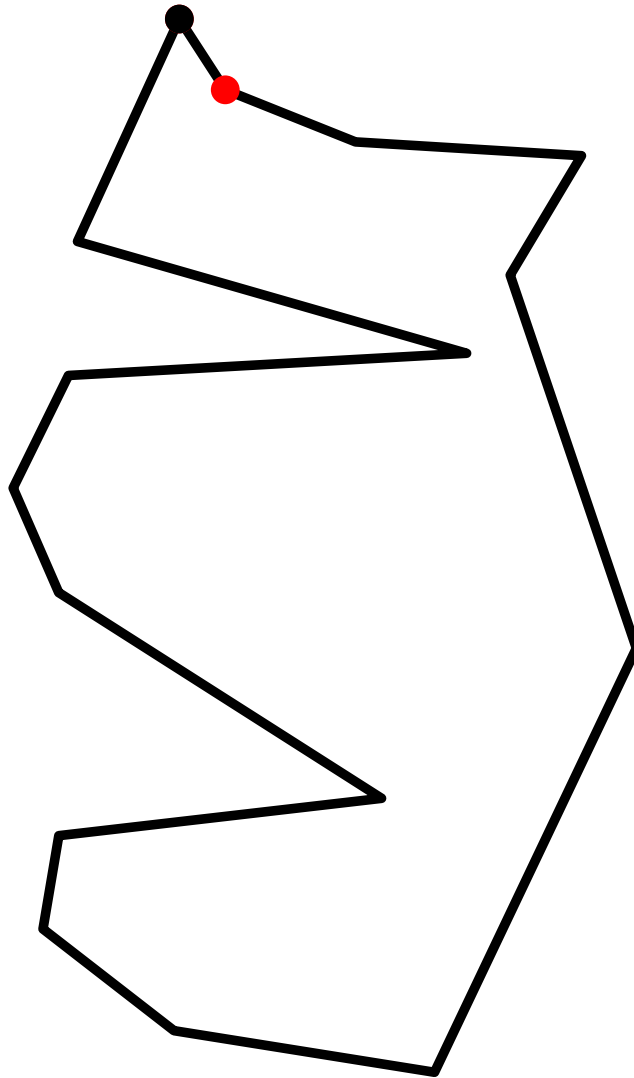
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

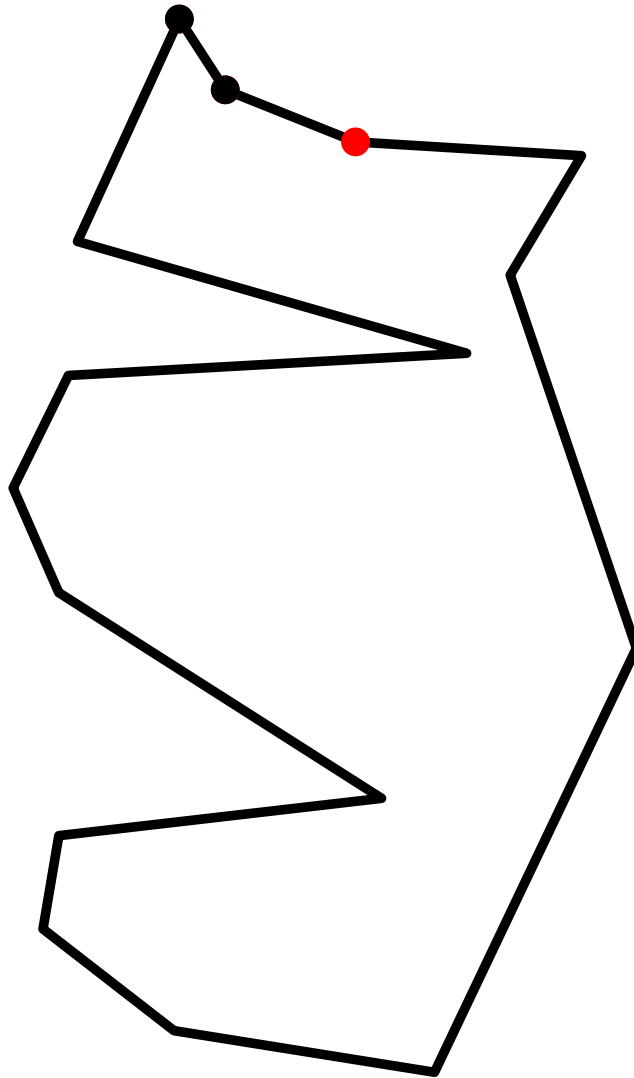
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

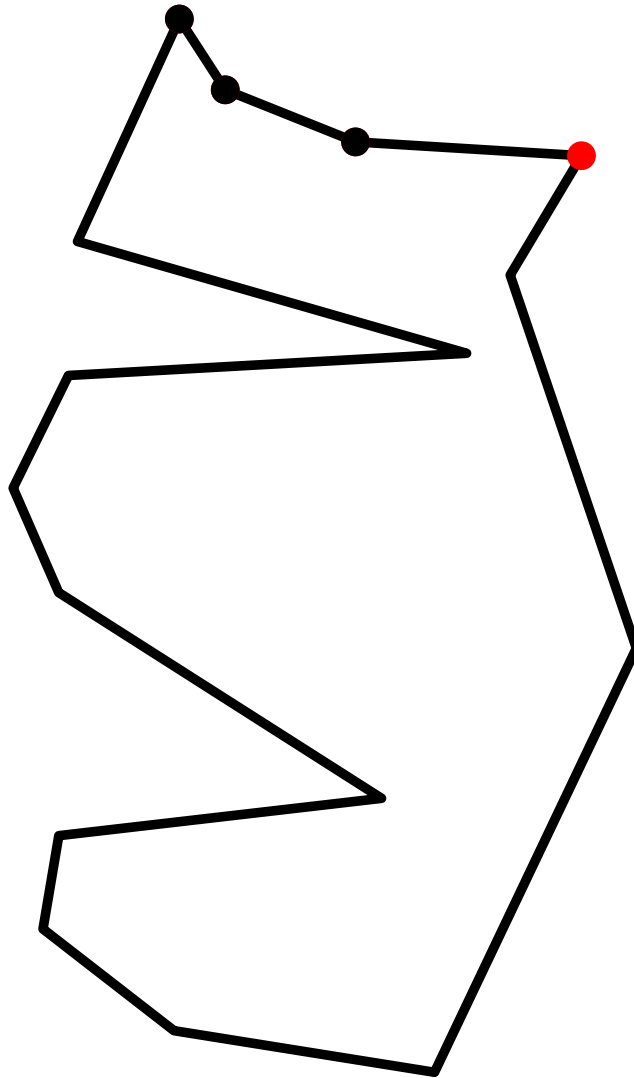
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

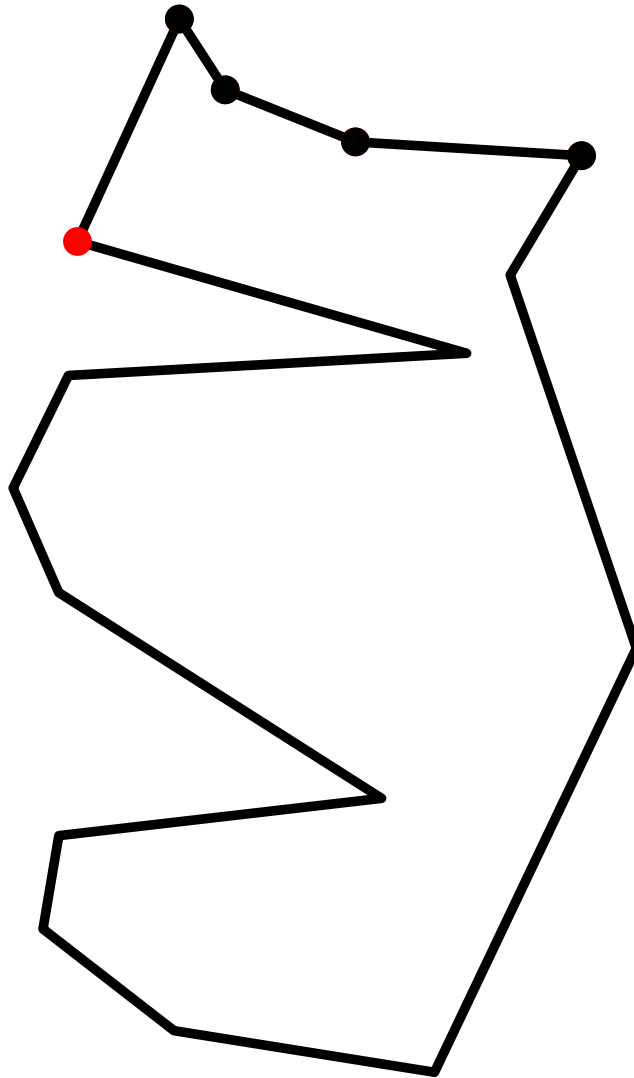
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

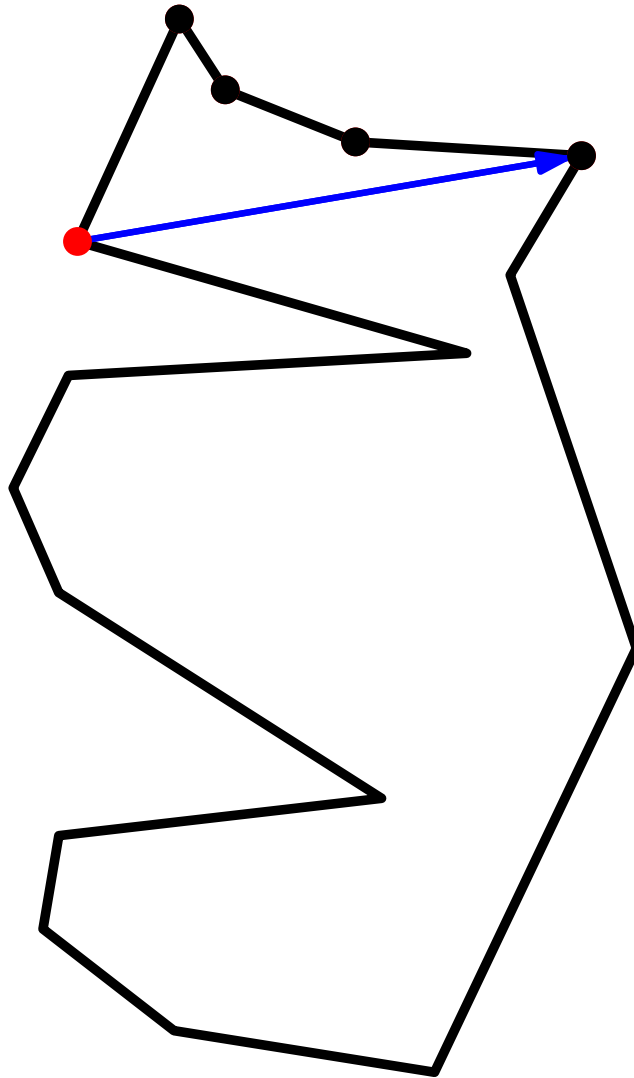
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

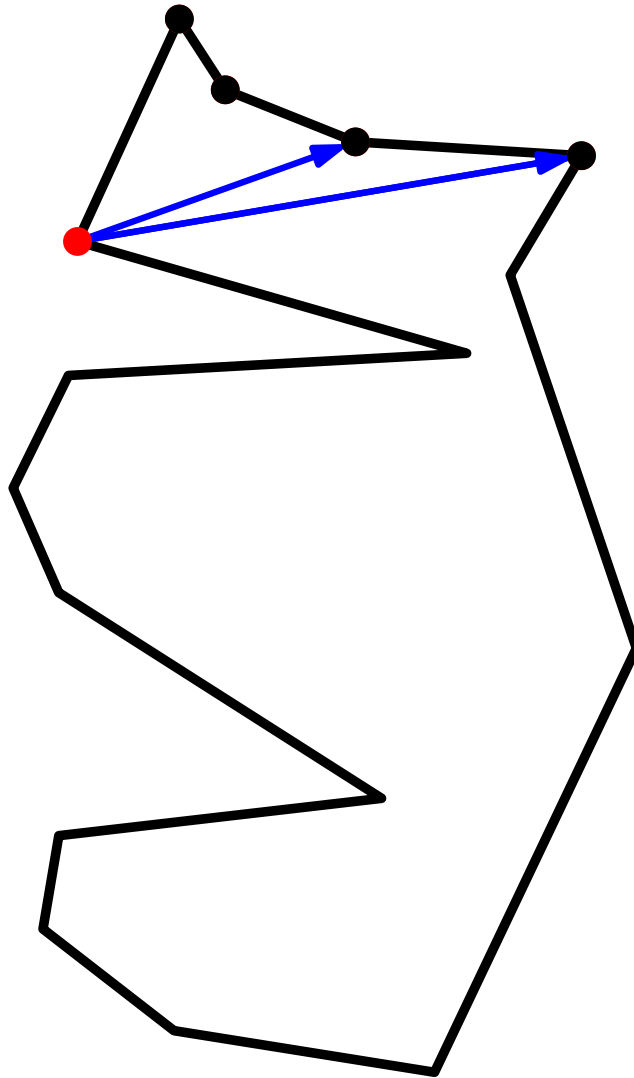
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

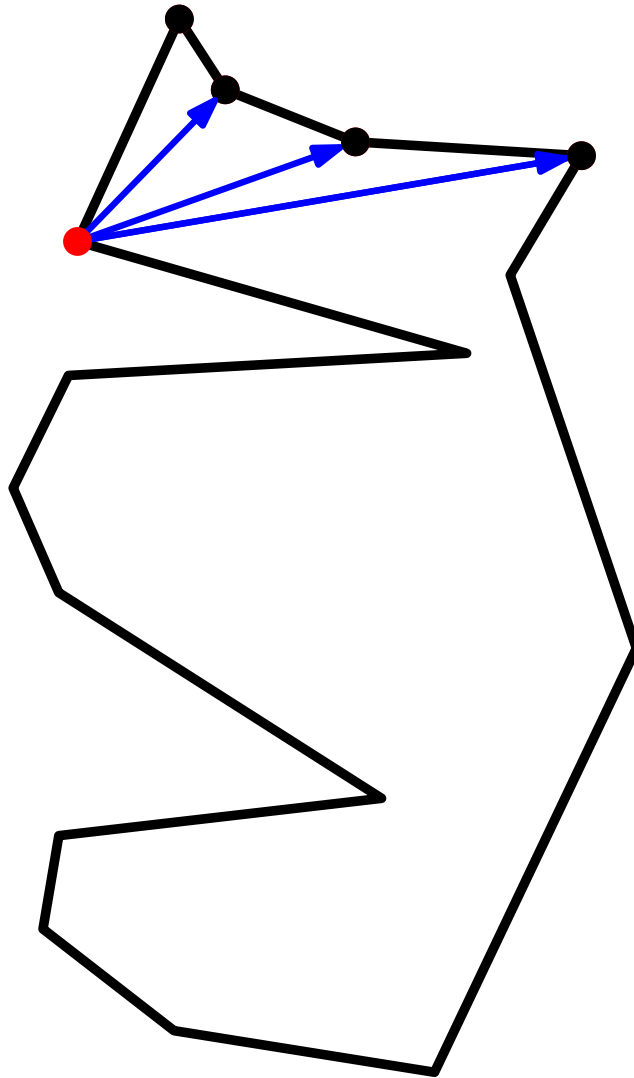
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

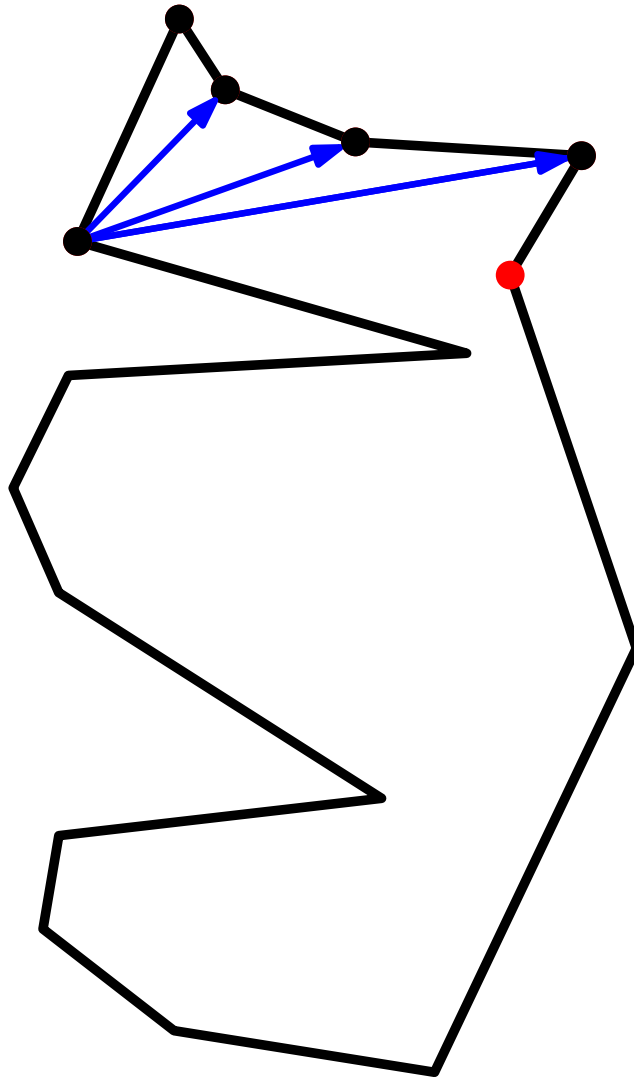
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

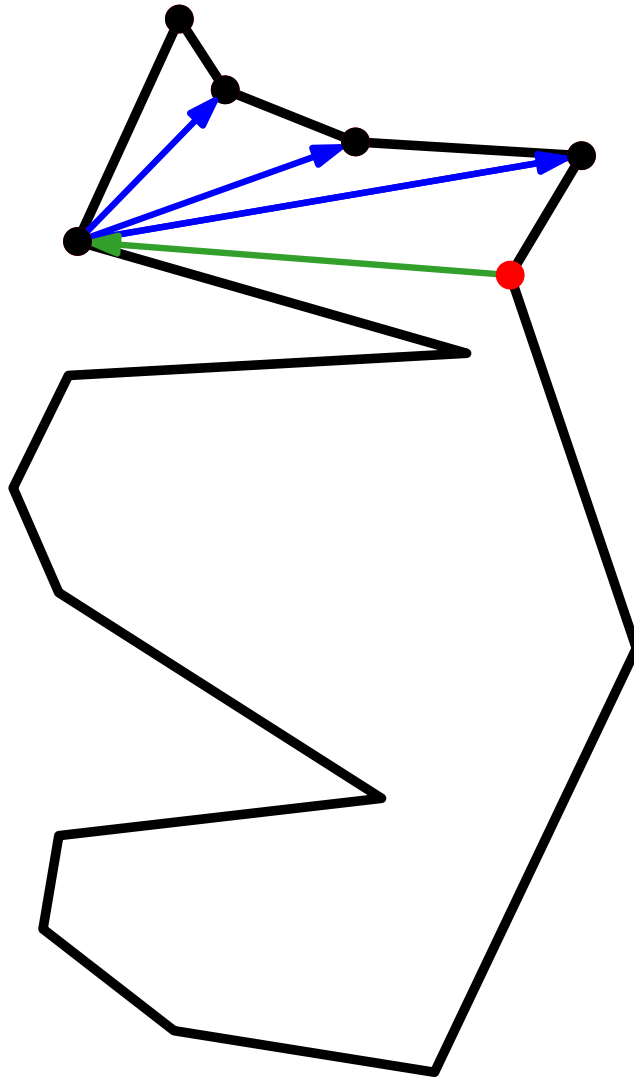
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

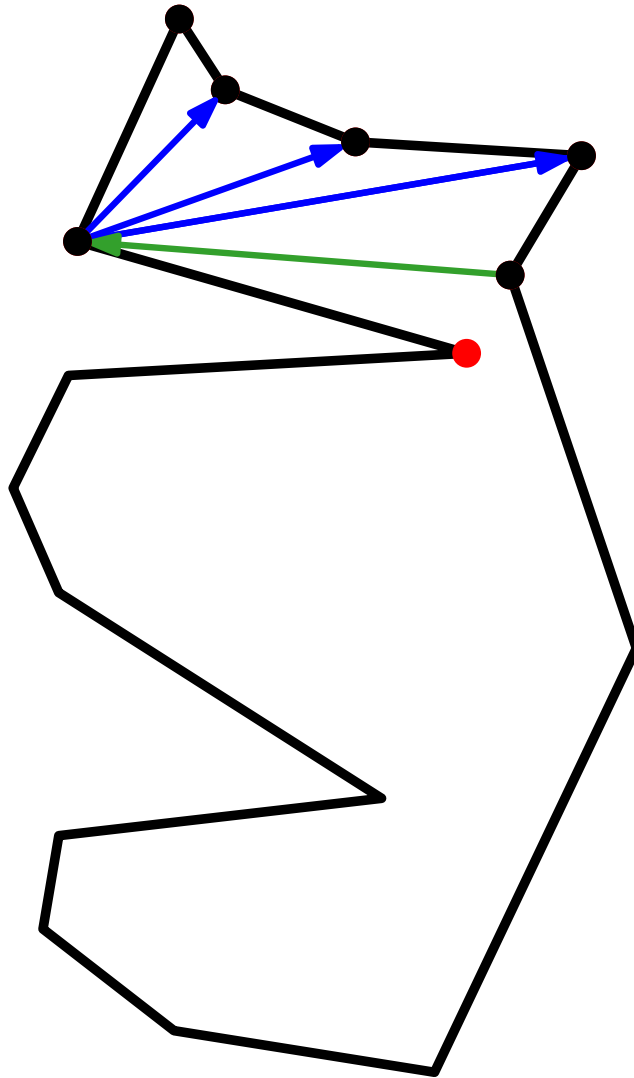
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

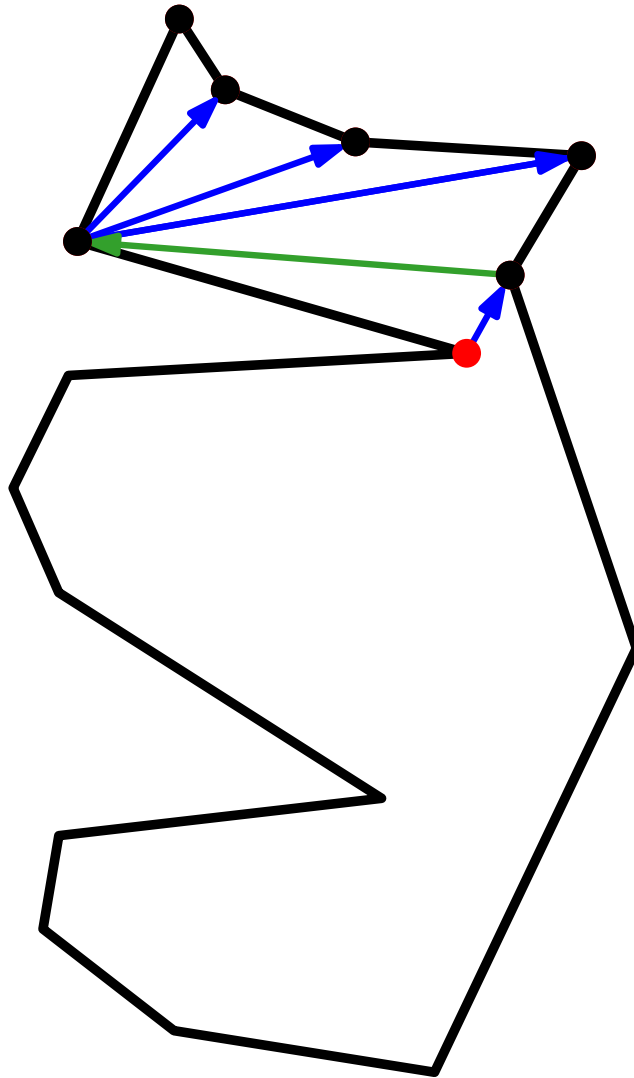
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

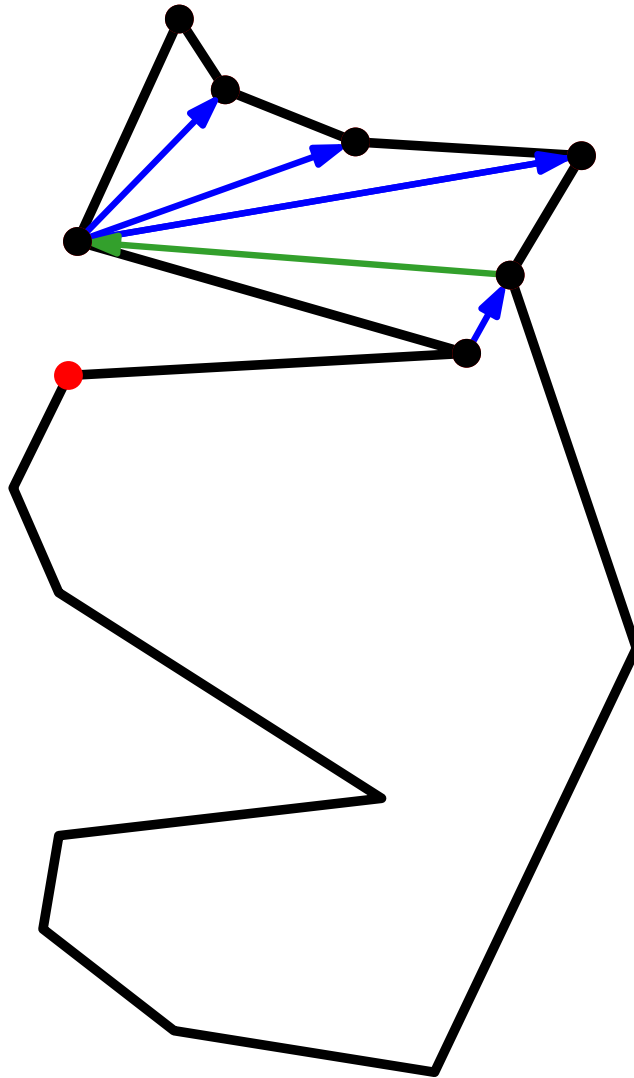
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

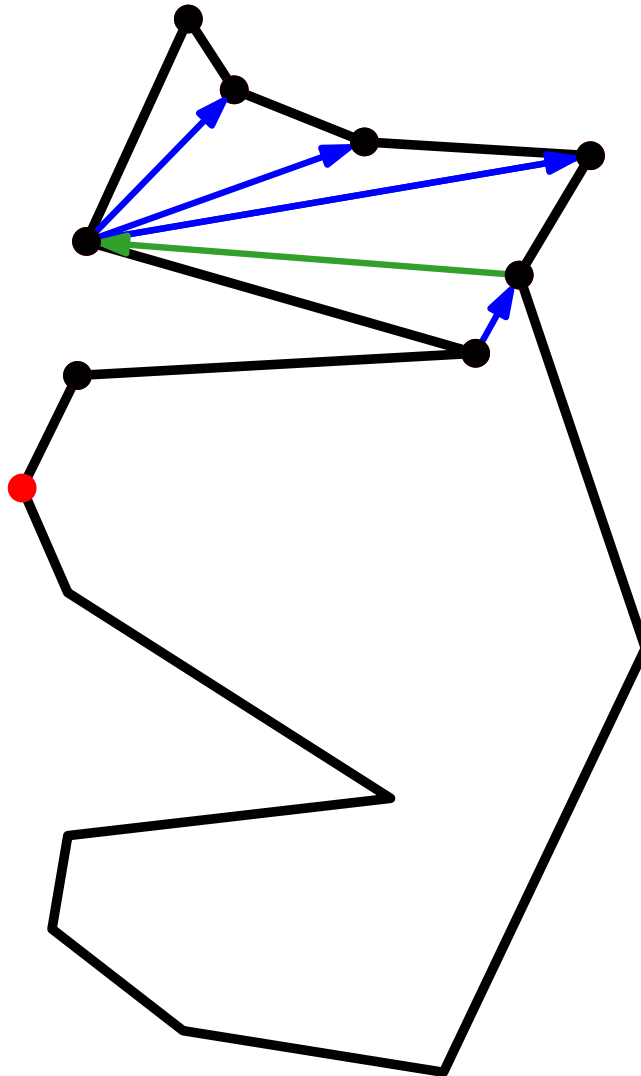
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

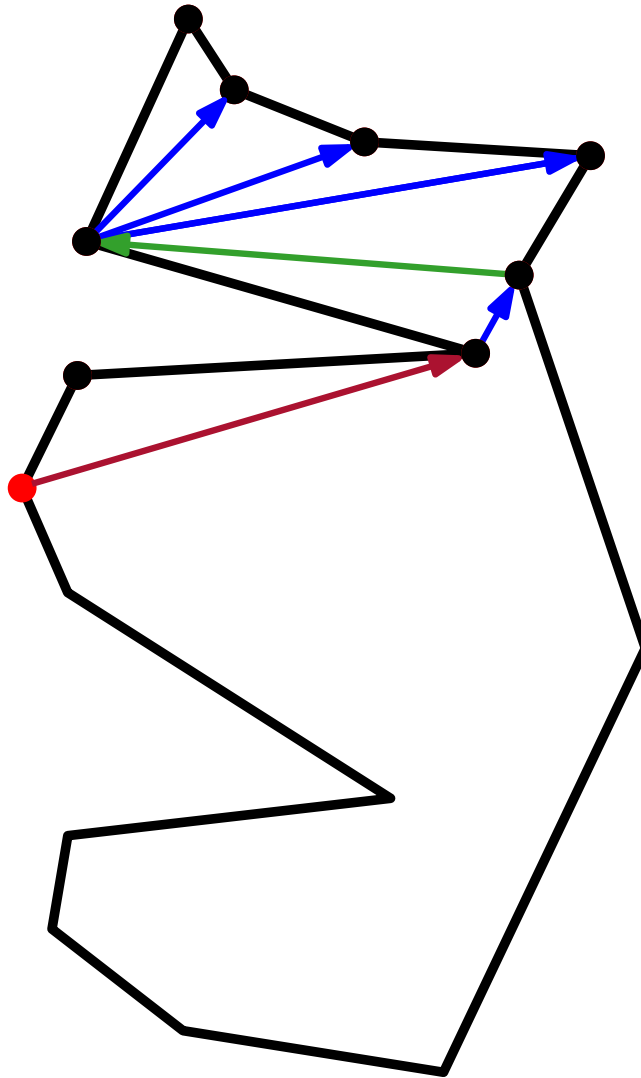
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

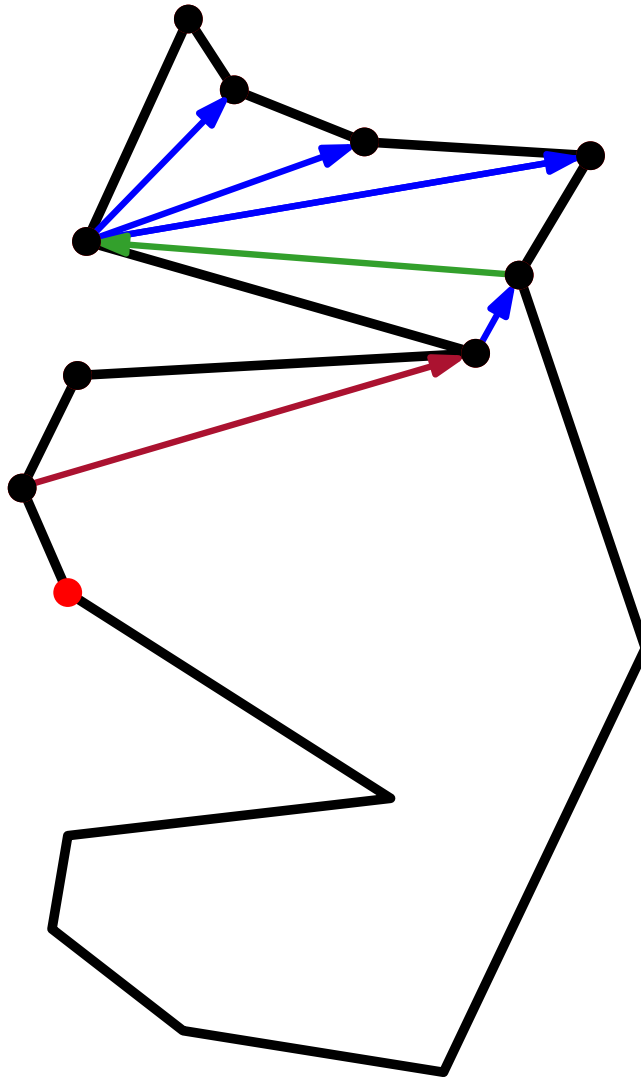
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

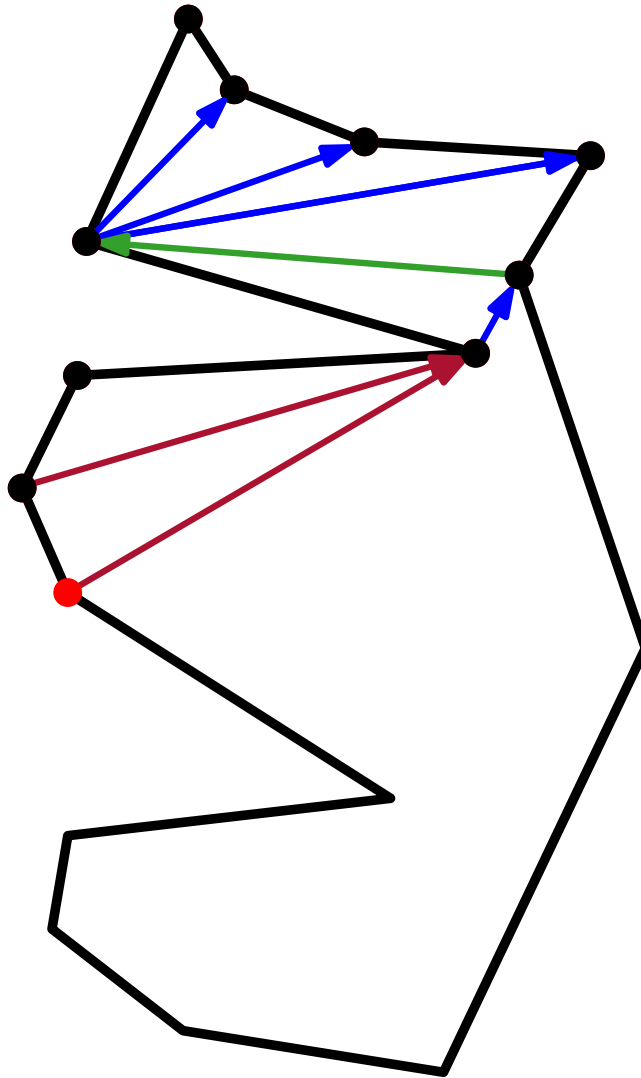
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

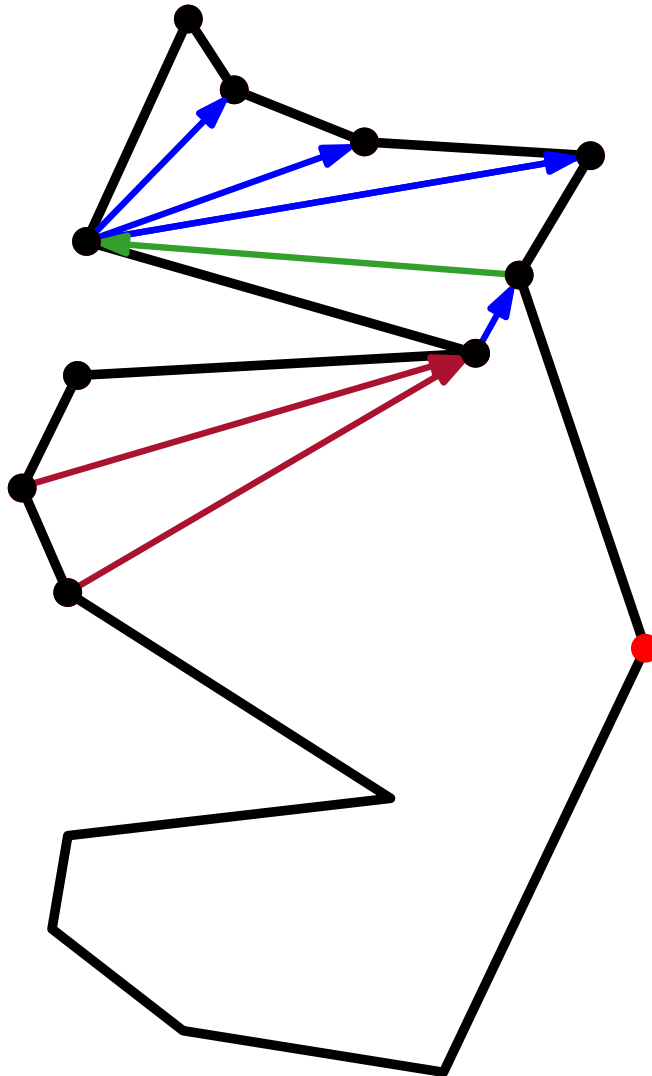
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

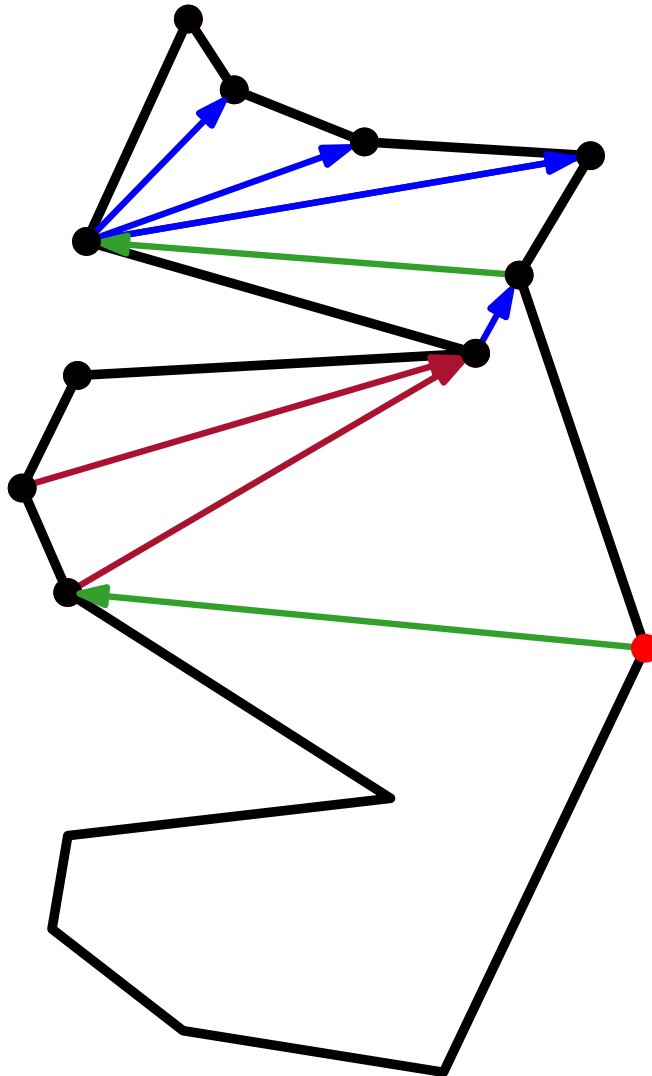
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

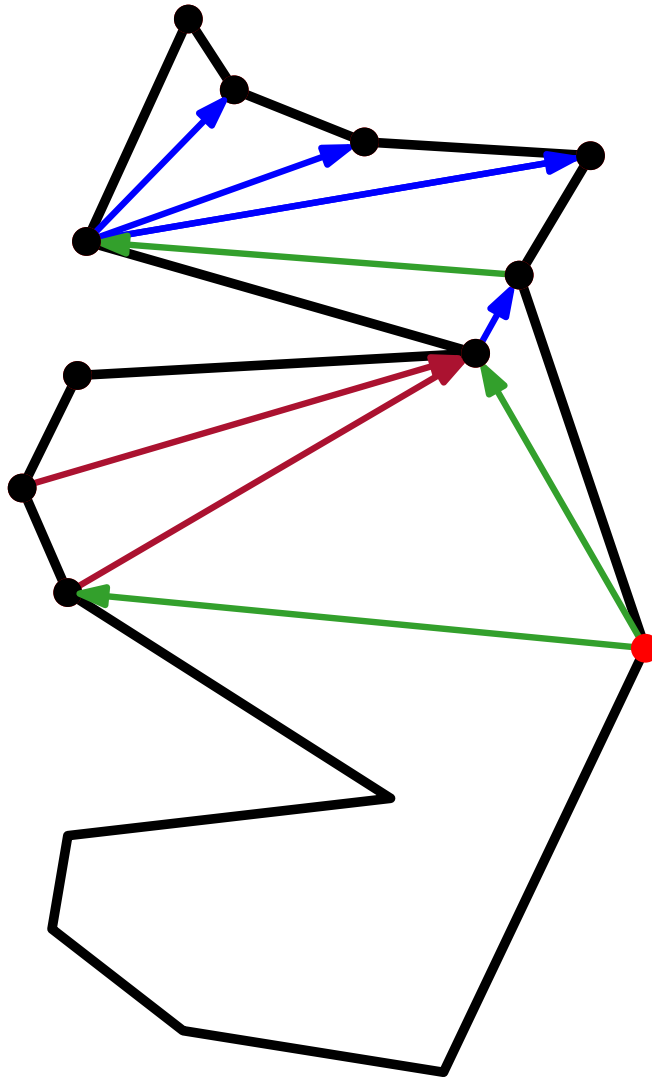
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

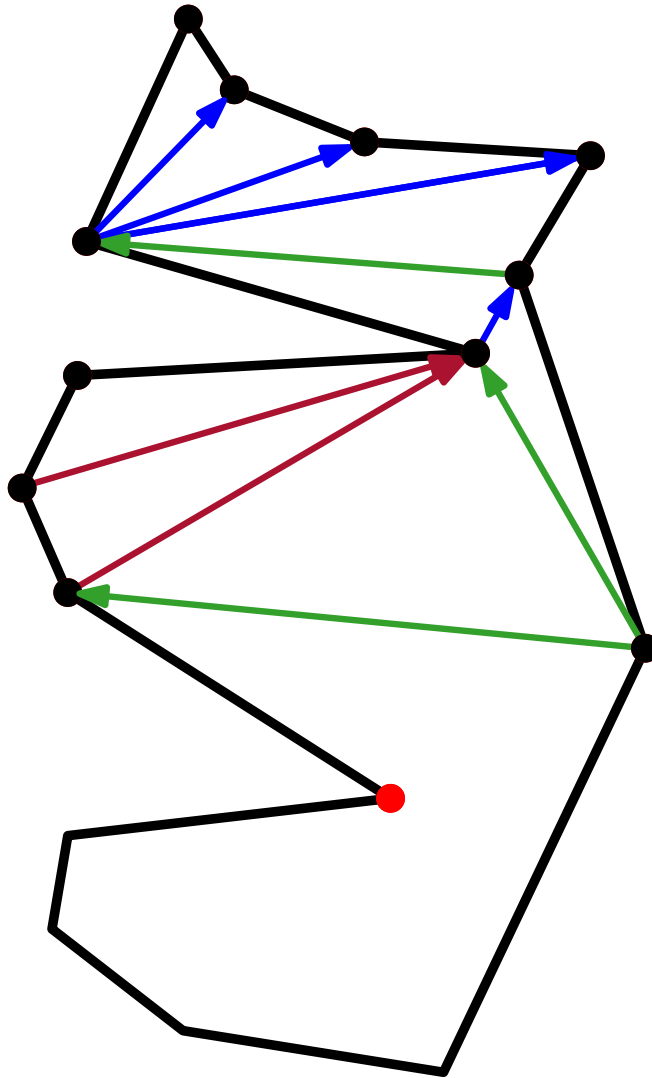
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

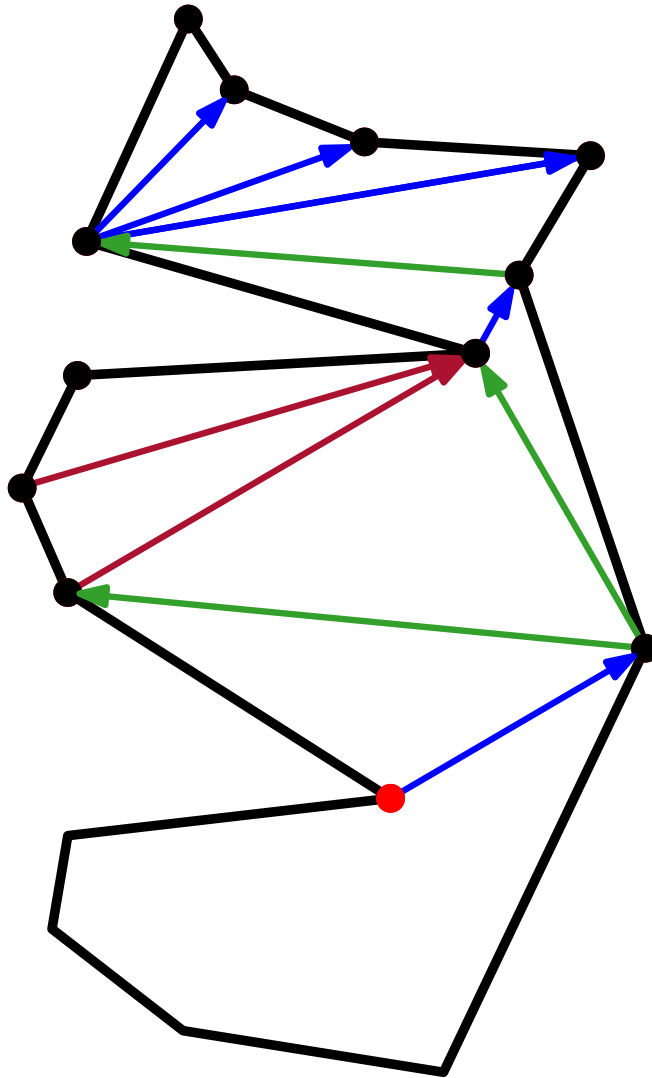
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

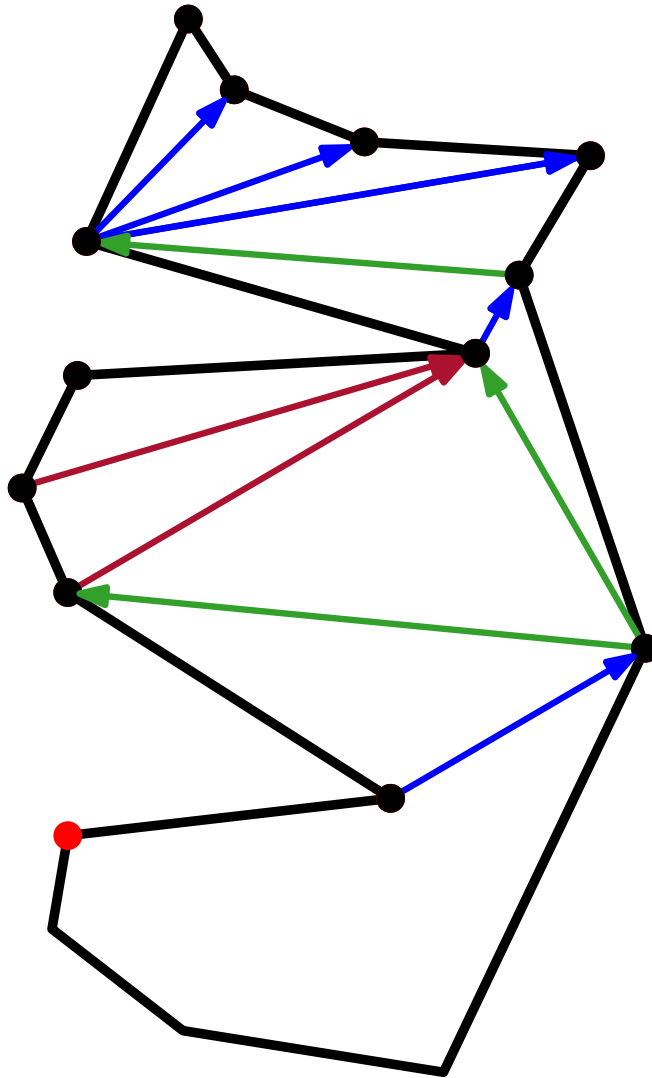
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

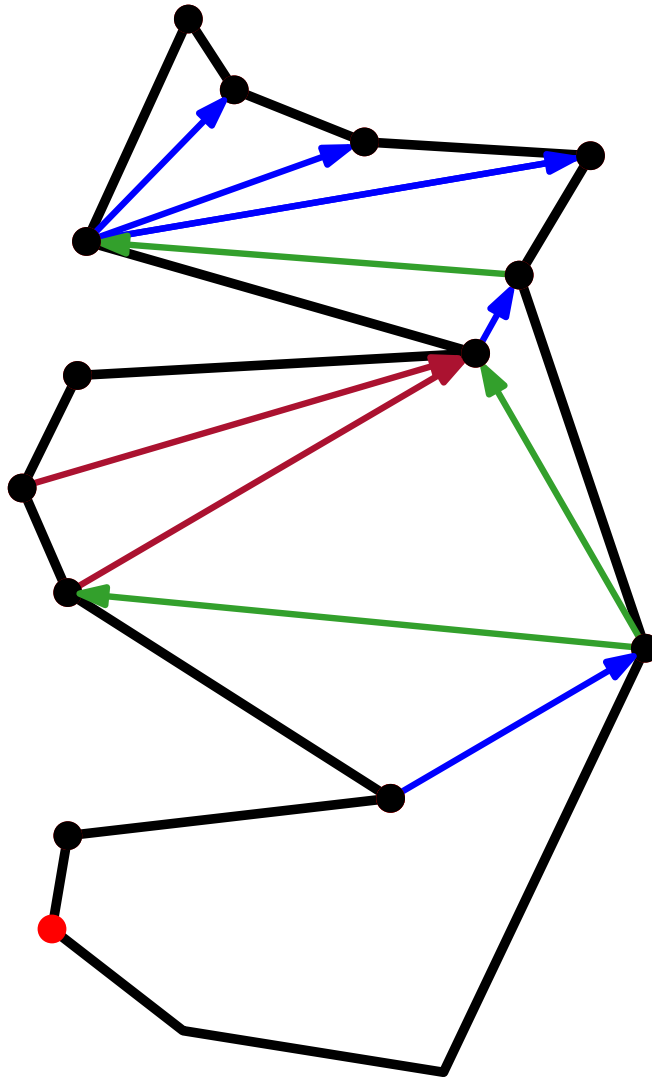
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

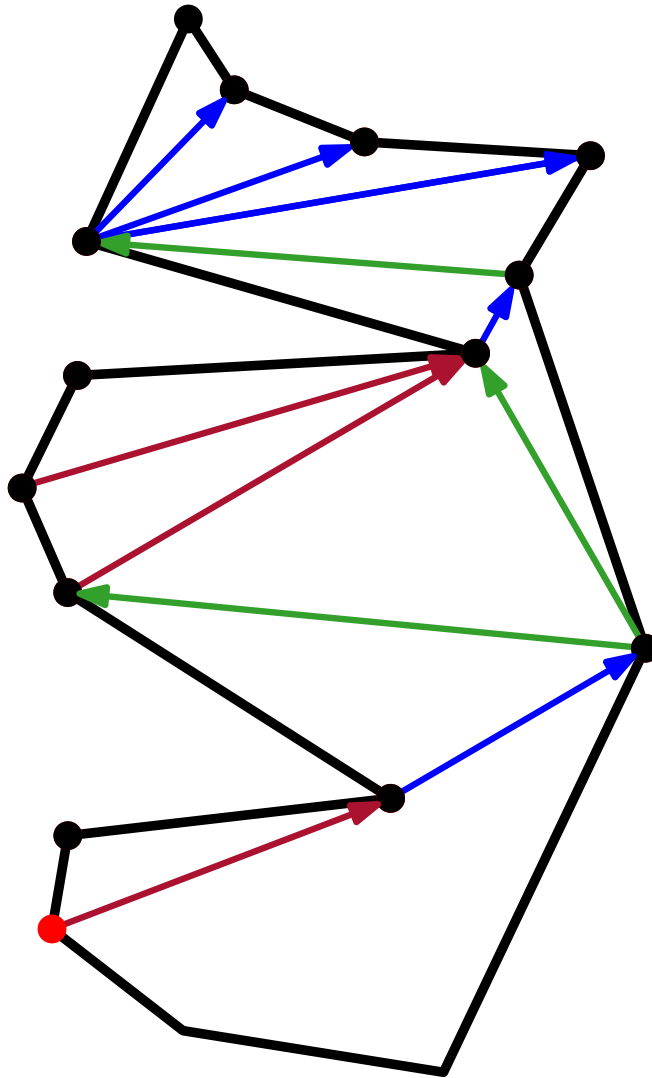
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

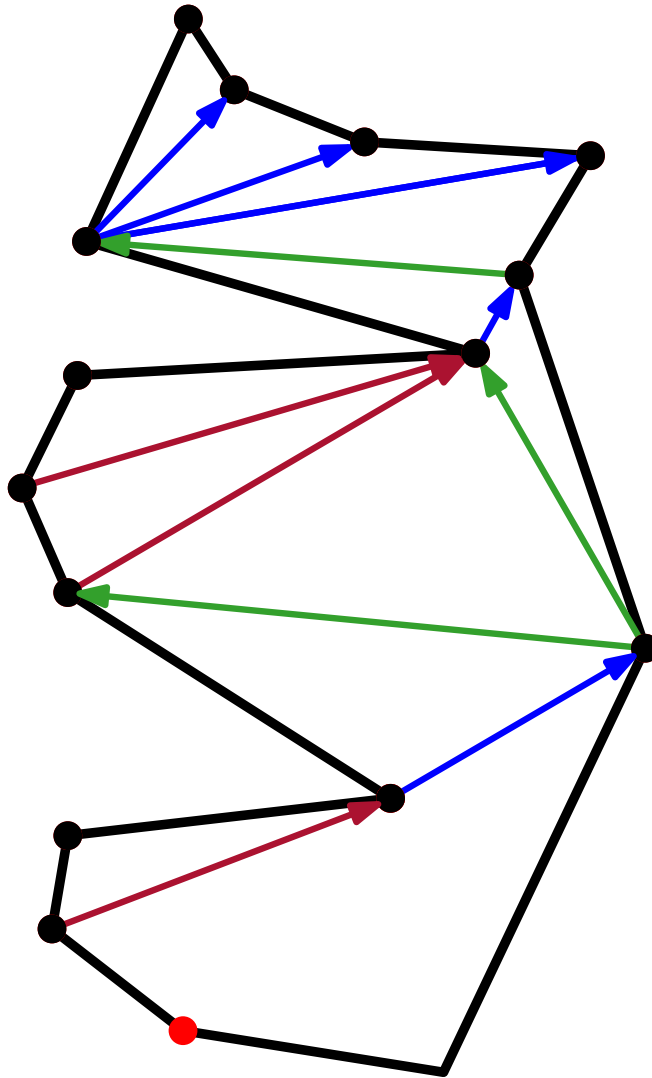
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

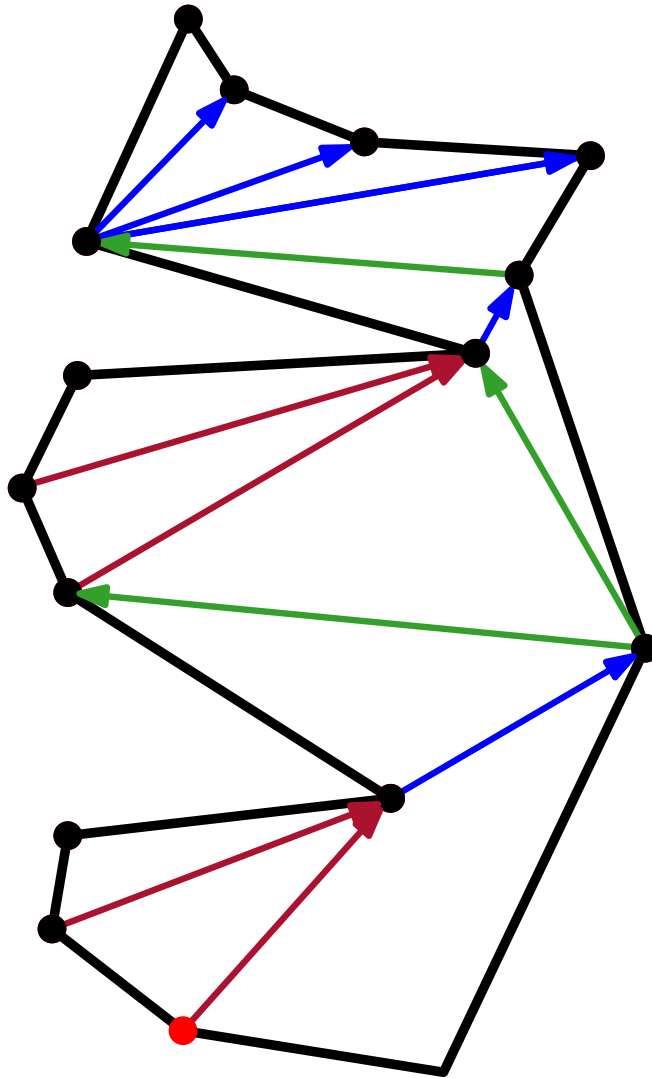
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

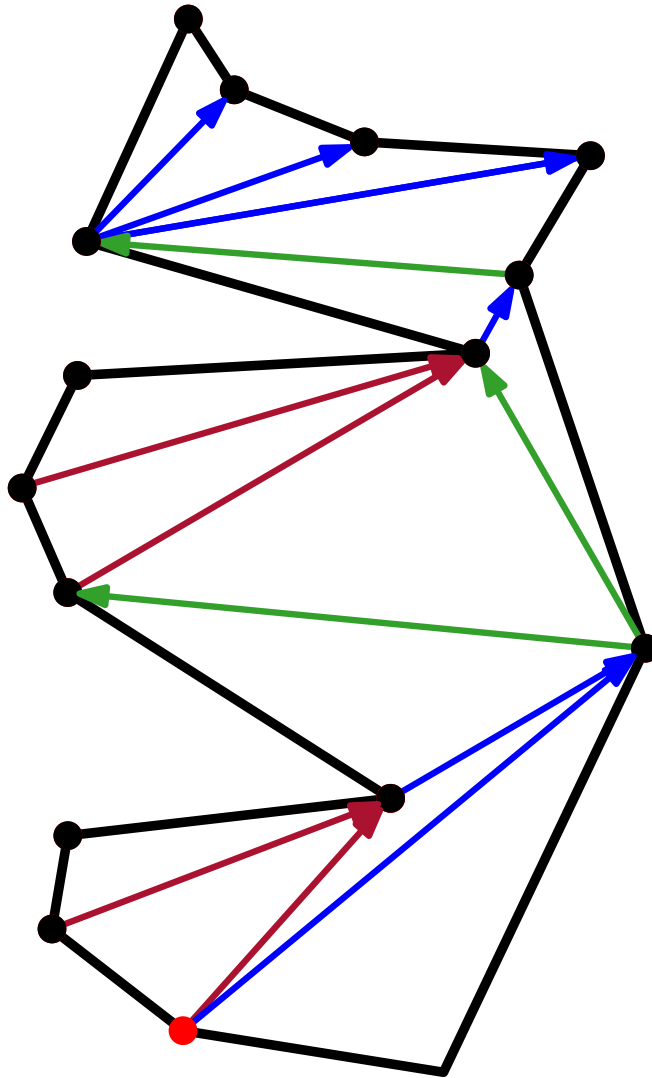
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

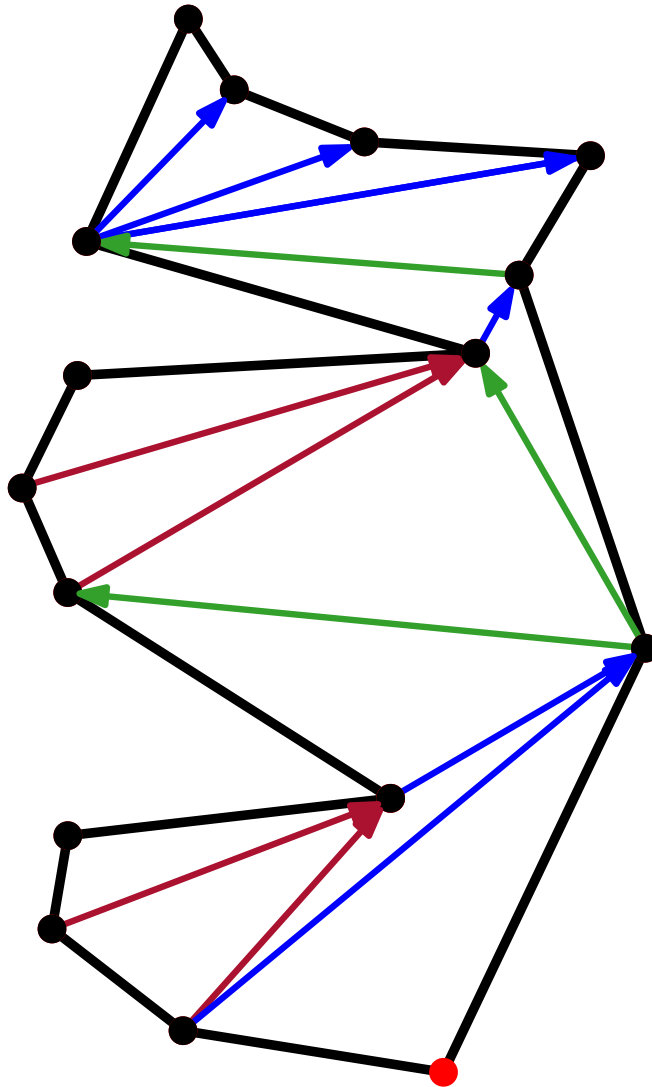
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

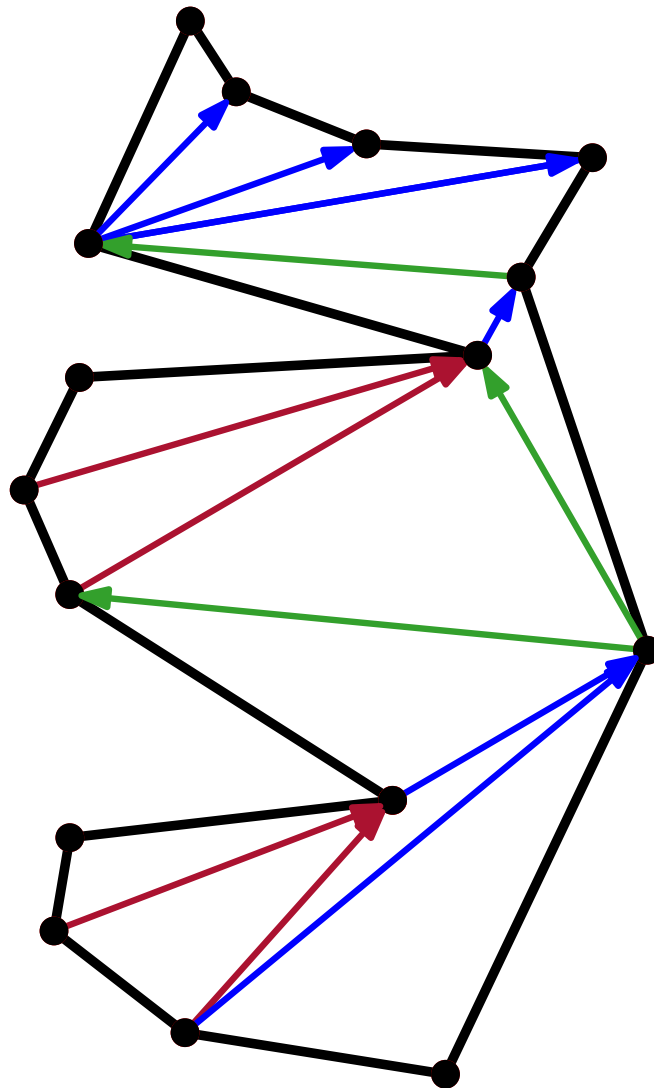
Approach: greedy top down traversal of both sides



Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

Approach: greedy top down traversal of both sides

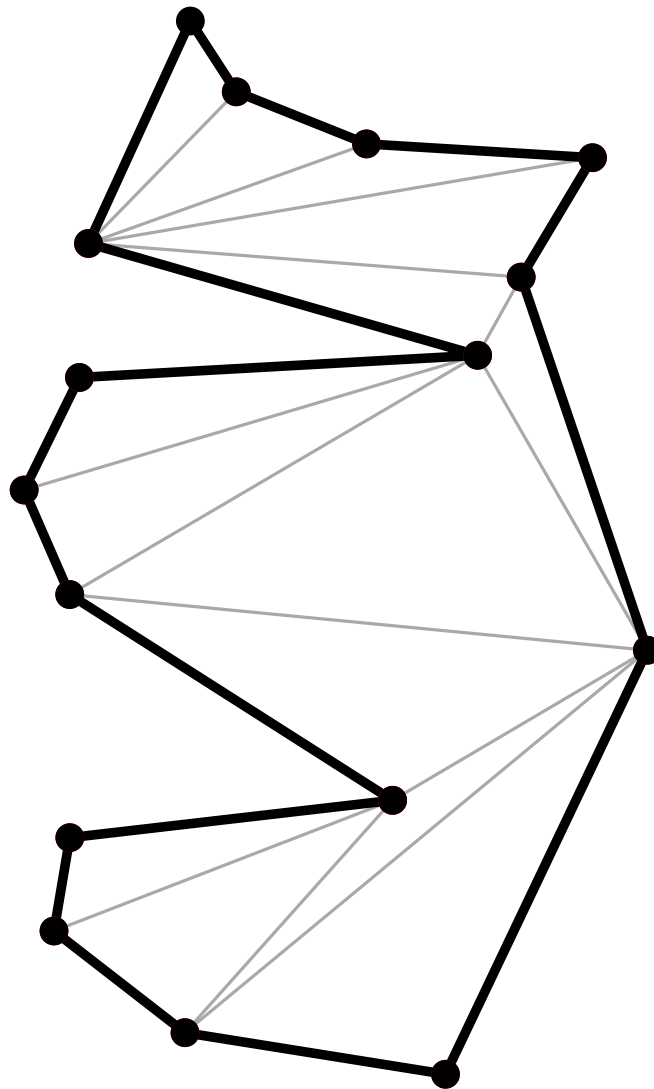


Invariant?

Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

Approach: greedy top down traversal of both sides

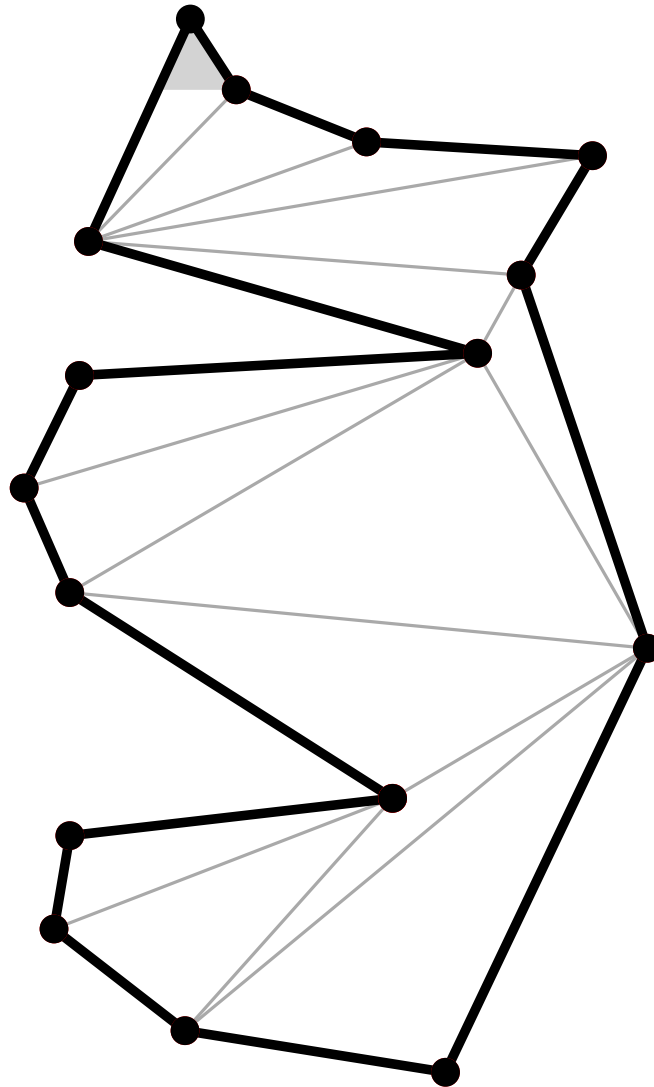


Invariant?

Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

Approach: greedy top down traversal of both sides

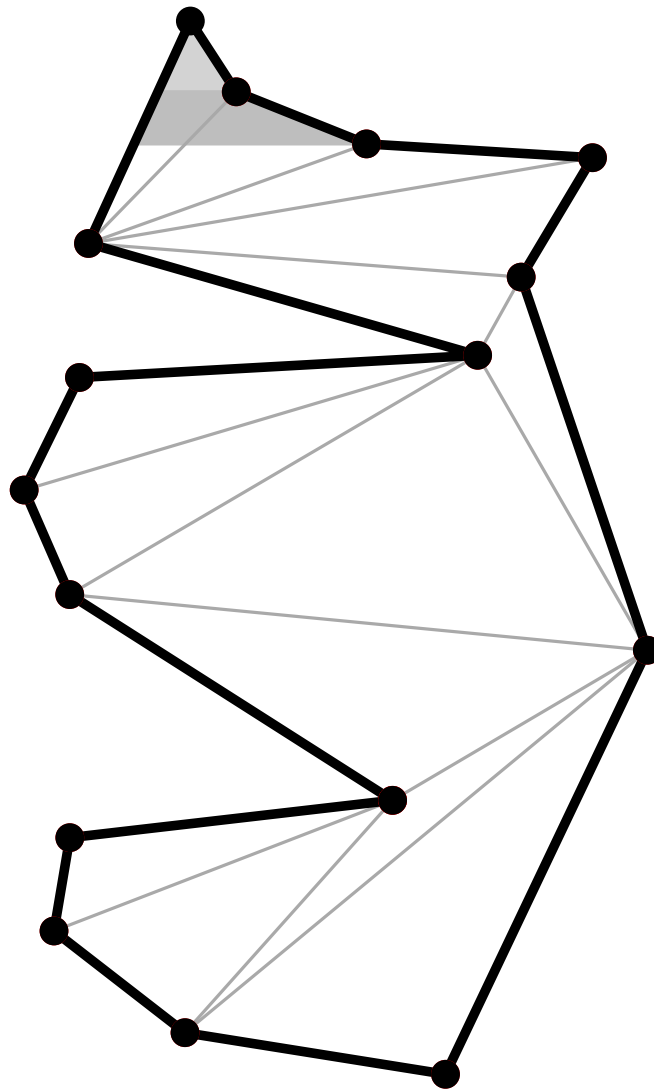


Invariant?

Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

Approach: greedy top down traversal of both sides

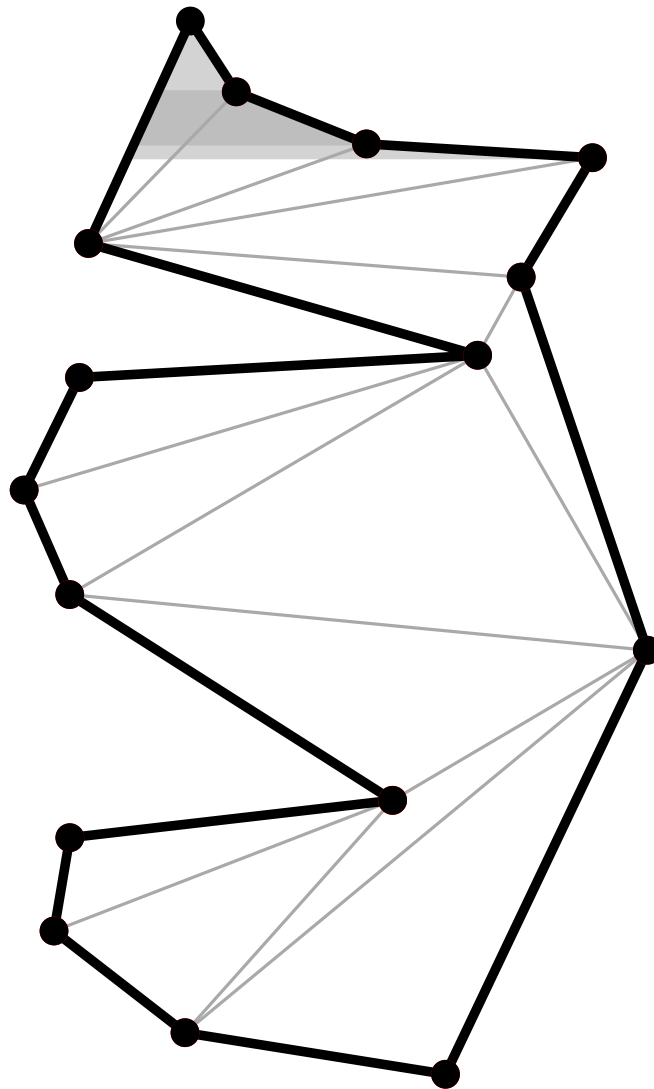


Invariant?

Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

Approach: greedy top down traversal of both sides

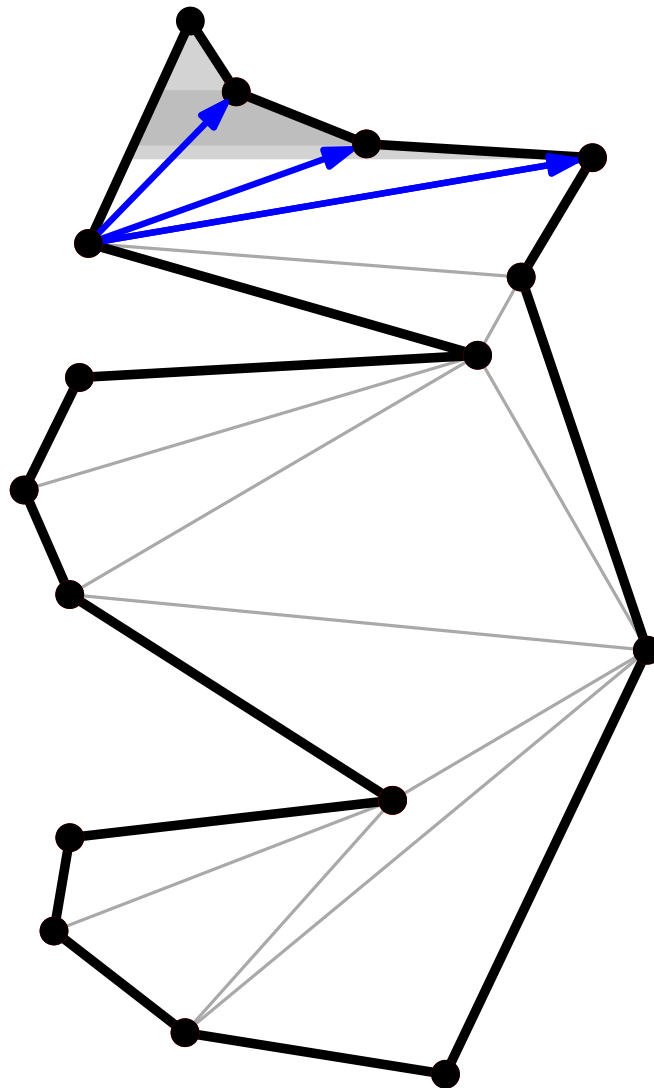


Invariant?

Triangulate y -monotone Polygons

Reminder: left and right boundary paths are descending

Approach: greedy top down traversal of both sides

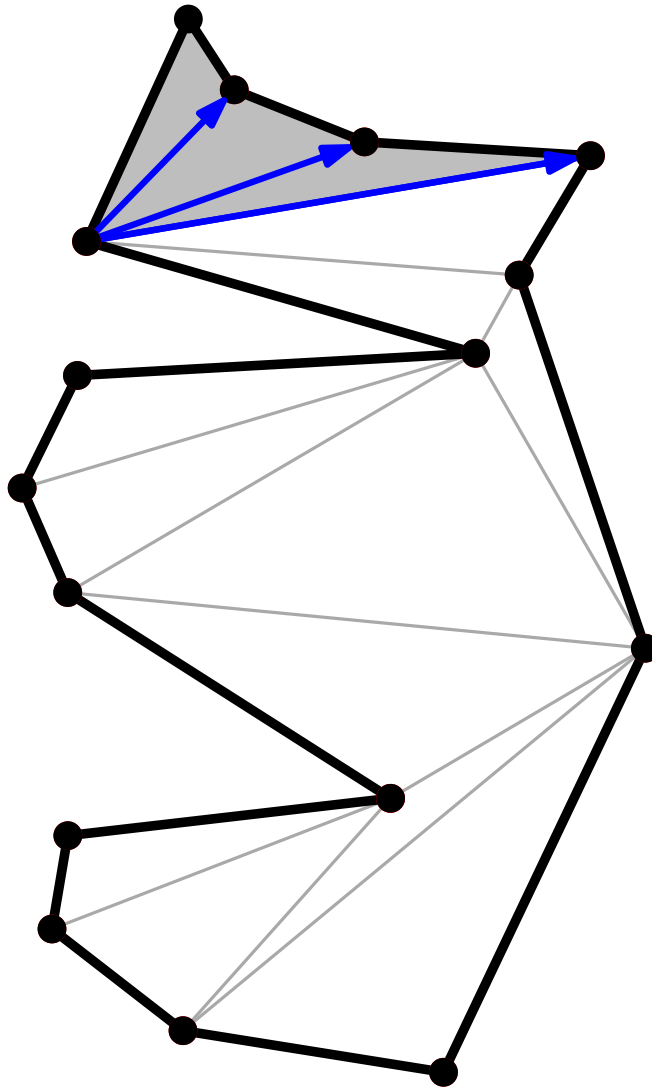


Invariant?

Triangulate y -monotone Polygons

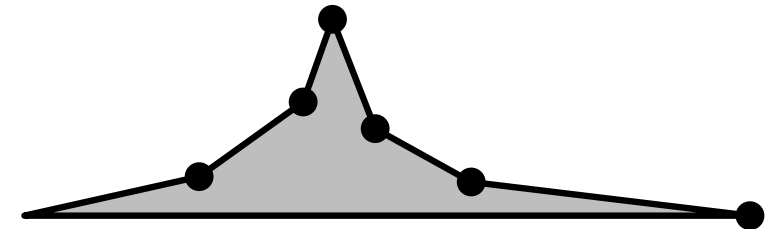
Reminder: left and right boundary paths are descending

Approach: greedy top down traversal of both sides



Invariant?

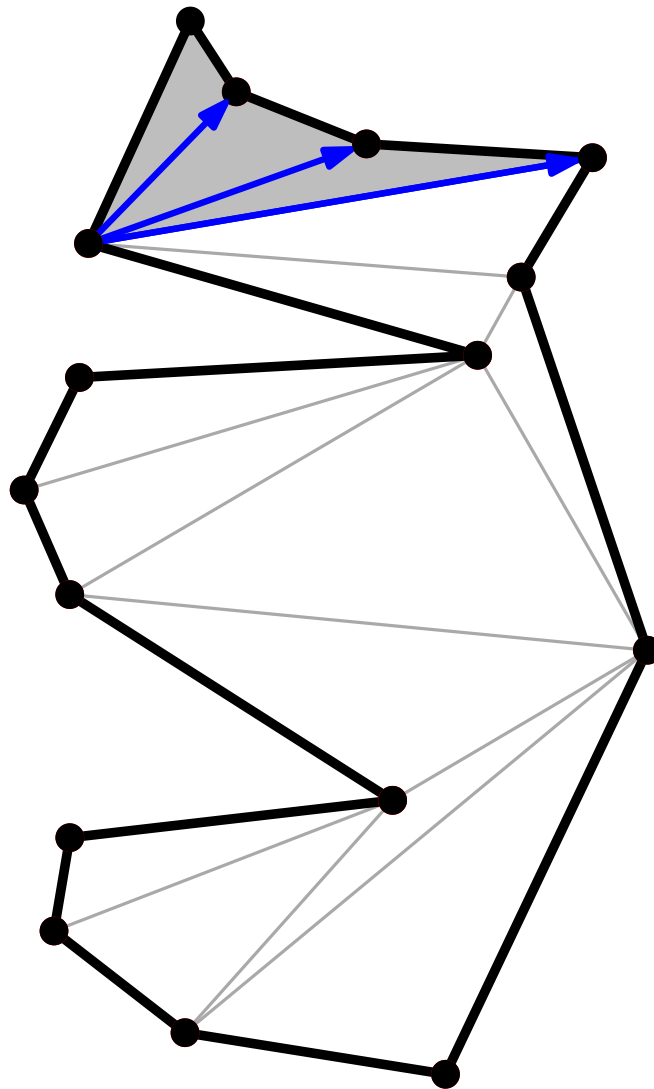
visited by not yet triangulated part of P is *funnel-shaped*.



Triangulate y -monotone Polygons

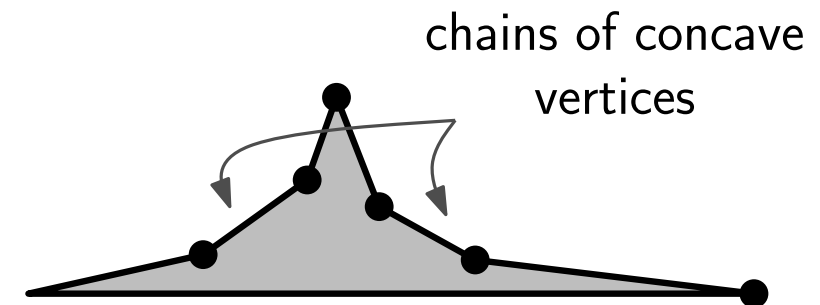
Reminder: left and right boundary paths are descending

Approach: greedy top down traversal of both sides



Invariant?

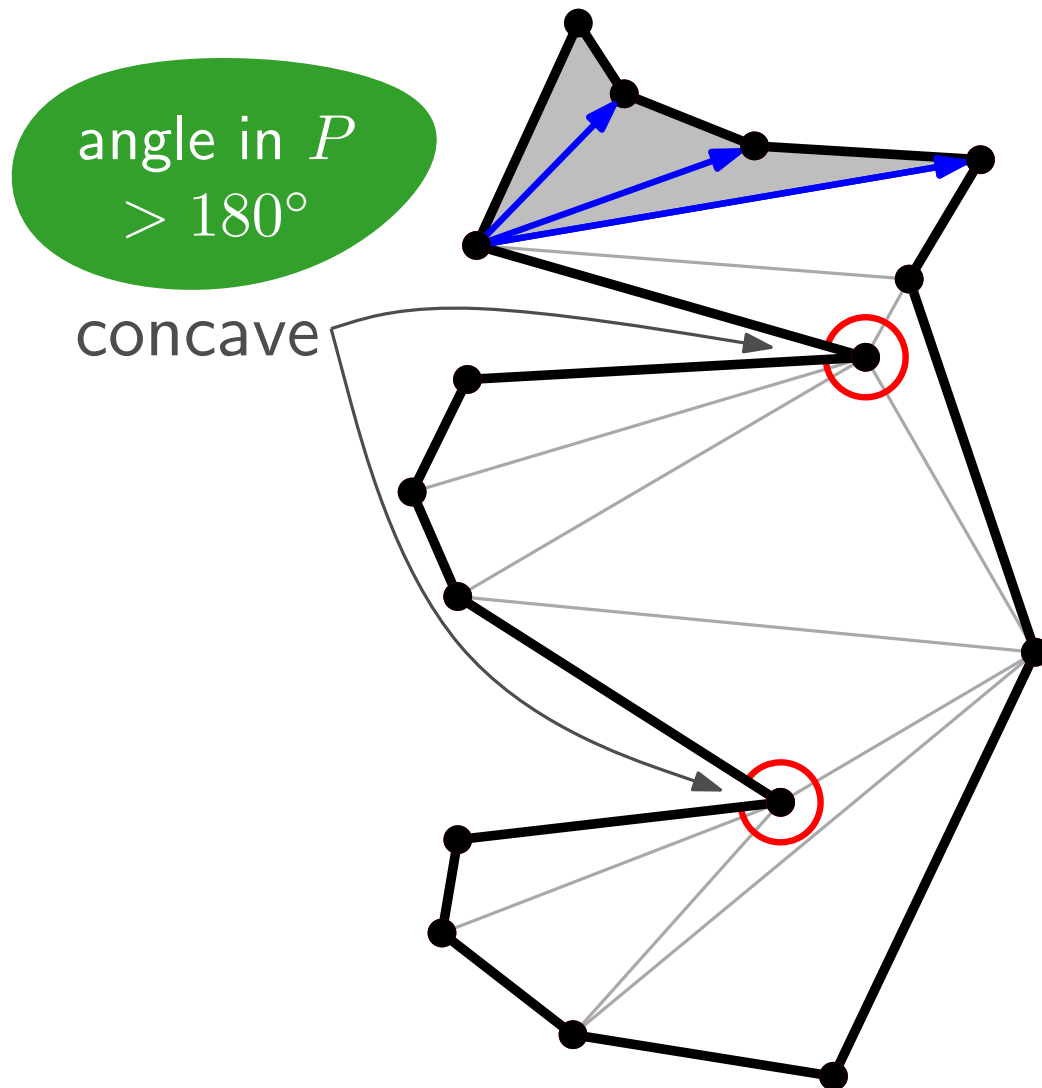
visited by not yet triangulated part of P is *funnel-shaped*.



Triangulate y -monotone Polygons

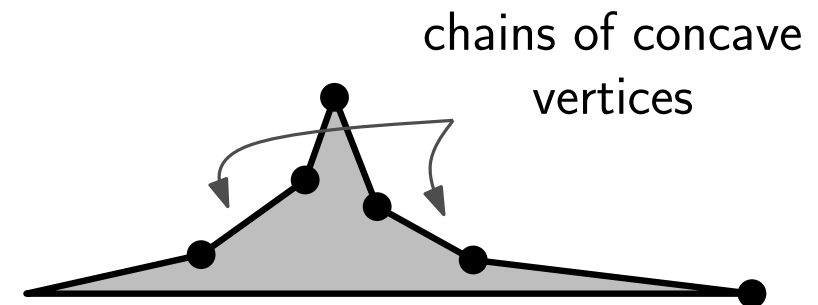
Reminder: left and right boundary paths are descending

Approach: greedy top down traversal of both sides



Invariant?

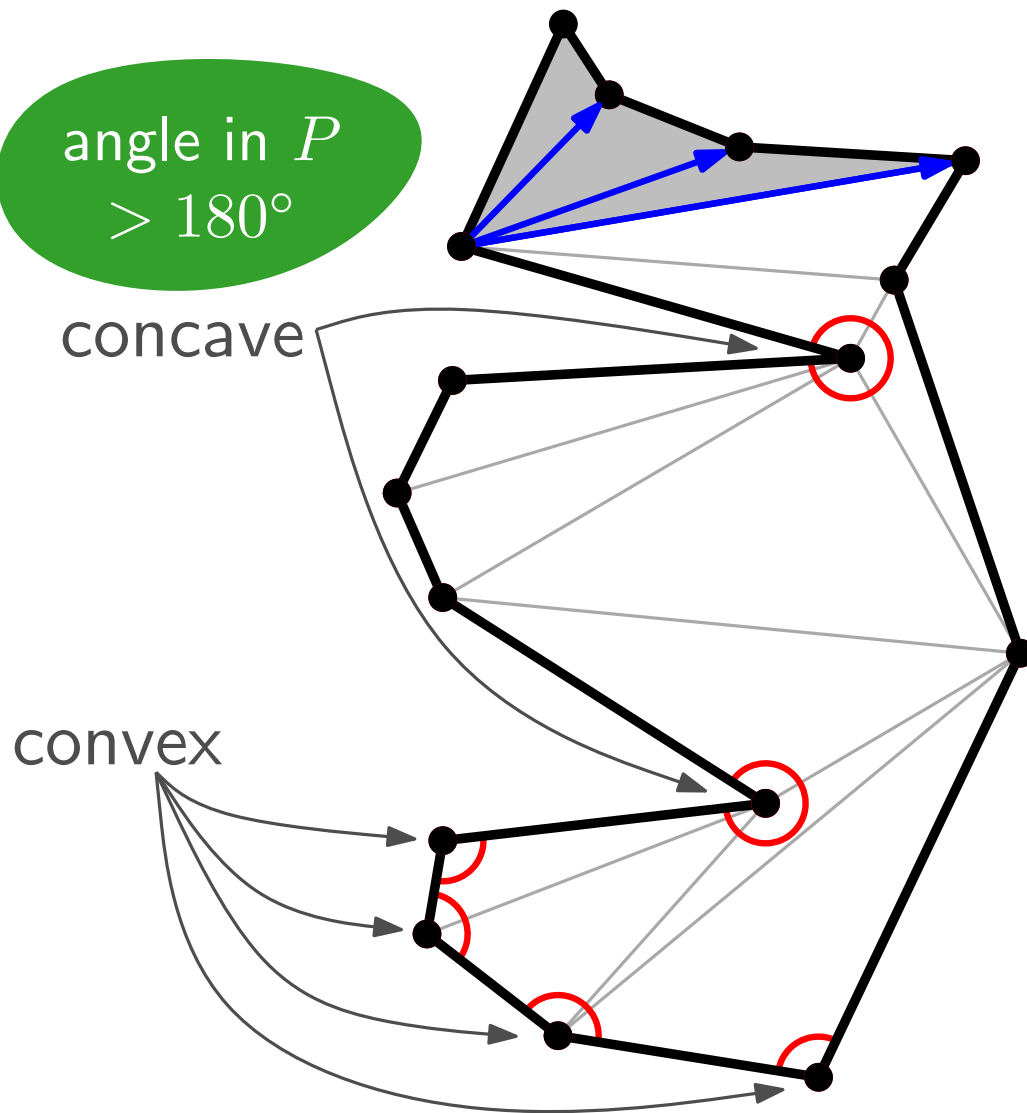
visited by not yet triangulated part of P is *funnel-shaped*.



Triangulate y -monotone Polygons

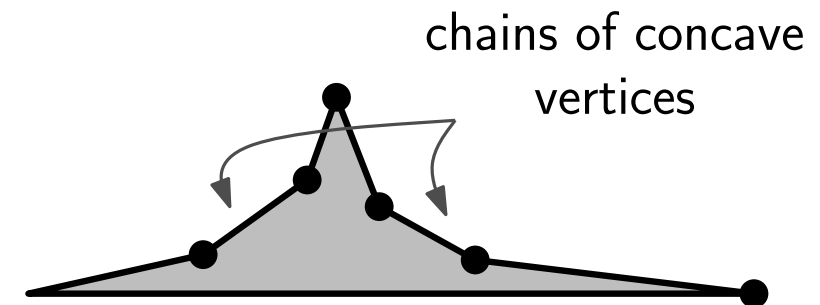
Reminder: left and right boundary paths are descending

Approach: greedy top down traversal of both sides



Invariant?

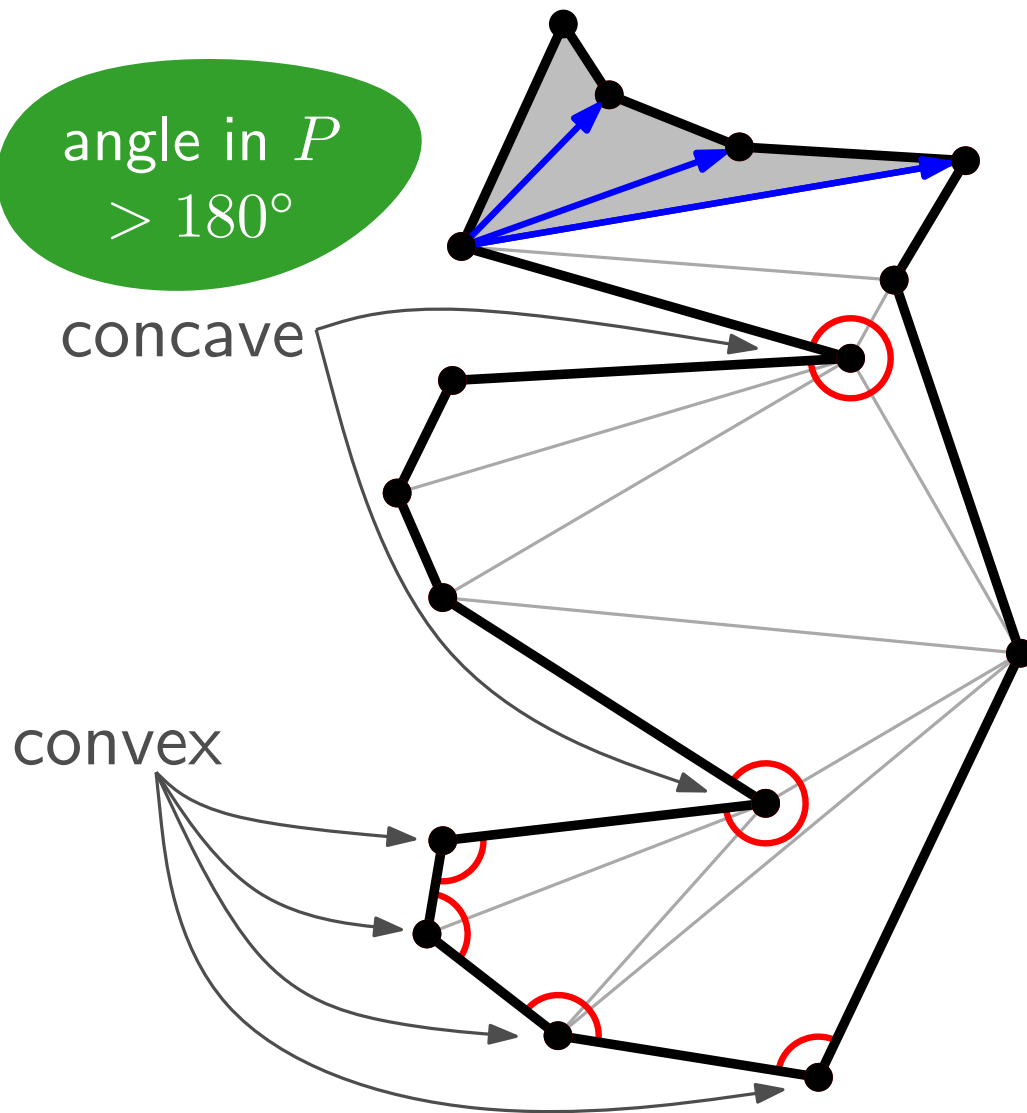
visited by not yet triangulated part of P is *funnel-shaped*.



Triangulate y -monotone Polygons

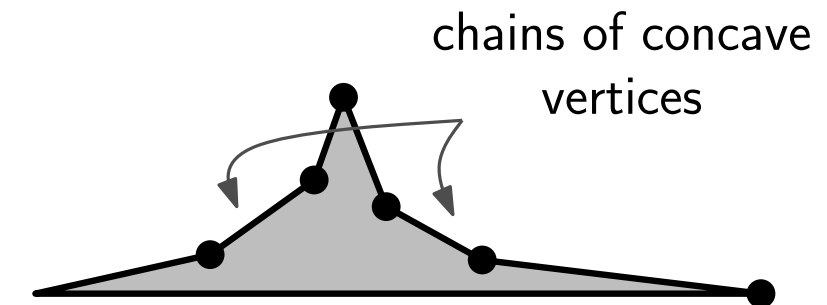
Reminder: left and right boundary paths are descending

Approach: greedy top down traversal of both sides

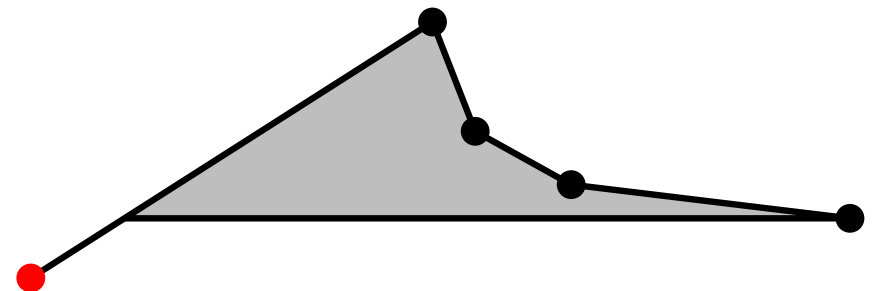


Invariant?

visited by not yet triangulated part of P is *funnel-shaped*.



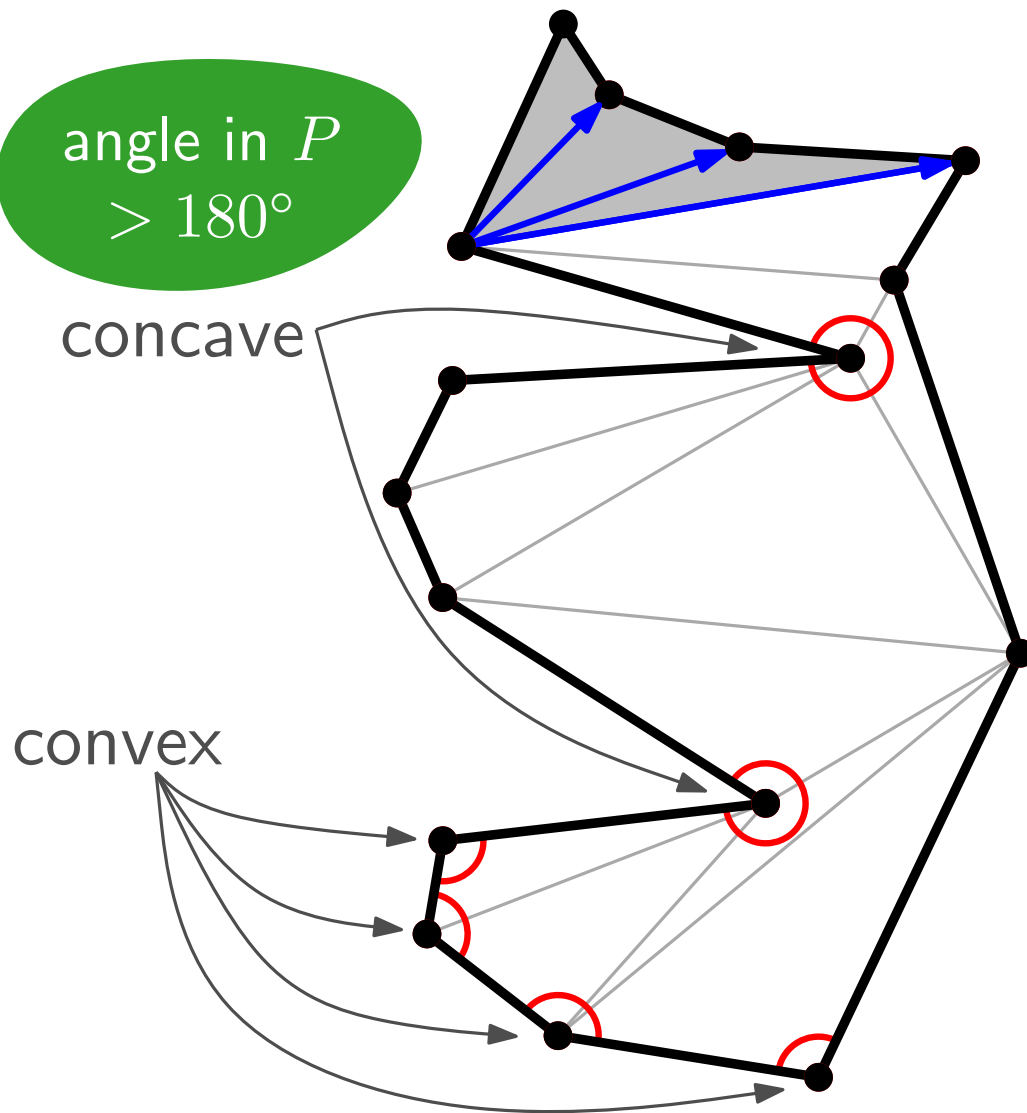
Here:



Triangulate y -monotone Polygons

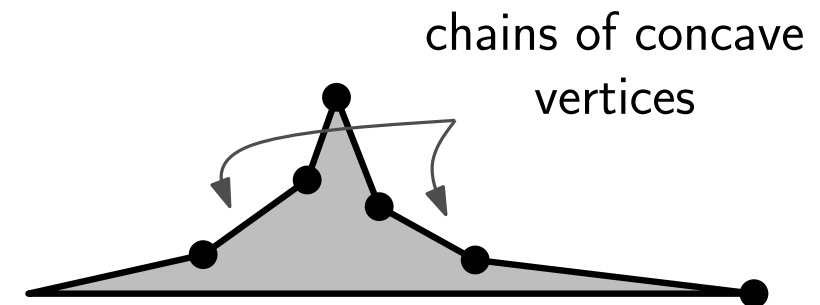
Reminder: left and right boundary paths are descending

Approach: greedy top down traversal of both sides

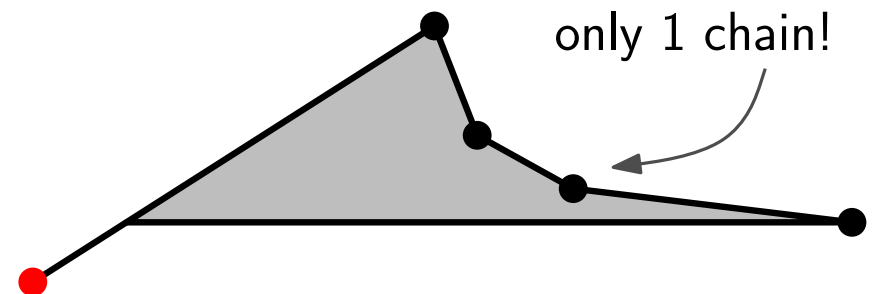


Invariant?

visited by not yet triangulated part of P is *funnel-shaped*.



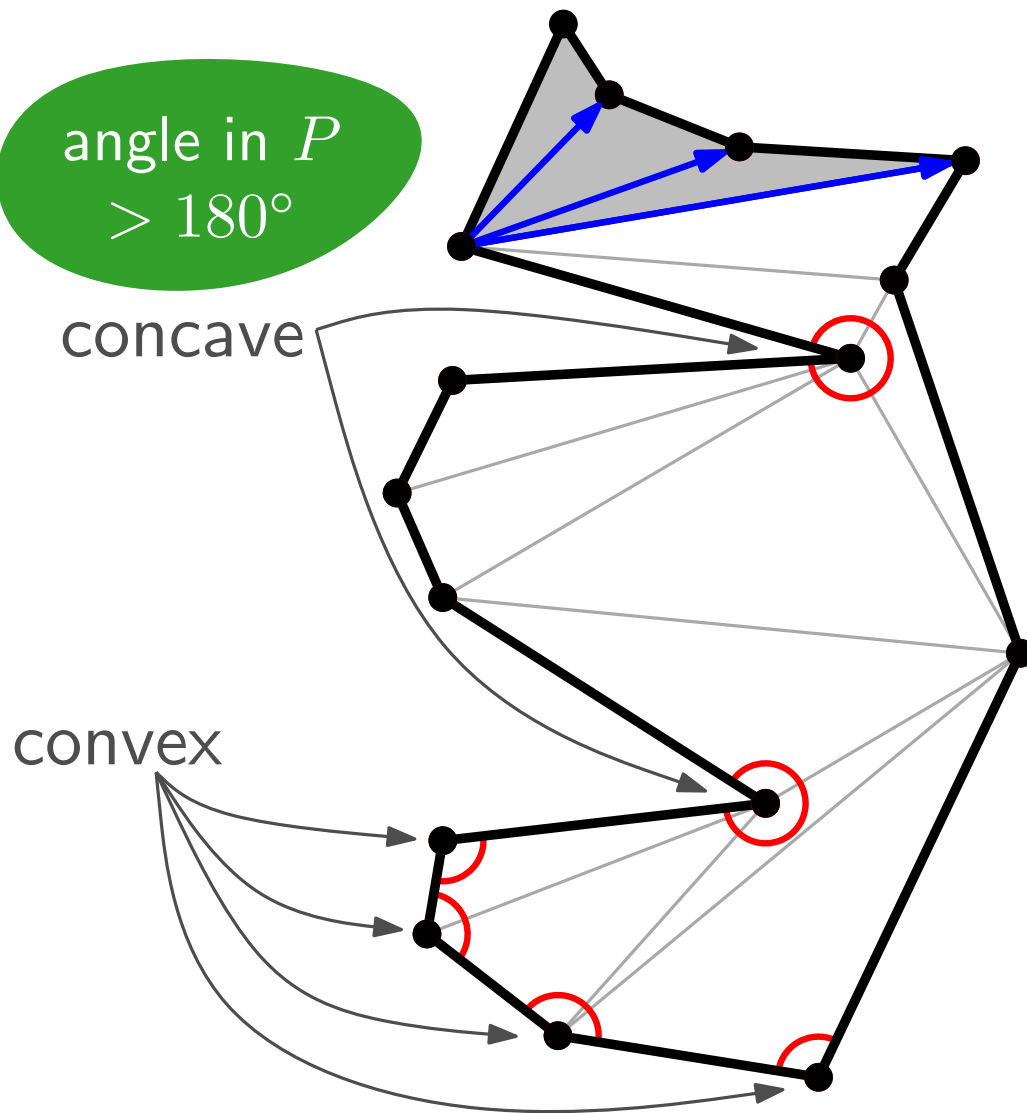
Here:



Triangulate y -monotone Polygons

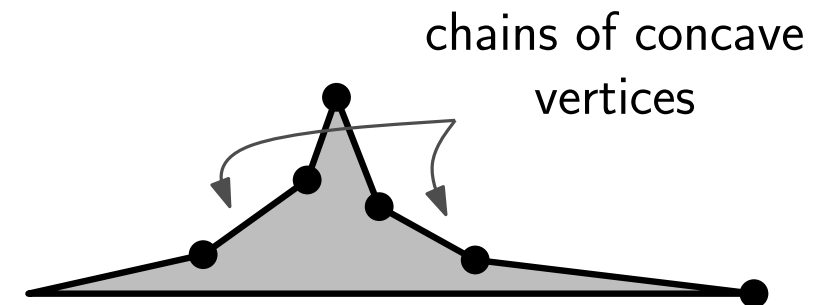
Reminder: left and right boundary paths are descending

Approach: greedy top down traversal of both sides

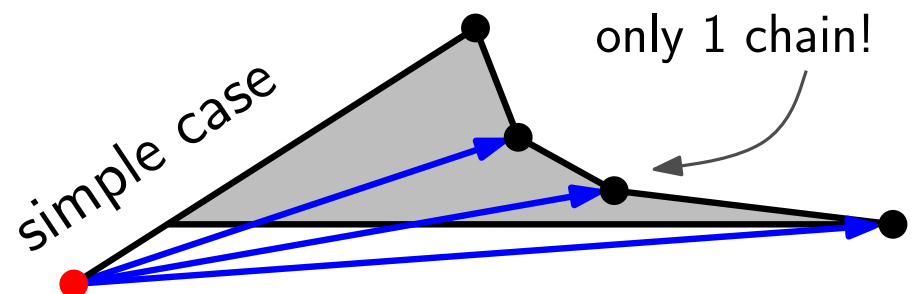


Invariant?

visited by not yet triangulated part of P is *funnel-shaped*.



Here:



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.\text{push}(u_1)$; $S.\text{push}(u_2)$

Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.\text{push}(u_1)$; $S.\text{push}(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.\text{top}()$ on different paths **then**

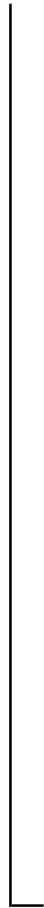
while not $S.\text{empty}()$ **do**

$v \leftarrow S.\text{pop}()$

if not $S.\text{empty}()$ **then** draw (u_j, v)

$S.\text{push}(u_{j-1})$; $S.\text{push}(u_j)$

else



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.push(u_1)$; $S.push(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.top()$ on different paths **then**

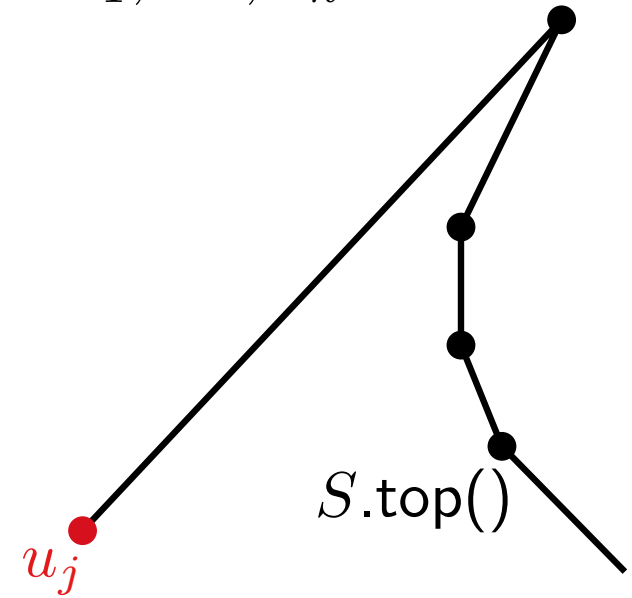
while not $S.empty()$ **do**

$v \leftarrow S.pop()$

if not $S.empty()$ **then** draw (u_j, v)

$S.push(u_{j-1})$; $S.push(u_j)$

else



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.\text{push}(u_1)$; $S.\text{push}(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.\text{top}()$ on different paths **then**

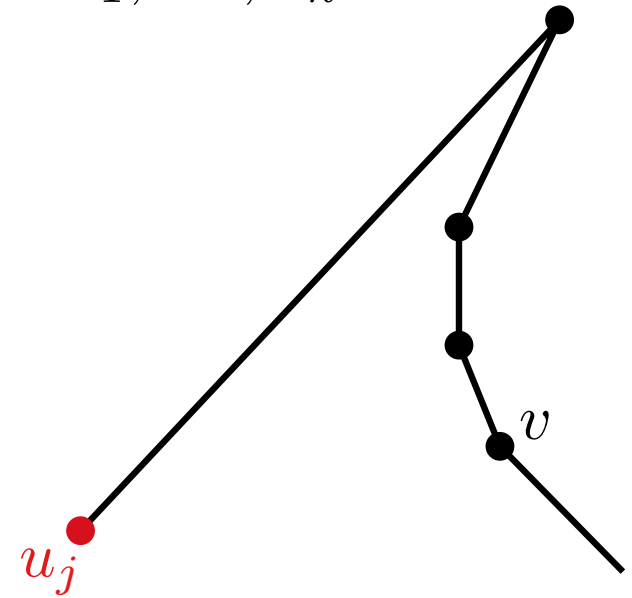
while not $S.\text{empty}()$ **do**

$v \leftarrow S.\text{pop}()$

if not $S.\text{empty}()$ **then** draw (u_j, v)

$S.\text{push}(u_{j-1})$; $S.\text{push}(u_j)$

else



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.\text{push}(u_1)$; $S.\text{push}(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.\text{top}()$ on different paths **then**

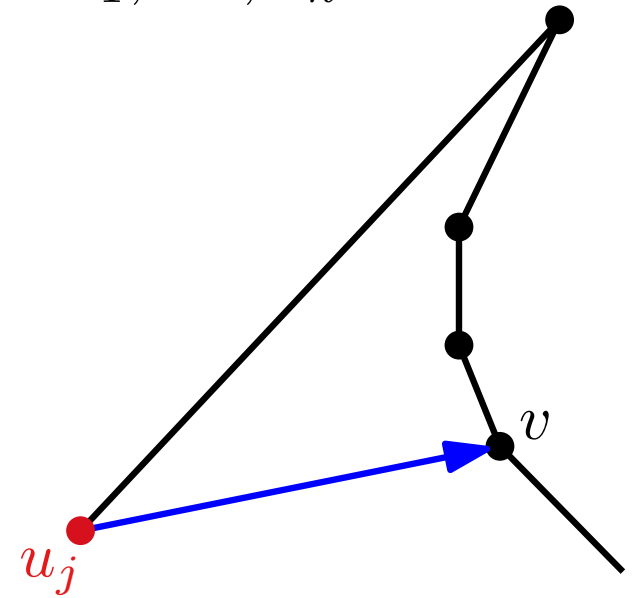
while not $S.\text{empty}()$ **do**

$v \leftarrow S.\text{pop}()$

if not $S.\text{empty}()$ **then** draw (u_j, v)

$S.\text{push}(u_{j-1})$; $S.\text{push}(u_j)$

else



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.\text{push}(u_1)$; $S.\text{push}(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.\text{top}()$ on different paths **then**

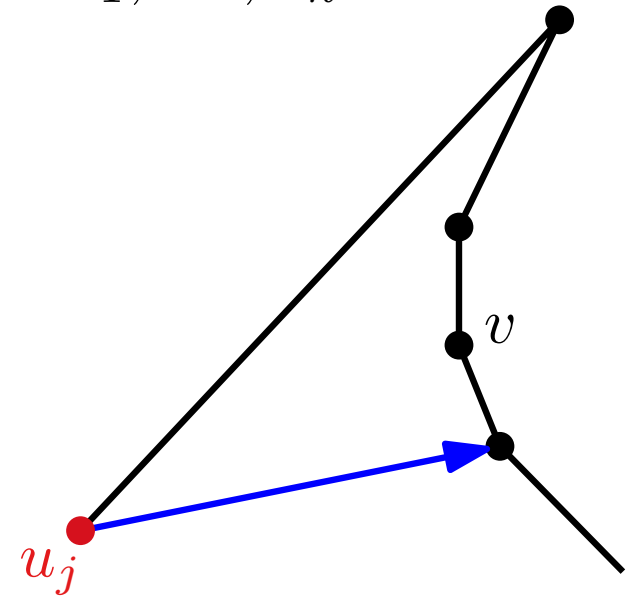
while not $S.\text{empty}()$ **do**

$v \leftarrow S.\text{pop}()$

if not $S.\text{empty}()$ **then** draw (u_j, v)

$S.\text{push}(u_{j-1})$; $S.\text{push}(u_j)$

else



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.\text{push}(u_1)$; $S.\text{push}(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.\text{top}()$ on different paths **then**

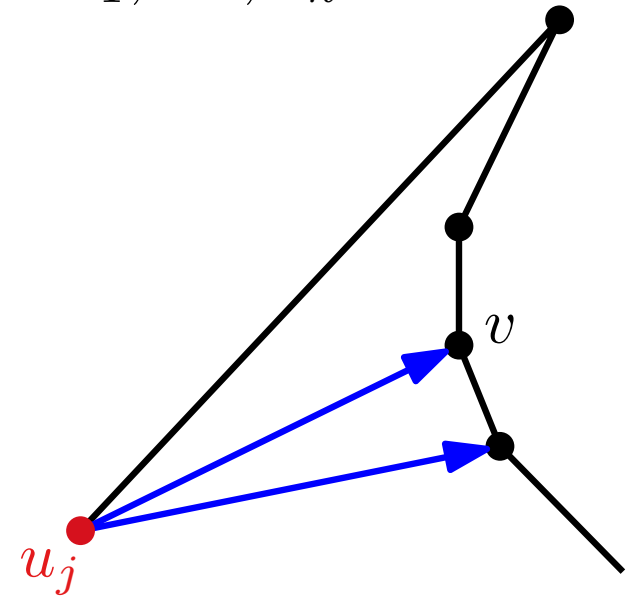
while not $S.\text{empty}()$ **do**

$v \leftarrow S.\text{pop}()$

if not $S.\text{empty}()$ **then** draw (u_j, v)

$S.\text{push}(u_{j-1})$; $S.\text{push}(u_j)$

else



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.\text{push}(u_1)$; $S.\text{push}(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.\text{top}()$ on different paths **then**

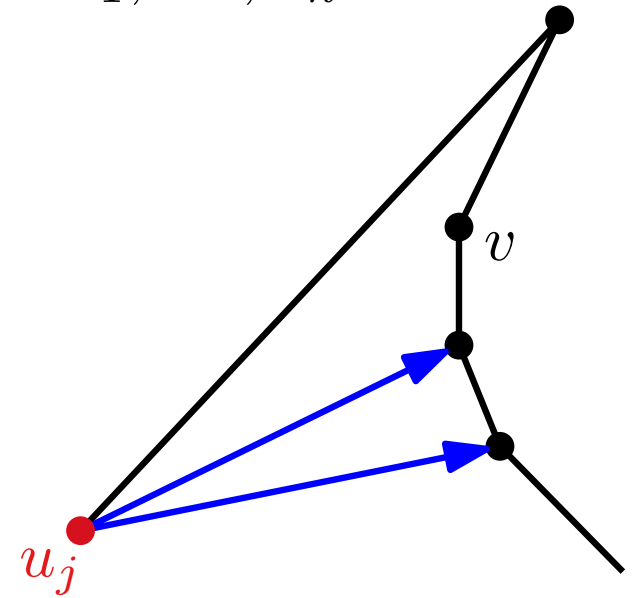
while not $S.\text{empty}()$ **do**

$v \leftarrow S.\text{pop}()$

if not $S.\text{empty}()$ **then** draw (u_j, v)

$S.\text{push}(u_{j-1})$; $S.\text{push}(u_j)$

else



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.\text{push}(u_1)$; $S.\text{push}(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.\text{top}()$ on different paths **then**

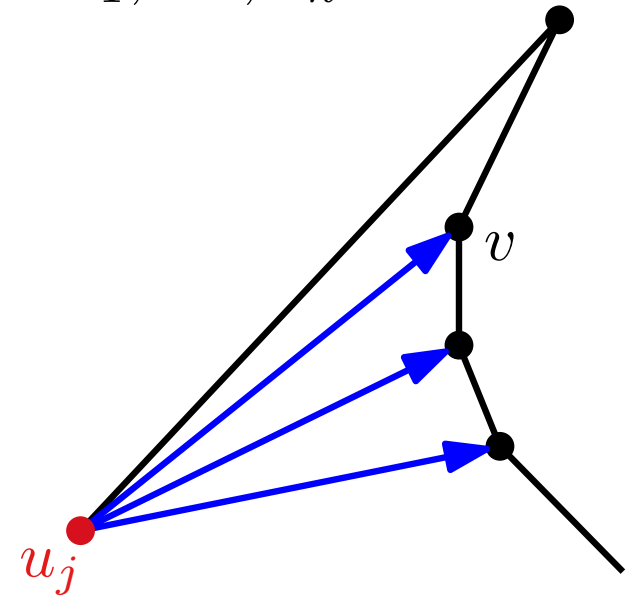
while not $S.\text{empty}()$ **do**

$v \leftarrow S.\text{pop}()$

if not $S.\text{empty}()$ **then** draw (u_j, v)

$S.\text{push}(u_{j-1})$; $S.\text{push}(u_j)$

else



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.\text{push}(u_1)$; $S.\text{push}(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.\text{top}()$ on different paths **then**

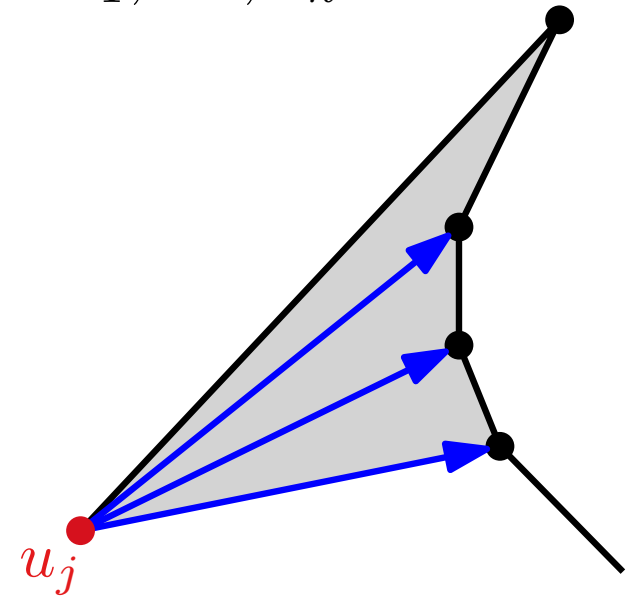
while not $S.\text{empty}()$ **do**

$v \leftarrow S.\text{pop}()$

if not $S.\text{empty}()$ **then** draw (u_j, v)

$S.\text{push}(u_{j-1})$; $S.\text{push}(u_j)$

else



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.\text{push}(u_1)$; $S.\text{push}(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.\text{top}()$ on different paths **then**

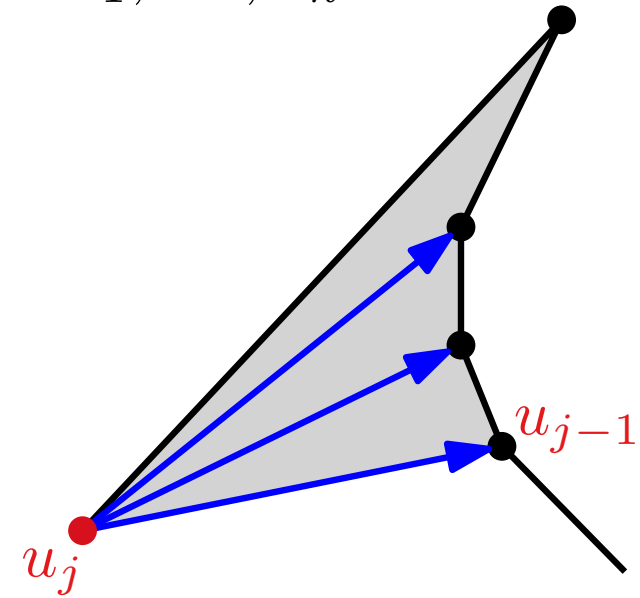
while not $S.\text{empty}()$ **do**

$v \leftarrow S.\text{pop}()$

if not $S.\text{empty}()$ **then** draw (u_j, v)

$S.\text{push}(u_{j-1})$; $S.\text{push}(u_j)$

else



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.push(u_1)$; $S.push(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.top()$ on different paths **then**

while not $S.empty()$ **do**

$v \leftarrow S.pop()$

if not $S.empty()$ **then** draw (u_j, v)

$S.push(u_{j-1})$; $S.push(u_j)$

else

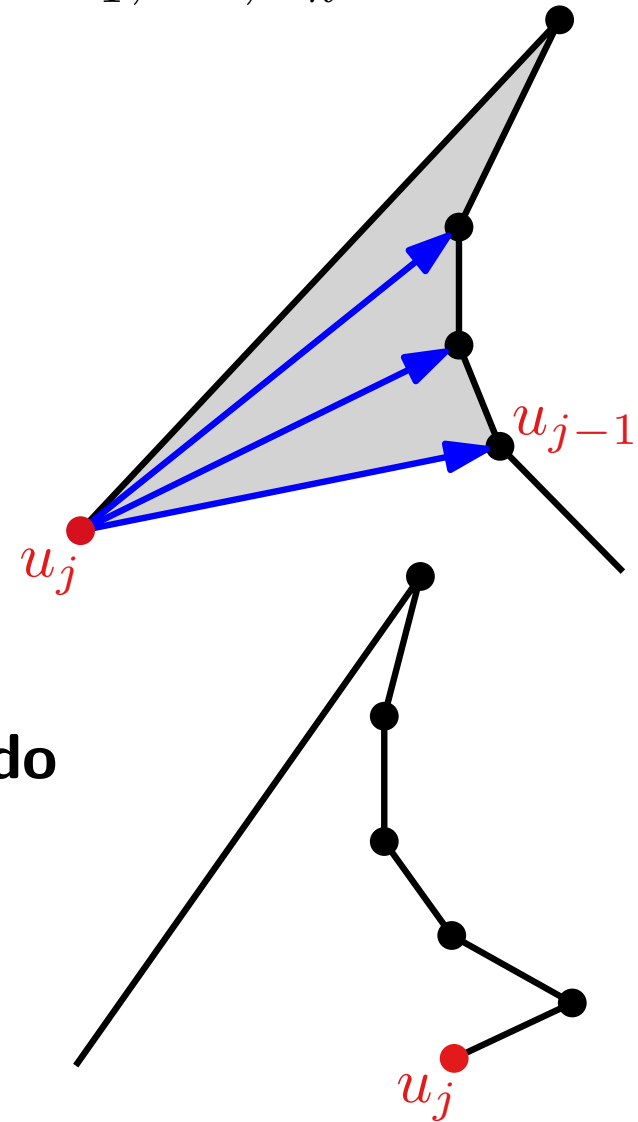
$v \leftarrow S.pop()$

while not $S.empty()$ **and** u_j sees $S.top()$ **do**

$v \leftarrow S.pop()$

 draw diagonal (u_j, v)

$S.push(v)$; $S.push(u_j)$



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.\text{push}(u_1)$; $S.\text{push}(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.\text{top}()$ on different paths **then**

while not $S.\text{empty}()$ **do**

$v \leftarrow S.\text{pop}()$

if not $S.\text{empty}()$ **then** draw (u_j, v)

$S.\text{push}(u_{j-1})$; $S.\text{push}(u_j)$

else

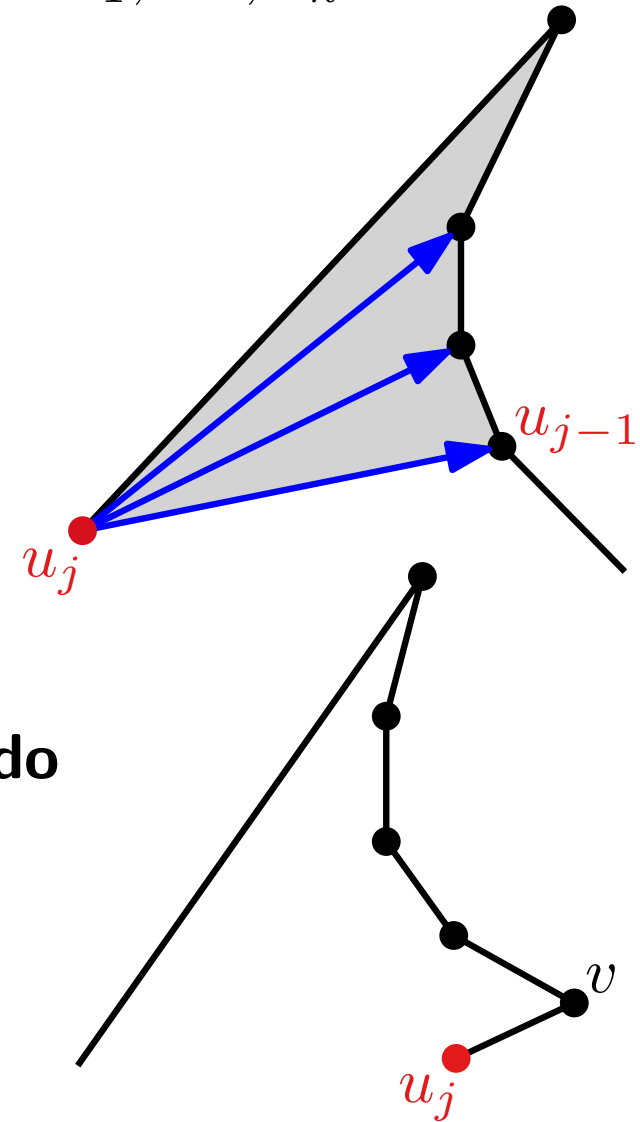
$v \leftarrow S.\text{pop}()$

while not $S.\text{empty}()$ **and** u_j sees $S.\text{top}()$ **do**

$v \leftarrow S.\text{pop}()$

 draw diagonal (u_j, v)

$S.\text{push}(v)$; $S.\text{push}(u_j)$



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.\text{push}(u_1)$; $S.\text{push}(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.\text{top}()$ on different paths **then**

while not $S.\text{empty}()$ **do**

$v \leftarrow S.\text{pop}()$

if not $S.\text{empty}()$ **then** draw (u_j, v)

$S.\text{push}(u_{j-1})$; $S.\text{push}(u_j)$

else

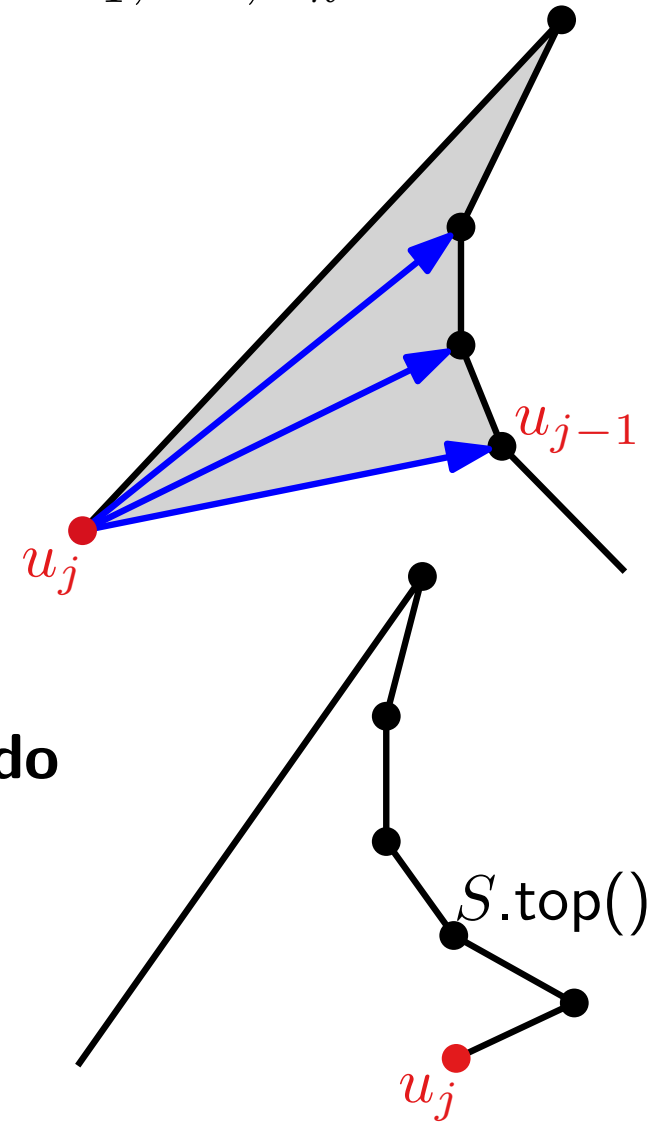
$v \leftarrow S.\text{pop}()$

while not $S.\text{empty}()$ **and** u_j sees $S.\text{top}()$ **do**

$v \leftarrow S.\text{pop}()$

 draw diagonal (u_j, v)

$S.\text{push}(v)$; $S.\text{push}(u_j)$



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.\text{push}(u_1)$; $S.\text{push}(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.\text{top}()$ on different paths **then**

while not $S.\text{empty}()$ **do**

$v \leftarrow S.\text{pop}()$

if not $S.\text{empty}()$ **then** draw (u_j, v)

$S.\text{push}(u_{j-1})$; $S.\text{push}(u_j)$

else

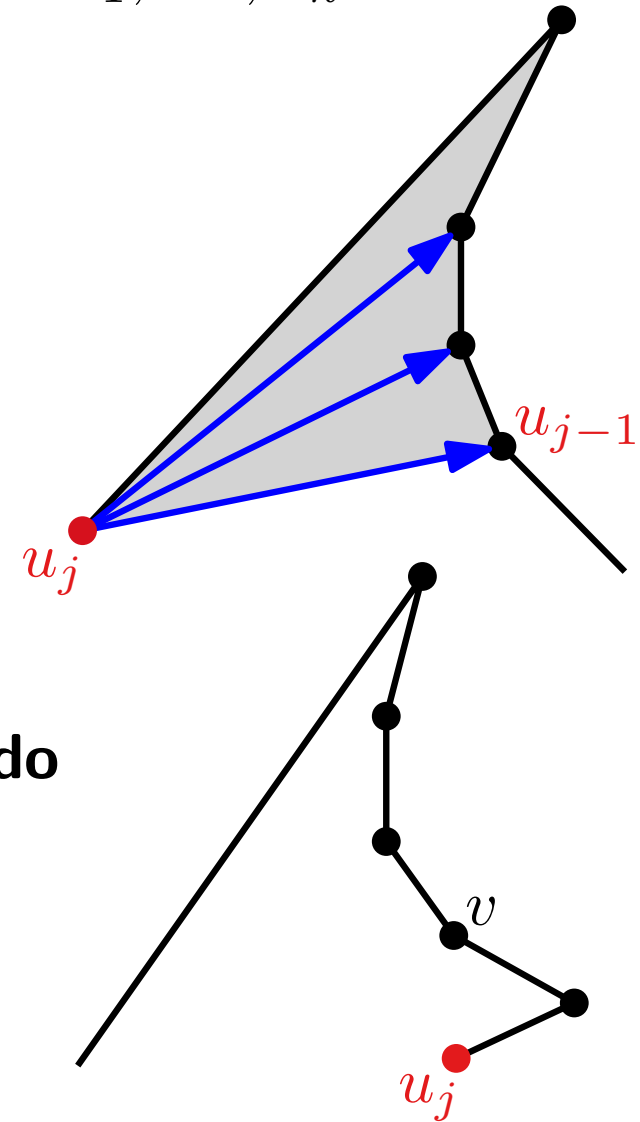
$v \leftarrow S.\text{pop}()$

while not $S.\text{empty}()$ **and** u_j sees $S.\text{top}()$ **do**

$v \leftarrow S.\text{pop}()$

 draw diagonal (u_j, v)

$S.\text{push}(v)$; $S.\text{push}(u_j)$



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.\text{push}(u_1)$; $S.\text{push}(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.\text{top}()$ on different paths **then**

while not $S.\text{empty}()$ **do**

$v \leftarrow S.\text{pop}()$

if not $S.\text{empty}()$ **then** draw (u_j, v)

$S.\text{push}(u_{j-1})$; $S.\text{push}(u_j)$

else

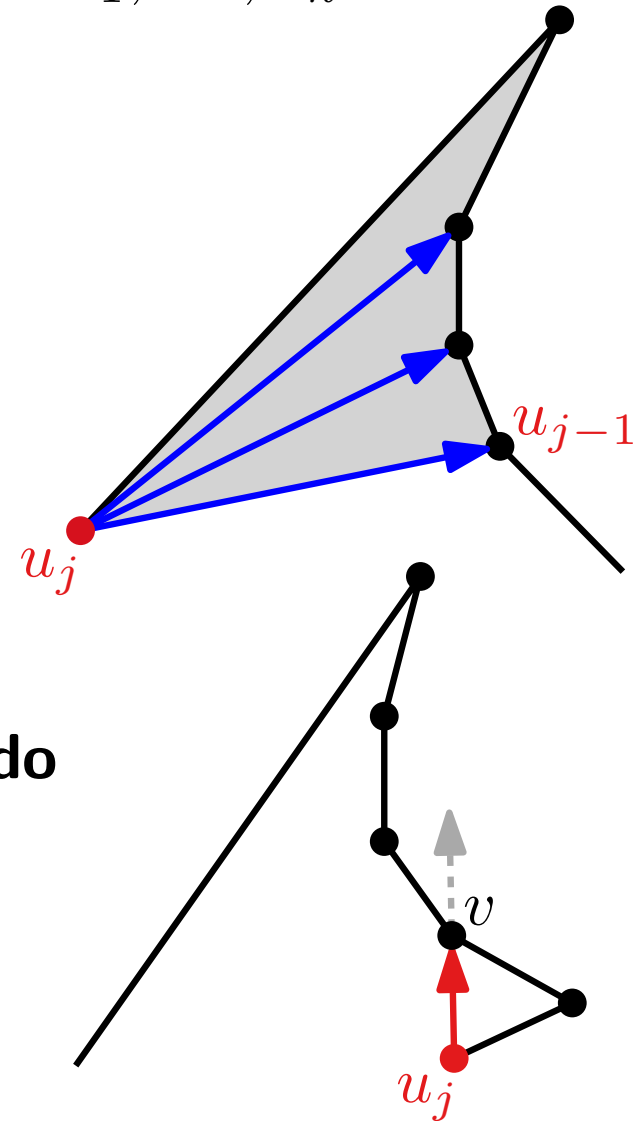
$v \leftarrow S.\text{pop}()$

while not $S.\text{empty}()$ **and** u_j sees $S.\text{top}()$ **do**

$v \leftarrow S.\text{pop}()$

 draw diagonal (u_j, v)

$S.\text{push}(v)$; $S.\text{push}(u_j)$



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.\text{push}(u_1)$; $S.\text{push}(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.\text{top}()$ on different paths **then**

while not $S.\text{empty}()$ **do**

$v \leftarrow S.\text{pop}()$

if not $S.\text{empty}()$ **then** draw (u_j, v)

$S.\text{push}(u_{j-1})$; $S.\text{push}(u_j)$

else

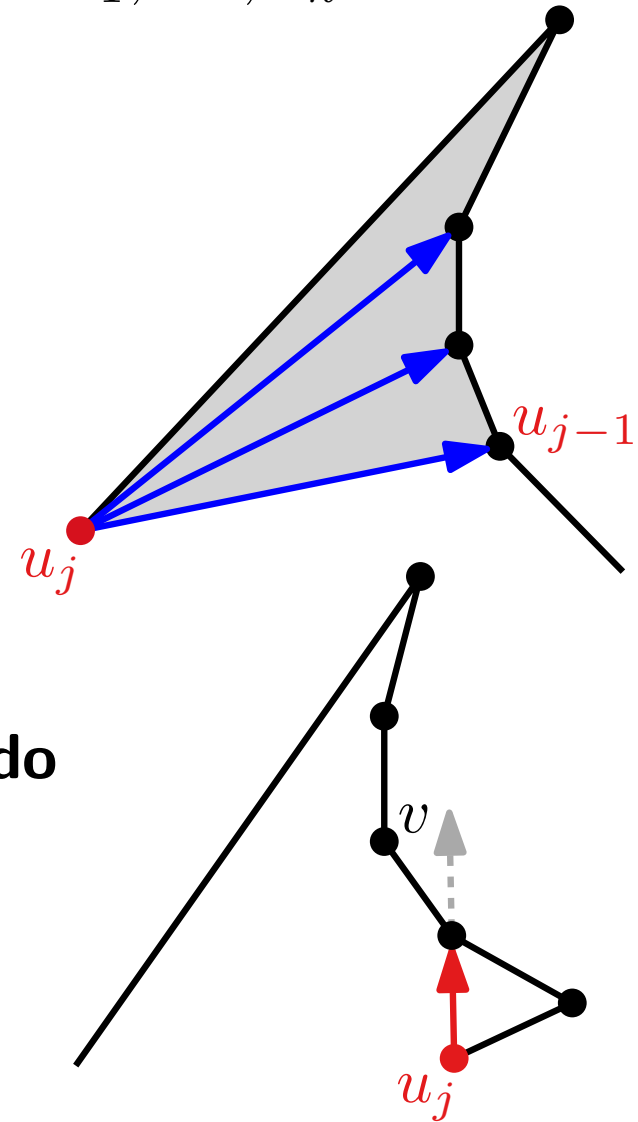
$v \leftarrow S.\text{pop}()$

while not $S.\text{empty}()$ **and** u_j sees $S.\text{top}()$ **do**

$v \leftarrow S.\text{pop}()$

 draw diagonal (u_j, v)

$S.\text{push}(v)$; $S.\text{push}(u_j)$



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.push(u_1)$; $S.push(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.top()$ on different paths **then**

while not $S.empty()$ **do**

$v \leftarrow S.pop()$

if not $S.empty()$ **then** draw (u_j, v)

$S.push(u_{j-1})$; $S.push(u_j)$

else

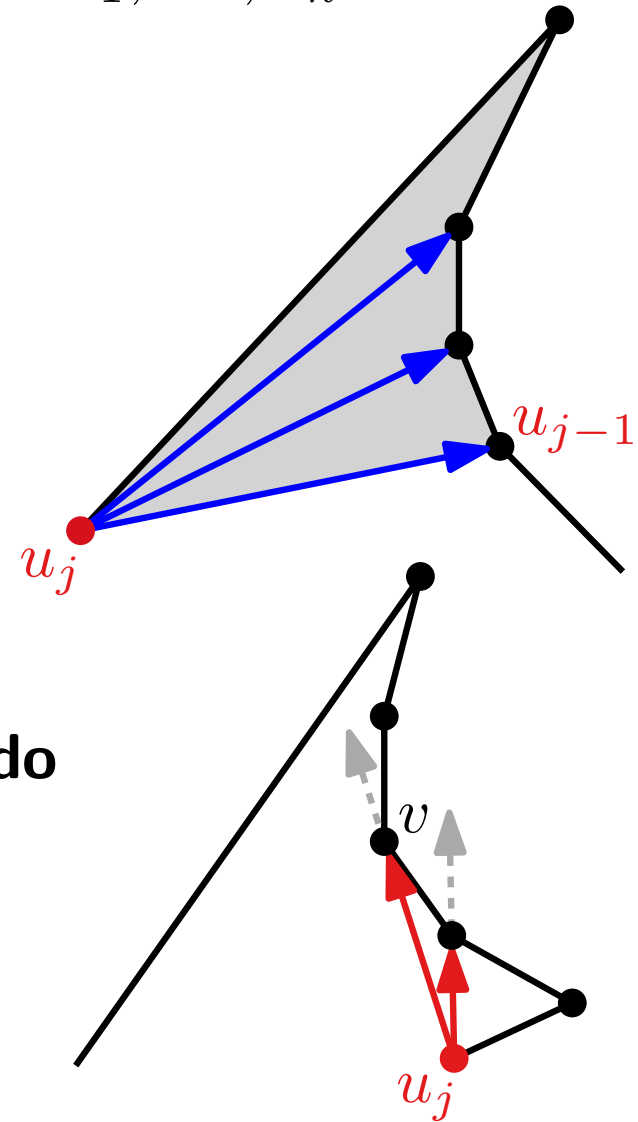
$v \leftarrow S.pop()$

while not $S.empty()$ **and** u_j sees $S.top()$ **do**

$v \leftarrow S.pop()$

 draw diagonal (u_j, v)

$S.push(v)$; $S.push(u_j)$



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.push(u_1)$; $S.push(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.top()$ on different paths **then**

while not $S.empty()$ **do**

$v \leftarrow S.pop()$

if not $S.empty()$ **then** draw (u_j, v)

$S.push(u_{j-1})$; $S.push(u_j)$

else

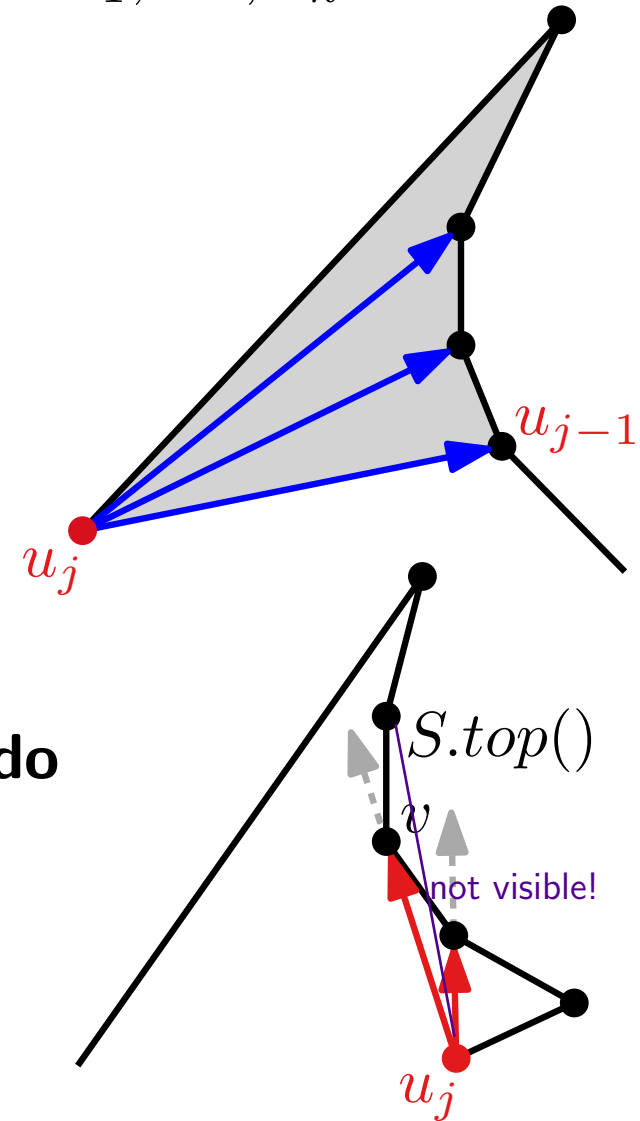
$v \leftarrow S.pop()$

while not $S.empty()$ **and** u_j sees $S.top()$ **do**

$v \leftarrow S.pop()$

 draw diagonal (u_j, v)

$S.push(v)$; $S.push(u_j)$



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.\text{push}(u_1)$; $S.\text{push}(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.\text{top}()$ on different paths **then**

while not $S.\text{empty}()$ **do**

$v \leftarrow S.\text{pop}()$

if not $S.\text{empty}()$ **then** draw (u_j, v)

$S.\text{push}(u_{j-1})$; $S.\text{push}(u_j)$

else

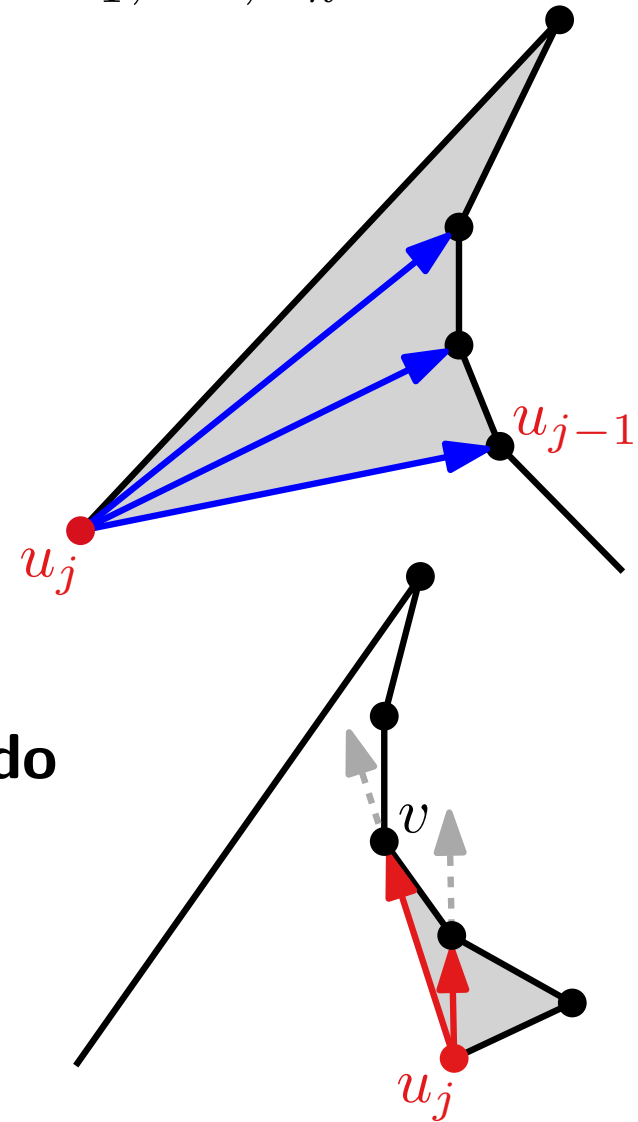
$v \leftarrow S.\text{pop}()$

while not $S.\text{empty}()$ **and** u_j sees $S.\text{top}()$ **do**

$v \leftarrow S.\text{pop}()$

 draw diagonal (u_j, v)

$S.\text{push}(v)$; $S.\text{push}(u_j)$



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.\text{push}(u_1)$; $S.\text{push}(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.\text{top}()$ on different paths **then**

while not $S.\text{empty}()$ **do**

$v \leftarrow S.\text{pop}()$

if not $S.\text{empty}()$ **then** draw (u_j, v)

$S.\text{push}(u_{j-1})$; $S.\text{push}(u_j)$

else

$v \leftarrow S.\text{pop}()$

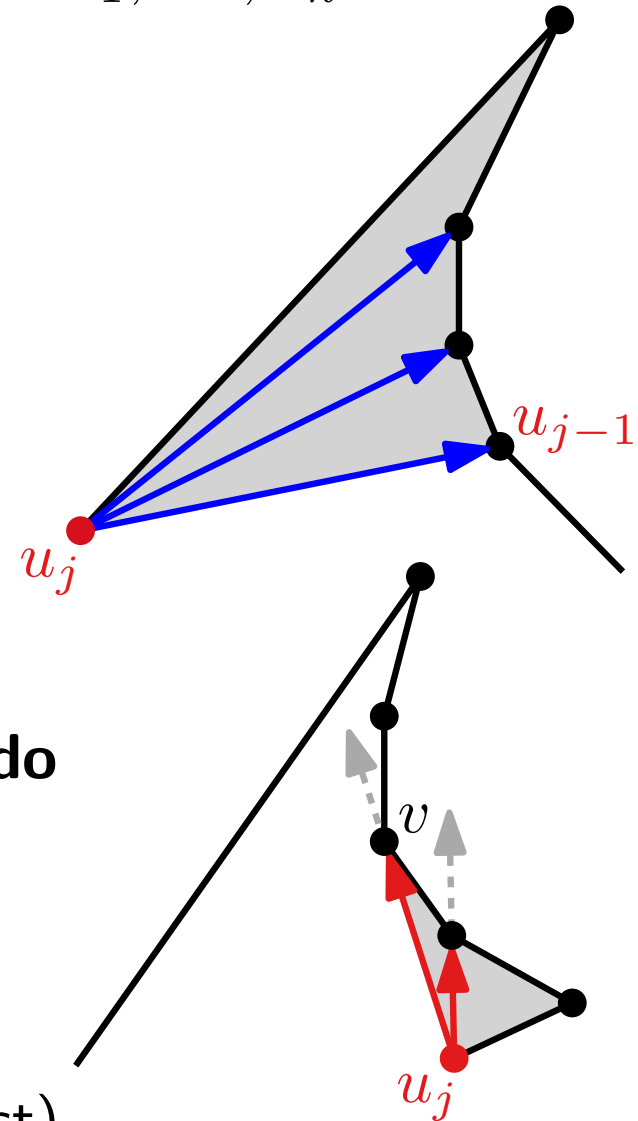
while not $S.\text{empty}()$ **and** u_j sees $S.\text{top}()$ **do**

$v \leftarrow S.\text{pop}()$

 draw diagonal (u_j, v)

$S.\text{push}(v)$; $S.\text{push}(u_j)$

connect u_n to all vertices in S (except first and last)



Algorithm TriangulateMonotonePolygon

TriangulateMonotonePolygon(Polygon P as doubly-connected edge list)

merge left and right paths into descending sequence $u_1, \dots, u_n$

Stack $S \leftarrow \emptyset$; $S.\text{push}(u_1)$; $S.\text{push}(u_2)$

for $j \leftarrow 3$ **to** $n - 1$ **do**

if u_j and $S.\text{top}()$ on different paths **then**

while not $S.\text{empty}()$ **do**

$v \leftarrow S.\text{pop}()$

if not $S.\text{empty}()$ **then** draw (u_j, v)

$S.\text{push}(u_{j-1})$; $S.\text{push}(u_j)$

else

$v \leftarrow S.\text{pop}()$

while not $S.\text{empty}()$ **and** u_j sees $S.\text{top}()$ **do**

$v \leftarrow S.\text{pop}()$

 draw diagonal (u_j, v)

$S.\text{push}(v)$; $S.\text{push}(u_j)$

connect u_n to all vertices in S (except first and last)

Run time - $O(n)!$

Summary

Theorem 4. A y -monotone polygon with n vertices can be triangulated in $O(n)$ time.

Summary

Theorem 4. A y -monotone polygon with n vertices can be triangulated in $O(n)$ time.

Theorem 3. A simple polygon with n vertices can be partitioned into y -monotone polygons in $O(n \log n)$ time and $O(n)$ space.

Summary

Theorem 4. A y -monotone polygon with n vertices can be triangulated in $O(n)$ time.

Theorem 3. A simple polygon with n vertices can be partitioned into y -monotone polygons in $O(n \log n)$ time and $O(n)$ space.



Theorem 5. A simple polygon with n vertices can be triangulated in $O(n \log n)$ time and $O(n)$ space.

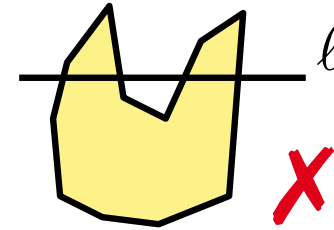
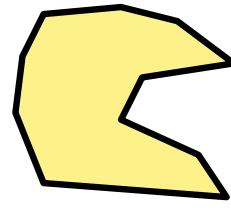
Faster Triangulation Algorithm: Overview

3-step process:

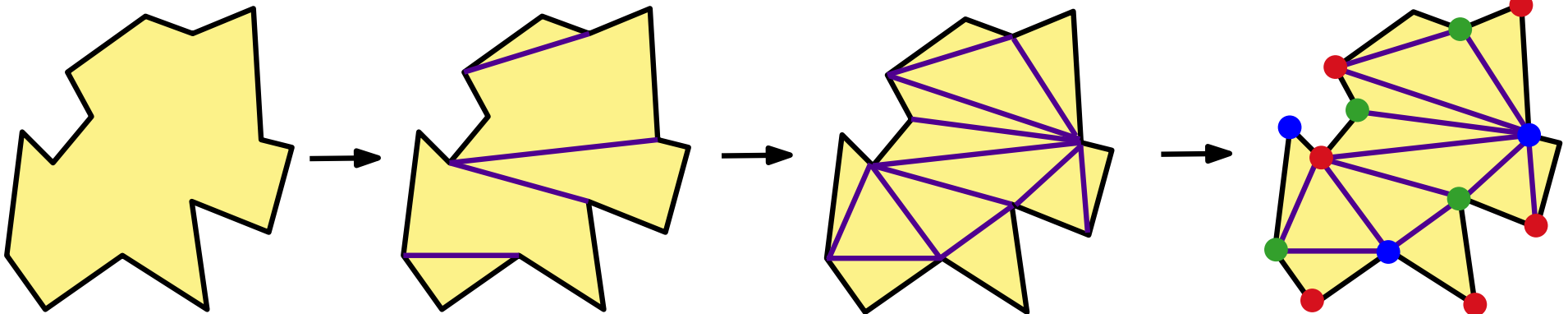
- Step 1: Decompose P into y -monotone polygons

Definition: A polygon is y -monotone, if for any horizontal line ℓ , the intersection $\ell \cap P$ is connected.

two paths from topmost to bottommost vertex never go upward



- Step 2: Triangulate the resulting y -monotone polygons
- Step 3: use DFS to color the vertices of the triangulation ✓



**Thank you
for your attention!**